

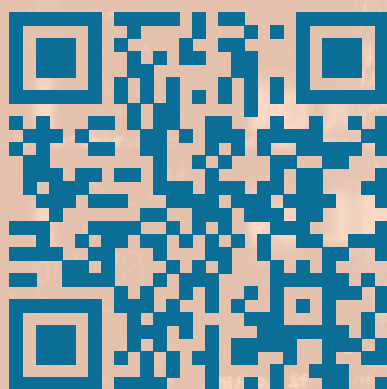
XOI

Xarxes d'Ordinadors i Internet

104353 - Enginyeria de Dades - 1^r curs

Universitat Autònoma de Barcelona

GUIA D'ESTUDI



Per Miguel
Hernández-Cabronero
<miguel.hernandez@uab.cat>

UAB



Aquest document està disponible en: [Castellano](#), [Català](#), [English](#).

Aquest document es va compilar el 22 de setembre de 2026.

Llicència i crèdits

Llicència

Copyright 2024-* © Miguel Hernández-Cabronero <miguel.hernandez@uab.cat>.

Aquest document i els materials que l'acompanyen es poden **distribuir i modificar lliurement** per a usos no comercials, sempre que se'n citi l'autor o autors i es mantinguin els mateixos termes de compartició que els originals (vegeu més avall).

La versió més recent d'aquest document i dels materials que l'acompanyen es pot obtenir a <https://github.com/miguelinux314/uab-xoi>.



Publicat sota la **Llicència Creative Commons Reconeixement-NoComercial-CompartirIgual 4.0** (la "Llicència"). No pots fer servir aquest fitxer si no és d'acord amb la Llicència. Pots obtenir-ne una còpia a <https://creativecommons.org/licenses/by-nc-sa/4.0/>. Llevat que ho exigeixi la legislació aplicable o que s'acordi per escrit, el programari distribuït sota la Llicència es distribueix "TAL COM ÉS", SENSE GARANTIES NI CONDICIONS DE CAP MENA, ni explícites ni implícites. Consulta la Llicència per conèixer els permisos i les limitacions que s'hi estableixen.

Crèdits

L'estructura i els continguts d'aquesta guia es basen en gran mesura en els materials de classe elaborats per Sergi Robles i altres professors del departament DEIC de la UAB. Molts dels continguts visuals els han creat artistes externs i s'han publicat sota llicències compatibles:

- [Mathias Legrand and Vel](#): plantilla base de $\text{\LaTeX}$.
- [TheDigitalArtist/pixabay](#): portada i capçalera del capítol 6.
- [victorsteep/pixabay](#): capçalera del capítol 1.
- [OpenClipart-Vectors/pixabay](#): capçalera del capítol 2.
- [D-Kuru](#), [Niridya](#), [Joe Ravi](#), [Shaddack](#) / Wikimedia Commons (WC) : Ex. 2.6
- [Timwether](#), [Timewalk](#) / WC : Ex. 2.7.
- [deepai](#) : iceberg de la secció 2.2.
- [Chetvorno](#) : diagrama de l'antena de la secció 2.2.
- [Berserkerus](#) : diagrama de modulació de la secció 2.2.
- [Graham Rhind](#) : capçalera del capítol 3.
- [mrcolo](#) : capçalera del capítol 4.
- [Geek2003](#) : switch (commutador) de la secció 3.1.
- [ignartonosbg](#) : capçalera del capítol 5.
- [Toffelginkgo](#) : capçalera del capítol 7.
- [Doodles43](#) : capçalera del capítol 8.
- [Bru-no](#) : ports del router (encaminador) de la secció 8.
- [PublicDomainPictures](#) : capçalera del capítol 9.
- [joo213](#) : capçalera del capítol 10.
- [Frank_Rietsch](#) : capçalera de la introducció a les missions.
- [succo](#) : icona de la missió 1 (capítol 11.1).
- [Pixabay](#) : icona de la missió 2 (capítol 11.2).
- [mmisof](#) : icona de la missió 4 (capítol 11.4).
- [DeepAI](#) : icona de la missió 5 (capítol 11.5).
- [1zoom.me](#) : icona de la missió 6 (capítol 11.6).
- [DeepAI](#) : icona de la missió 7 (capítol 11.7).
- [StockSnap](#) : icona de la missió 9 (capítol 11.9).
- [jarmoluk](#) : capçalera de l'índex de conceptes.

Ús de la IA

Ús recomanat de la IA

La IA és una eina excitant però, a dia d'avui (curs 2026-27), té dificultats per respondre correctament moltes de les preguntes importants del curs. En altres paraules, *no sempre te'n pots confiar*.

Desaconsellem completament l'ús de la IA com a font de solucions a definicions, exercicis, missions i pràctiques.

Ús de la IA per crear aquesta guia

En aquesta guia s'ha fet servir la IA per:

- **Creació d'imatges:** il·lustracions decoratives (no els diagrames amb contingut).
- **Creació de codi:** part del programari que genera la guia (lloc web i PDF).
- **Revisió:** ortografia i gramàtica del text.



Índex

1	Intro: Xarxes d'Ordinadors i Internet (XOI)	6
----------	--	----------

I

Xarxes d'ordinadors

2	Travessant el vel analògic-digital	9
2.1	Informació ↔ Dades ↔ Missatges binaris	9
2.2	Analògic ↔ Digital	11
3	On ets?	13
3.1	Si us plau, contacta'm	13
3.2	Necessito trobar-te	14
3.3	Com hi arribo?	15
4	Això és teu?	17
4.1	Tinc una entrega per a tu	17
4.2	Si us plau, omple el formulari	18
4.3	No paren d'arribar	20
5	Una pila de capes	21
5.1	Enviament de dades	21
5.2	Recepció de dades	22

II

Internet

6	Internet	26
6.1	La pila TCP/IP	26
6.2	Visió general dels protocols TCP/IP	27
6.3	Qui mana a Internet?	28
7	Capa 2: Comunicació de xarxa (LAN)	29
7.1	Adreçament de capa 2	30
7.2	Protocol de capa 2 Ethernet	30
8	Capa 3: comunicació entre xarxes (Internet)	33
8.1	Adreçament de la capa 3	34
8.2	IP – Internet Protocol	37
8.3	Encaminament IP	39
8.4	Divisió en subxarxes IP	41
8.5	Fragmentació IP	43

8.6	ARP – Address Resolution Protocol	44
8.7	ICMP – Internet Control Message Protocol	46
9	Capa 4: Transport	48
9.1	Adreçament de la capa 4	49
9.2	UDP – User Datagram Protocol	49
9.3	TCP – Transport Control Protocol	51
9.4	TCP: “Connectant”	52
9.5	TCP: “Connexió establerta”	53
9.6	TCP: “Tancant la connexió”	56
10	Capa 7: Aplicació	60
10.1	DNS – Sistema de noms de domini	60
10.2	DHCP – Protocol de configuració dinàmica de hosts	62
10.3	Sistemes autònoms	64

III

Missions

11	Missions	67
11.1	Missió 1: Compartir és estimar	68
11.2	Missió 2: Pergamí infinit	69
11.3	Missió 3: Pasturant el caos	70
11.4	Missió 4: Inter { planetary net	71
11.5	Missió 5: Els diners són l'arrel	72
11.6	Missió 6: Empipa dos planetes sencers	73
11.7	Missió 7: TCP o no TCP	74
11.8	Missió 8: Que em parles a mi?	75
11.9	Missió 9: Vestíbul interplanetari	76

1. Intro: Xarxes d'Ordinadors i Internet (XOI)

Què hem pagat

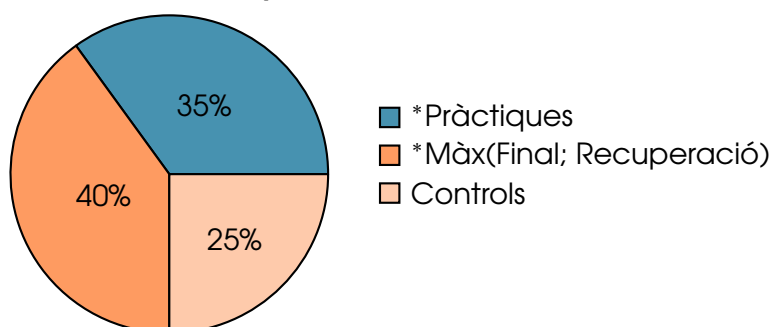
	Dilluns	Dimarts	Dimecres	Dijous	Divendres
8:00					
8:30					
9:00				Problemes PAUL/811	Pràctiques PLAB/812
9:30					
10:00				Pràctiques PLAB/813	Pràctiques PLAB/814
10:30					
11:00				Pràctiques PLAB/815	Problemes PAUL/812
11:30					
12:00				Pràctiques PLAB/811	Problemes PAUL/813
12:30					
13:00		Teoria Classe completa			
13:30					
14:00					
14:30					
15:00					

- 13x Sessions de **teoria** de 2h
- 12x Sessions de **problemes** d'1h
- 12x Sessions de **laboratori** d'1h
- Il·limitades Sessions de **tutoria** (tutories), individuals o en grup: envia un correu
- 90h **Treball autònom** (el teu)
- 3x **Proves de control**
- 1x **Examen final** + revisió
- 1x **Recuperació** de l'**examen final** + revisió
- 3x **Lliuraments** de **laboratori** amb retroacció
- 1x **Examen** de **laboratori** + revisió

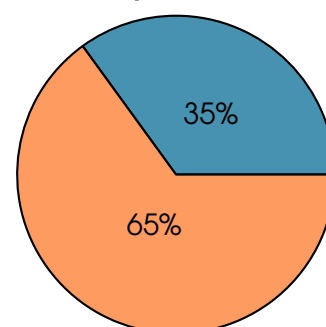
Avaluació

Per determinar la teva nota s'utilitzarà la millor de les dues opcions següents:

Opció A



Opció B



*: Cal una nota mínima en aquesta part per aprovar l'assignatura. Tots els detalls a la [guia docent de l'assignatura](#).



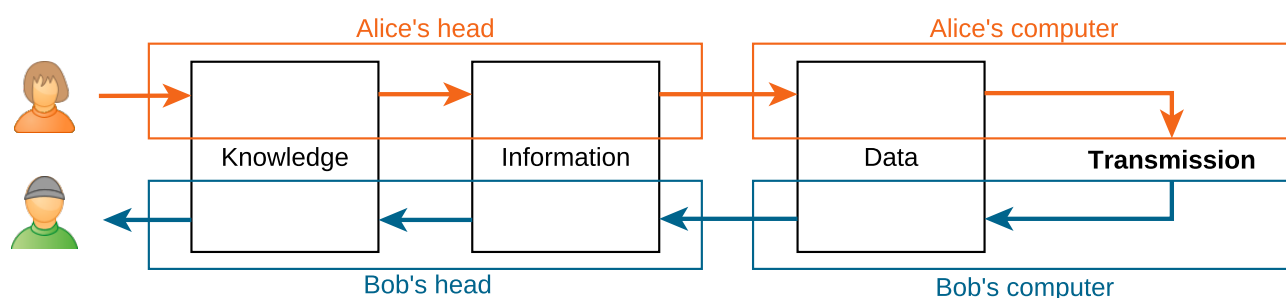
Xarxes d'ordinadors

Com intercanvien dades els ordinadors? Com podem localitzar altres màquines situades arreu del món i comunicar-nos-hi? Aquesta part de la guia explora alguns dels principals reptes que hem d'afrontar si volem interconnectar dispositius a escala planetària, i més enllà.

2	Travessant el vel analògic-digital	9
2.1	Informació ↔ Dades ↔ Missatges binaris	
2.2	Analògic ↔ Digital	
3	On ets?	13
3.1	Si us plau, contacta'm	
3.2	Necessito trobar-te	
3.3	Com hi arribo?	
4	Això és teu?	17
4.1	Tinc una entrega per a tu	
4.2	Si us plau, emplena el formulari	
4.3	No paren d'arribar	
5	Una pila de capes	21
5.1	Enviament de dades	
5.2	Recepció de dades	

2. Travessant el vel analògic-digital

Enviem i rebem missatges constantment en els nostres àmbits personal, professional i polític. La disponibilitat gairebé instantània de pràcticament qualsevol informació desitjada és una característica clau de l'era actual. Quan fem servir un ordinador de qualsevol mena per compartir o obtenir coneixement, primer cal transformar-lo en dades que puguin entendre tant els humans com els ordinadors. Això es pot imaginar com una seqüència de passos:



Els ordinadors treballen amb dades digitals i les transmeten, és a dir, seqüències binàries de 0 i 1 com a símbols. Tanmateix, la transmissió de dades té lloc en el món real, on 0 i 1 són idees i no objectes tangibles.

Aquest capítol estudia com aquests símbols digitals *travessen el vel* del món digital al món analògic (físic) en la transmissió, i de tornada de l'analògic al digital en la recepció.

2.1 Informació ↔ Dades ↔ Missatges binaris

El coneixement, la informació i les dades no són el mateix, tot i que sovint s'utilitzen indistintament. En contextos tècnics, cal utilitzar-los i interpretar-los amb precisió.

Exercici 2.1 Suposem que som a l'aire lliure i volem explicar al nostre amic quin temps fa. Tenim coneixement de tot el que està relacionat amb el temps, perquè som allà. Per a la nostra comunicació, ens hem de centrar en una part d'aquest coneixement i decidir quina informació enviem. Per exemple, podríem voler comunicar la temperatura i dir alguna cosa com "la temperatura és d'uns 22°C". Hi ha moltes maneres d'emmagatzemar aquesta informació com a dades dins d'un ordinador. El més probable és que representem el nombre 22 com a part d'aquestes dades. Com definiríes coneixement, informació i dades?

Tot i que hi ha molts tipus de dades, és a dir numèriques, textuais, visuals, científiques, *etc.*, al final totes es representen amb nombres. Per exemple, es podria fer servir la codificació ASCII per representar la cadena "Bobby" com la seqüència següent: 66,111,98,98,121.

Dins d'un ordinador, aquests nombres s'emmagatzemen en registres binaris, p. ex. a la CPU i a la RAM. Tanmateix, hi ha diverses maneres de representar dades numèriques en format binari. Més concretament, hem de parar molta atenció, com a mínim, als aspectes següents sempre que llegim o escrivim dades:

- enter o amb decimals?
- profunditat de bits? (p. ex., 8 bits per nombre)
- amb signe o sense signe?
- Per a nombres de diversos bytes: big endian o little endian?
- Per a nombres amb decimals: coma flotant o coma fixa?

Exercici 2.2 Explica **com completar la segona fila a partir de la primera** (i viceversa), i quines suposicions has de fer en cada cas.

Nombre	7	2	-3	0.75	260
Binari	00111	00000010	1101	1100	0000 0100 0000 0001

I què passa amb $\leftrightarrow$ 1100 0011 1010 0000?

De vegades hem d'inspeccionar el contingut d'un registre binari (p. ex., per depurar o amb altres finalitats d'anàlisi) o establir-ne el contingut manualment. Per als humans, és incòmode i *molt* propens a errors manipular cadenes binàries de més d'uns quants bits. Quan hem d'inspeccionar o modificar el contingut d'un registre binari (p. ex., per depurar), sovint fem servir la base 16, és a dir, la notació hexadecimal.

En hexadecimal, hi ha exactament $16 = 2^4$ dígits diferents (de 0 a 9, de a a f) amb valors decimals de 0 a 15, cadascun dels quals representa exactament 4 bits. Quan escrivim valors hexadecimal per a un ordinador, és a dir, en codi font, els valors hexadecimal normalment van precedits de 0x, p. ex., 0x58a5b0. De manera semblant, les expressions binàries van precedides de 0b, p. ex., 0b0011.



- Els espais i fins i tot els salts de línia entre dígits no tenen cap significat; només s'utilitzen per facilitar la inspecció visual.
- En alguns textos, quan en un context hi ha diverses bases possibles, s'indiquen com a subíndexs, com a 58a5b0₁₆ i 0011₂.

Exercici 2.3 Considera el missatge següent, format per una concatenació d'enters sense signe de 3 bits: a4 3f 20.

- Quants bits i quants bytes fa?
- Quins són els cinc primers enters** continguts en el missatge?

La majoria de vegades, deixarem que el nostre codi s'encarregui de manipular les dades. Per fer-ho, necessitem operacions a nivell de bits i de lògica booleana com les següents

Operació	AND (bits)	AND lògica	OR (bits)	OR lògica	Despl. esq.	Despl. dre.
Python	&	and		or	<<	>>

Altres eines útils de Python són les funcions `bin()` i `hex()`, que converteixen un enter en la seva representació binària i hexadecimal, respectivament, i `int()`, que pot analitzar una cadena que descriu un nombre i retornar aquest nombre. També podem controlar el format amb què mostrem els nombres, p. ex., `print(f'{n:08b}')` mostraria el valor de la variable `n` en forma binària, fent servir com a mínim 8 posicions i omplint amb zeros les posicions buides de l'esquerra (més informació a la [documentació de Python](#)).

Exercici 2.4 Considera el codi que es mostra a continuació. Els nombres que s'imprimeixen en executar-lo tenen una importància especial en xarxes i en informàtica. Què tenen en comú? (Pista: potser et convé modificar el codi perquè en mostri les representacions binàries).

```

.py .out snippets/bitwisemanipulation.py
1 a = 0xff
2 print(a)
3 for _ in range(8):
4     a = (a << 1) & 0xff
5     print(a)

```

2.2 Analògic ↔ Digital

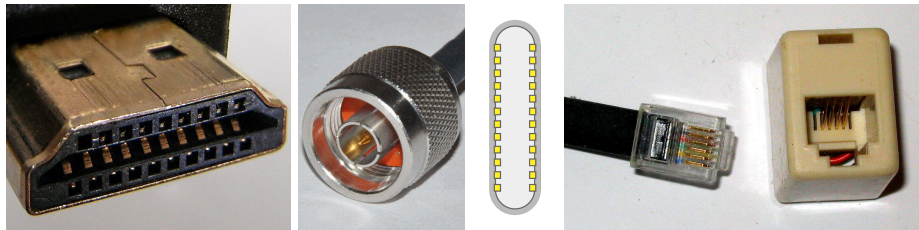
Un cop decidim quins bits volem transmetre, hem de trobar una manera de fer que aquests bits arribin a una altra màquina a distància. Primer, hem de decidir quin mitjà fem servir i quines propietats físiques podem controlar i monitorar. Entre les opcions més habituals hi ha fer servir el voltatge en cables de coure, enviar llum per fibra òptica i manipular el camp electromagnètic mitjançant antenes.

Exercici 2.5 Reflexiona:

- Podem fer un cable de coure tan llarg com vulguem?
- Quin mitjà fan servir les connexions Wi-Fi?
- Necessitem fibres òptiques per fer comunicacions basades en la llum?

Si optem per cables de coure, en podem posar diversos en paral·lel i enviar més dades a la mateixa velocitat de relloige. Això, però, té el preu d'una complexitat i un cost més grans que els dissenys en sèrie. Es poden produir interferències de voltatge a les línies de dades per diversos motius, incloent-hi fenòmens físics com la inducció electromagnètica al cable. Tot i així, els cables que compleixen els estàndards moderns de transmissió de dades (p. ex., IEEE 802.3) sovint fan servir parells trenats de cables i apantallament, entre altres estratègies, per garantir errors de bit per sota d'1 de cada 10^{12} bits.

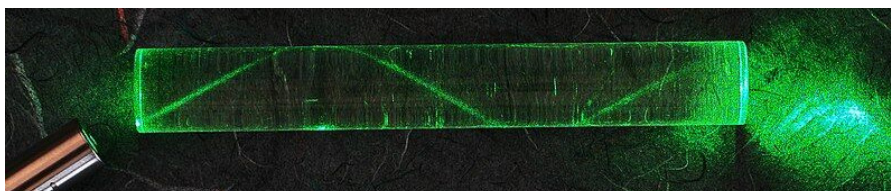
Exercici 2.6 Observa i identifica els connectors de cable següents:



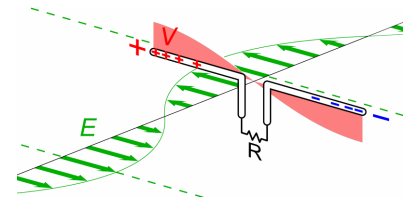
Alguna vegada t'has capbussat en una piscina, has mirat cap amunt i l'aigua semblava un mirall? Els cables de fibra òptica funcionen gràcies a aquest fenomen, anomenat refracció. La llum que entra a la fibra per un extrem s'hi queda a dins fins que surt per l'altre extrem. El recobriment de la fibra és important per preservar-ne la integritat, però aquest recobriment no participa en el "rebot" de la llum mentre avança per la fibra.

En una fibra òptica monomode, el sensor de l'extrem receptor detecta la presència o l'absència d'un únic rang de longituds d'ona. Aquests cables són relativament prims i barats, però transporten un sol senyal. Una altra opció és transmetre diverses longituds d'ona (p. ex., fent servir diverses fonts làser) alhora, per la mateixa fibra. A l'extrem receptor, es fa servir un prisma per tornar a dividir el feix de llum en els seus components monocromàtics, que es detecten per separat. D'aquesta manera, la fibra òptica multimode permet multiplexar diversos senyals, cosa que augmenta molt la taxa de transmissió efectiva.

Exercici 2.7 Quan un feix de llum entra en una fibra òptica, ho fa amb un cert angle d'incidència. Aquest angle afecta el temps que triga a arribar a l'altre extrem. Si projectem un con de llum, la llum entra a la fibra amb diversos angles d'incidència. Què passa a l'altre extrem si **encenem i apaguem aquest con de llum** molt ràpidament? És rellevant si fem servir feixos monomode o multimode?



Tot i que hi ha moltes classes d'antenes, la seva finalitat principal és detectar camps electromagnètics i/o crear-ne. L'equació de Maxwell-Faraday ens diu que els canvis en el camp elèctric $\vec{E}$ creen un camp magnètic perpendicular $\vec{B}$, i a l'inrevés.



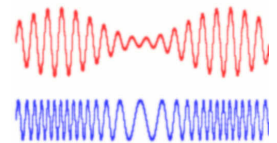
Una antena de dipol com la de la dreta aprofita aquest concepte aplicant un voltatge altern V , que genera ones electromagnètiques que es propaguen cap a fora de l'eix del dipol.



Quan emetem una certa energia E des d'un punt en totes direccions, aquesta energia es distribueix al voltant d'una esfera creixent. Com que l'àrea d'una esfera de radi r és $4\pi r^2$, la quantitat d'energia que arriba a un punt a distància r serà de l'ordre de magnitud de E/r^2 .

Això de vegades s'anomena llei de l'invers del quadrat, i és el motiu pel qual els corrents induïts a l'antena del receptor solen estar entre els nanovolts (10^{-9} V) i els picovolts (10^{-12} V). Sorprenentment, això és suficient perquè els detectors moderns lleixin el senyal de dades.

Independentment del mètode triat perquè les dades viatgin, cal travessar igualment el vel entre els senyals analògics i les dades digitals. Una manera de fer-ho és la modulació: es genera un senyal sinusoidal de base anomenat portadora, i les dades es codifiquen modificant aquesta portadora. Per exemple, es pot canviar l'amplitud de la portadora (ASK), la seva freqüència (FSK), o fins i tot l'amplitud i la fase alhora (QAM).



Exercici 2.8 La figura anterior mostra un exemple de modulació d'amplitud i un de modulació de freqüència. Quin és quin? Per a cada cas: **aquesta modulació transmet un senyal analògic o digital?**

Exercici 2.9 Quina és la **diferència entre amplada de banda i velocitat de transmissió**? Per què creus que s'utilitzen indistintament en contextos no tècnics?

3. On ets?

Connectar dos ordinadors els fa molt més útils que dues màquines aïllades. Tanmateix, el seu veritable potencial només es desbloqueja quan es poden connectar a *moltíssims* altres ordinadors.

Limitacions pràctiques com el cost i la ubicació geogràfica fan que sigui impossible connectar directament cada parell d'ordinadors. En lloc d'això, els dispositius es distribueixen en un graf de xarxes interconnectades, que es pot estendre fins i tot més enllà de l'escala d'un planeta.

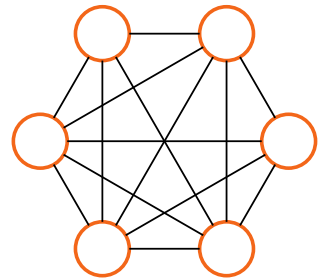
Tant si és localment, a l'abast físic, com en un altre continent, cal un sistema d'adreçament per identificar, localitzar i encaminar els missatges perquè arribin als peers (parells) previstos, tal com fem amb les ubicacions físiques.

3.1 Si us plau, contacta'm

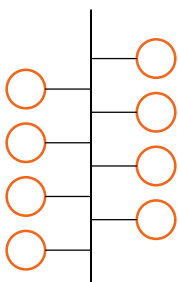
Quan només hi intervenen 2 dispositius, es pot fer servir una connexió punt a punt. En aquest tipus de connexió, només hi ha "aquest costat" i "l'altre costat" del cable. A més, quan un dispositiu vol comunicar-se amb l'altre extrem, només hi ha un enllaç (p. ex., un cable) on triar, de manera que no hi ha cap confusió possible. Com que podem fer funcionar aquesta connexió amb només 2 targetes de xarxa i 1 cable, les connexions punt a punt són viables per a $N = 2$ dispositius.

Si volem connectar $N > 2$ dispositius, una opció ingènua és connectar cada dispositiu amb tots els altres en un graf complet. Ara cada dispositiu ha de gestionar $N - 1$ cables cap als altres dispositius, però comunicar-se amb qualsevol altre dispositiu és tan fàcil com triar el cable correcte i establir una connexió punt a punt.

El problema principal d'aquest enfocament és que el nombre total de connexions és $N \cdot (N - 1) / 2 = O(N^2)$ (en saber més al teu estimat curs de *Matemàtica Discreta*). Això vol dir que, per tenir $N = 10$ dispositius connectats d'aquesta manera, necessaries per exemple 40 cables i 90 targetes de xarxa. Per a $N = 1000$ dispositius, hi hauria mig milió de connexions punt a punt, cosa que ja és força ingestible.



$O(N^2)$ és la notació d'una fita de complexitat quadràtica. No és el mateix que exponencial i no s'han de confondre mai. ([llegir-ne més...](#))



Tenim un truc amagat a la màniga: els busos de dades, en què una o més línies de dades estan connectades simultàniament a diversos dispositius. Això conserva la simplicitat de les connexions punt a punt, perquè cada dispositiu només ha de gestionar 1 cable i el pot fer servir per comunicar-se amb tots els altres dispositius. També es conserva la rendibilitat, ja que només calen N targetes de xarxa per a N dispositius ($O(N)$).

El principal avantatge dels busos de dades és també la seva principal debilitat: tots els dispositius envien i reben dades per la mateixa línia de dades, però no ho poden fer alhora perquè només hi ha una línia. Si dos o més ho fan, es produeix una col·lisió que fa impossible la transmissió de dades mentre dura.



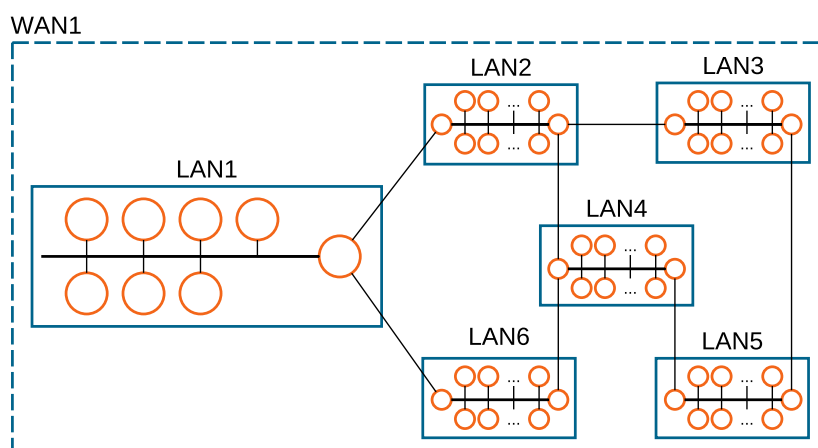
A mesura que es desenvolupaven les xarxes d'ordinadors, es van provar moltes alternatives a la topologia de bus. Un exemple curiós és Token_Ring, en què els dispositius estan connectats al voltant d'un cercle i els missatges hi circulen en un sol sentit.

A la pràctica, no és trivial implementar una tecnologia de bus on es puguin afegir nous dispositius amb el temps. Històricament, de vegades es feia perforant físicament el cable principal del bus i afegint un cable nou cap a l'ordinador que s'hi connectava. Més endavant, això es va simplificar amb uns dispositius anomenats hubs (concentradors). Funcionaven de manera semblant, però oferien ports ja integrats i connectors extraïbles. Avui dia, els hubs han estat substituïts pels switches (commutadors). Els switches ofereixen la funcionalitat d'un bus, connectant cada dispositiu amb tots els altres, i a més eviten les col·lisions. Per raons òbvies, estructurar les connexions al voltant d'un switch s'anomena topologia en estrella.

Exercici 3.1 Què cal en un switch que no calgui en un hub?



Les topologies de bus funcionen molt bé a escala local i s'utilitzen àmpliament en xarxes d'àrea local (LANs). Tanmateix, no són rendibles (de vegades ni tan sols físicament possibles) quan els dispositius són lluny els uns dels altres. En lloc d'això, els dispositius es poden organitzar en LANs separades, cadascuna conceptualment un bus, i després aquestes LANs separades es poden connectar mitjançant connexions punt a punt més barates i factibles. Considerats en conjunt, aquests dispositius formarien llavors una xarxa d'àrea extensa (WAN).



3.2 Necessito trobar-te

Un cop més, el principal avantatge dels busos de dades és també la seva principal debilitat: tots els dispositius envien i reben dades per la mateixa línia de dades. Encara que no hi hagi col·lisions, cada missatge enviat al bus el reben tots els dispositius. Tanmateix, el que realment volem és que el nostre missatge arribi al seu destí, i només al seu destí. Anomenem-ho el *problema de trobar el meu dispositiu*.

El problema de trobar el meu dispositiu no és exclusiu dels busos o de les LANs. Al contrari, esdevé molt important quan considerem les WANs, on no tots els dispositius estan interconnectats directament. En aquest cas, hem d'assegurar-nos que podem encaminar els nostres missatges entre diferents LANs i que arriben al seu destí.

Sorprenentment, ni tan sols les connexions punt a punt estan a resguard del problema de trobar el meu dispositiu. Suposant que els dispositius A i B estan connectats d'aquesta manera, dos programes diferents poden voler intercanviar informació al mateix temps, p. ex.,

un programari de control d'alarmes i una aplicació de reproducció de música en temps real. En aquest escenari, vols que el client i el servidor del servei de monitoratge de l'alarma es comuniquin entre ells, però no amb el servei de música en temps real, i viceversa.

La solució més habitual a aquest problema és fer servir identificadors que ens permetin diferenciar entre dispositius o fins i tot entre processos en execució dins d'un sistema operatiu (SO). Alguns d'aquests identificadors s'anomenen adreces, precisament perquè s'utilitzen per trobar i arribar a un dispositiu remot.

Les adreces solen ser identificadors *numèrics*, és a dir, només un nombre. Com a tals, es poden expressar en bases diferents, incloent-hi la decimal i l'hexadecimal. Aquests nombres s'extreuen d'un conjunt predefinit, i l'elecció d'aquest conjunt determina el nombre màxim d'elements que podem identificar de manera única.



Les adreces expressades com `d8:43:ae:61:ed:f1` o `192.168.1.1` (com es veurà més endavant al curs) també són nombres que hauríem pogut expressar com `237785200061937` i `3232235777`, respectivament (cosa que no és tan pràctica).

Exercici 3.2 Quants elements es poden identificar de manera única (com a màxim) amb una adreça que expresseu amb 5 bits? I amb 10? I amb 20? Expressa la solució general matemàticament.

Exercici 3.3 Suposem que hi ha 1000 màquines en una LAN. Quants bits d'adreça calen? I per a 800 màquines? I per a 1100? Expressa la solució general matemàticament.



El 2019 ens vam quedar sense adreces d'Internet (v4). Ups!

3.3 Com hi arribo?

En molts escenaris, calen diverses adreces per dur a terme la comunicació extrem a extrem desitjada. Per exemple, pot ser que hàgim d'identificar un únic dispositiu dins d'una LAN, i aquesta LAN dins d'una WAN. De vegades, caldrà identificar un procés (un programa en execució) dins d'aquest dispositiu. No és gaire diferent de quan algú et ve a visitar en persona:

Joseph Edgar Foreman
123 South Fake St.
Adams County
Ohio

Process #1717403
Device #51688798
LAN 3141 (Data Engineering Labs)
WAN 442 (UAB)

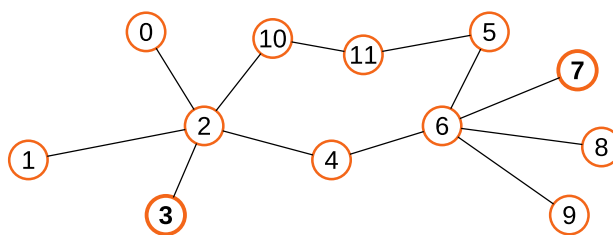
Sigui quin sigui el sistema d'adreçament que fem servir, ha de contenir prou informació per localitzar i arribar al destí, és a dir, per encaminar els nostres missatges a través del graf de connexions. El procés d'encaminament implica diversos passos o "salts" entre xarxes fins que s'arriba a la LAN de destí.

El nom router (encaminador) designa un tipus de dispositiu de xarxa la feina del qual és reenviar aquests missatges entre xarxes. Aquests dispositius han de contenir prou informació perquè, quan reben un missatge amb una adreça de destí, puguin decidir quina interfície de xarxa han de fer servir. Per la seva banda, els dispositius que no són routers només cal que estiguin configurats per poder arribar al router següent.



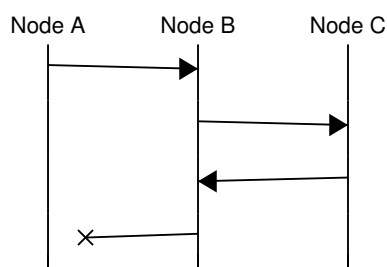
A casa, el teu router normalment també fa el paper de switch, però són conceptes diferents.

Exercici 3.4 Aquesta figura representa un grapat de dispositius i les seves connexions físiques. Aquests dispositius només es poden comunicar enviant missatges a través d'aquestes connexions. Tot i així, ③ vol enviar un missatge a ⑦.



- Pots identificar algun dispositiu que probablement actuï com a switch? I com a router?
- Quin és el **nombre mínim de missatges** enviats perquè ③ pugui contactar amb ⑦, i ⑦ pugui respondre a ③? Enumera'ls en l'ordre en què es produeixen.
- Quina informació s'ha d'incloure en aquests missatges perquè la comunicació ③ ↔ ⑦ pugui tenir lloc?

Els diagrames de carrils com el de sota s'utilitzen per representar les interaccions (p. ex., intercanvi de missatges) entre actors (p. ex., dispositius de xarxa). En aquests diagrames, l'eix vertical representa el temps, cosa que és molt valuosa per identificar relacions de causa-efecte.



Exercici 3.5 A partir de la teva resposta a l'Ex.3.4, **elabora un diagrama de carrils** que representi els intercanvis de missatges. Per a cada missatge (p. ex., damunt de les fletxes), inclou-hi la informació d'adreçament que conté.



4. Això és teu?

La informació viatja per aquest graf de xarxes en forma de paquets, que contenen una petita quantitat d'informació i de metainformació.

Els dos extrems de la comunicació han d'acordar el format dels paquets, de manera que aquests paquets es puguin produir i interpretar de manera eficient i automàtica.

Quan els paquets viatgen per una o més xarxes, es poden perdre o desordenar. Si entre dos ordinadors ha de circular un flux continu de dades, els paquets han de contenir mecanismes per detectar les pèrdues i restaurar l'ordre correcte.

4.1 Tinc una entrega per a tu

No és viable establir connexions físiques entre cada parell d'ordinadors que es volen comunicar. En lloc d'això, es comparteixen relativament poques connexions per transportar missatges entre molts parells de dispositius.

Si un sol dispositiu transmet dades de manera contínua durant molt de temps, aquella connexió queda bloquejada i pot esdevenir un coll d'ampolla que impedeix que altres dispositius es comuniquin. Per evitar aquest problema, les connexions limiten la quantitat màxima de dades que es pot enviar sense interrupció. Com a resultat, els dispositius es veuen obligats a enviar dades en ràfegues discretes anomenades paquets. L'intercanvi d'aquest tipus de missatges a través d'una o més xarxes s'anomena commutació de paquets.



Leonard Kleinrock, un dels pioners de la interconnexió de xarxes, disputa l'autoria de la commutació de paquets, que sovint s'atribueix a Paul Baran i Donald Davies.

Els paquets han de ser petits perquè les connexions no quedin bloquejades durant massa temps. Alhora, volem que els paquets continguin tantes dades com sigui possible, perquè cada paquet conté dades de sobrecàrrega (com ara les adreces que es tracten a la secció 3) que cal enviar a més de les dades de payload (càrrega útil) de l'usuari. Les xarxes fixen la longitud màxima dels paquets mitjançant un paràmetre anomenat unitat màxima de transmissió (MTU), p. ex., 1500 bytes a Ethernet.

La transmissió de paquets a través d'un enllaç de dades s'ha de fer amb cura, sobretot quan aquest enllaç és un bus compartit per diversos dispositius. Cada paquet s'ha de poder distingir individualment de la resta, cosa que es pot fer amb almenys tres estratègies:

1. *Longitud fixa*: tots els paquets tenen la mateixa longitud. L'emissor i el receptor han d'haver acordat aquest valor per endavant.
2. *Longitud explícita*: la longitud del paquet s'inclou dins del paquet, p. ex., fent servir el primer byte del paquet. L'emissor i el receptor han d'acordar com s'inclou aquesta informació i com s'ha d'interpretar.
3. *Seqüències d'escapament*: certes seqüències d'escapament binàries dins de les dades tenen un significat especial. Per exemple, es podria definir que el byte **11110000** (**0xf0**) significa "final de paquet": aquests bits han d'aparèixer després de cada paquet, i només aleshores. Llavors l'emissor podria començar a transmetre el contingut d'un paquet, seguit de **0xf0**. Les dues parts han d'acordar quines seqüències d'escapament fan servir, què signifiquen i com enviar dades iguals a aquestes seqüències (p. ex., hauria de ser possible enviar **0xf0f0f0** sense provocar un "final de paquet" fins que s'hagin transmès tots els bytes).



També és possible senyalitzar l'*inici* d'un paquet. En aquest cas, es fa servir una seqüència prèvia anomenada preàmbul.

Exercici 4.1 Totes les estratègies descrites més amunt es fan servir avui dia en escenaris reals, segons les característiques, els costos i els compromisos de cadascuna.

Discuteix quina d'aquestes estratègies seria la més adequada per a cadascun d'aquests escenaris:

- Un sol fabricant dissenya el maquinari i el programari que comunica dos extrems. Una sola línia de dades s'ha de multiplexar per a diverses ordres de control i també per a diversos fluxos de dades. La prioritat és un ús eficient de l'energia i del buffer (memòria intermèdia).
- Diversos dispositius comparteixen un bus. De tant en tant envien missatges de longituds diferents, però no un volum enorme de dades. La prioritat és el cost.
- Diversos dispositius comparteixen un bus. Envien contínuament missatges de longituds diferents, intentant maximitzar la taxa de transmissió efectiva a través del canal. La prioritat és el throughput (cabal efectiu).

4.2 Si us plau, emplena el formulari

Independentment de l'estratègia emprada, els paquets de dades normalment consten de dues parts:

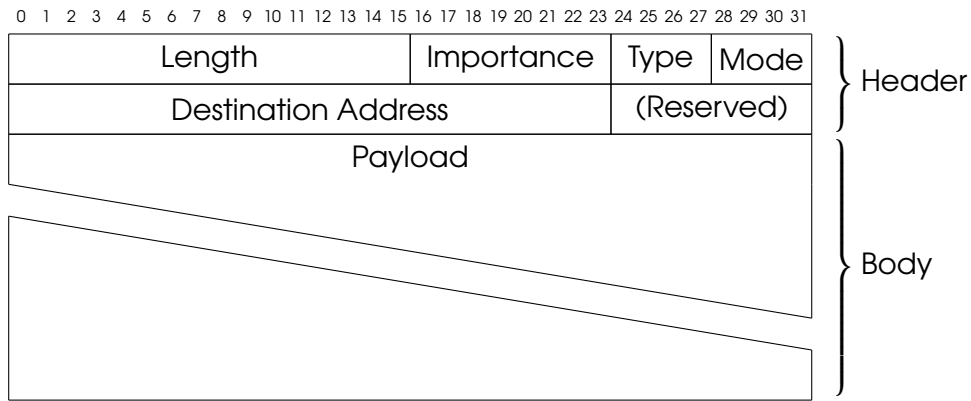
- El payload de dades: la informació que cal transmetre, i
- Les metadades: la metainformació necessària per lliurar el payload.

La longitud pot ser un dels camps de metadades inclosos en el paquet, juntament amb altres que ajuden a identificar-ne el tipus i la funció. Els dos extrems de la comunicació han de conèixer la ubicació del payload i de les metadades, així com els camps exactes de metadades inclosos, la seva longitud i el seu significat. Totes aquestes decisions constitueixen el format del paquet o l'estructura del paquet.

Una de les maneres més habituals d'organitzar el payload i les metadades és amb un format capçalera-cos. En aquest cas, primer s'inclou tota la metainformació, seguida de les dades. Altres formats poden incloure un trailer o cua final (també conegut com a tail, que s'usa sobretot per a la detecció d'errors i la correcció), en combinació amb la capçalera o substituint-la. Totes les configuracions següents són possibles.

Header		Body	
Header	Body		Trailer/Tail
Body			Trailer/Tail
Header			

El format del payload d'un paquet normalment el defineix l'usuari. En canvi, el format de la capçalera està definit estrictament perquè es pugui produir i analitzar fàcilment. Aquest format sovint es presenta amb un diagrama que ajuda a identificar les posicions individuals de cada bit i de cada byte, com en l'exemple (no real) següent:



🎵 Els camps també poden tenir longituds fixes o variables, que no necessàriament s'alineen amb els límits de byte. Els camps també poden tenir una longitud igual a 1 bit, i en aquest cas normalment se'ls anomena flag (indicador) o bit de flag.

🎵 En diagrames com l'anterior, sovint es fan servir diverses línies (files) perquè es puguin imprimir i llegir fàcilment. En aquest cas, el significat és el mateix que si tots els camps s'haguessin presentat en una mateixa línia (fila).

Exercici 4.2 El contingut de paquet següent s'ha capturat d'una xarxa, expressat en hexadecimal. Suposant que el paquet té el format de l'exemple anterior:

- Indica el valor de cada camp (longitud, importància, tipus, mode, adreça de destinació i payload).
- Pots endevinar el significat del payload?

00 0c 00 f3 bb bb bb 00 68 65 6c 70

Exercici 4.3 El codi següent accepta dos arguments: (1, sys.argv[1]) la ruta d'un fitxer d'entrada, i (2, sys.argv[2]) la ruta d'un fitxer de sortida. Es llegeixen els primers 255 bytes del contingut del fitxer d'entrada, es formategen com un paquet, i els bytes d'aquest paquet s'escriuen al fitxer de sortida.

```
.py .out snippets/simplepacketoutput.py
1 with (open(__file__, "rb") as input_file,
2       open("/dev/stdout", "wb") as output_file):
3     # Read at most 255 bytes from the file
4     payload: bytes = input_file.read(255)
5     # The + operation concatenates bytes
6     output_file.write(bytes(len(payload)) + payload)
```

- Descriu el format de paquet que es fa servir al codi, i les seves limitacions.
- Amplia el format de paquet de manera que
 - Els paquets puguin tenir més de 255 bytes
 - Es pugui codificar l'adreça del dispositiu de destinació
- Proporciona un diagrama de bytes del nou format, així com una explicació del sistema d'adreçament que fas servir.
- Modifica el codi perquè generi paquets amb el format que proposes.

🎵 Llenguatges com Python poden distingir entre fitxers que contenen text i fitxers que contenen dades binàries. Al codi anterior, els fitxers s'obren en mode binari amb els modes `'rb'` i `'wb'`.

Quan s'obren en mode binari, els bytes del fitxer es representen directament amb un objecte `bytes`, un vector d'enters entre 0 i 255. En mode text, el mètode `open` processa (descodifica) aquests bytes una mica més, i retorna una cadena que pot contenir caràcters especials com accents o glifs no llatins. ([llegir-ne més...](#))

4.3 No paren d'arribar

Quan es desenvolupen aplicacions que fan servir capacitats de xarxa, sovint és útil imaginar la comunicació com un flux, és a dir, com una canonada conceptual on posem les nostres dades (qualsevol quantitat de dades) per un extrem, i surten per l'altre. Si tenim aquesta capacitat, aleshores és molt més fàcil enviar fitxers de qualsevol mida i transmetre una quantitat inacabable d'àudio o de vídeo. Malauradament, tot el que tenim per simular aquests fluxos són paquets.

Sovint cal reenviar els paquets a través de diverses xarxes. Els routers (encaminadors) no són perfectes i de vegades estan inoperatius o mal configurats, de manera que no tots els paquets arriben a la seva destinació. A més, paquets diferents es poden lliurar per rutes diferents que poden tenir longituds diferents, de manera que els paquets poden arribar desordenats.

Si volem simular un flux de dades, els paquets han de contenir prou metainformació perquè es puguin detectar les pèrdues i restaurar l'ordre correcte. Això introdueix una sobrecàrrega que no és adequada en tots els escenaris –p. ex., latència molt baixa–, de manera que algunes aplicacions basen la seva comunicació de xarxa en missatges en lloc de fluxos.

Quan calen fluxos, es fa servir el concepte d'offset (desplaçament) per tornar a muntar diversos missatges en un sol flux. Cada paquet conté alguns bytes del flux de dades, i l'offset n'indica la ubicació exacta dins d'aquest flux.

Exercici 4.4 El diagrama següent descriu un flux de 48 bytes produït per un node d'origen, així com els paquets en què es va dividir abans de la transmissió:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	32	33	34	35	36	37	38	39	40	41	42	43	44	45	46	47			
<u>Packet 0</u> Offset 0 Length 10										<u>Packet 1</u> Offset 10 Length 14										<u>Packet 2</u> Offset 24 Length 6						<u>Packet 3</u> Offset 30 Length 18																								

- Descriu la capçalera mínima que es pot fer servir per transmetre aquest flux.
- Suposant que el destinatari rep primer el paquet 2, seguit del paquet 0, i que els altres paquets es perden: quins bytes del flux coneix el node de destinació, i quins bytes són desconeguts?
- Com pot saber el node d'origen que alguns paquets no s'han lliurat?
- Què passaria si la destinació rebés un missatge fals, amb offset 7 bytes i longitud 4 bytes?

5. Una pila de capes

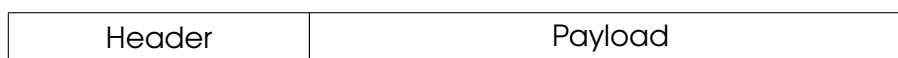
La comunicació moderna entre ordinadors s'aconsegueix fent servir una pila de capes de xarxa. Cada pila se centra en alguns problemes i és responsable de proporcionar certes funcionalitats, p. ex., l'adreçament i la seqüenciació correcta (presentats en els capítols anteriors).

Cada capa defineix una parella de funcions `send/receive`, que és responsable d'algunes d'aquestes funcionalitats. Cada capa fa servir el `send` i el `receive` de la capa inferior, és a dir, les funcionalitats d'una capa es basen en les funcionalitats de totes les capes que té per sota.

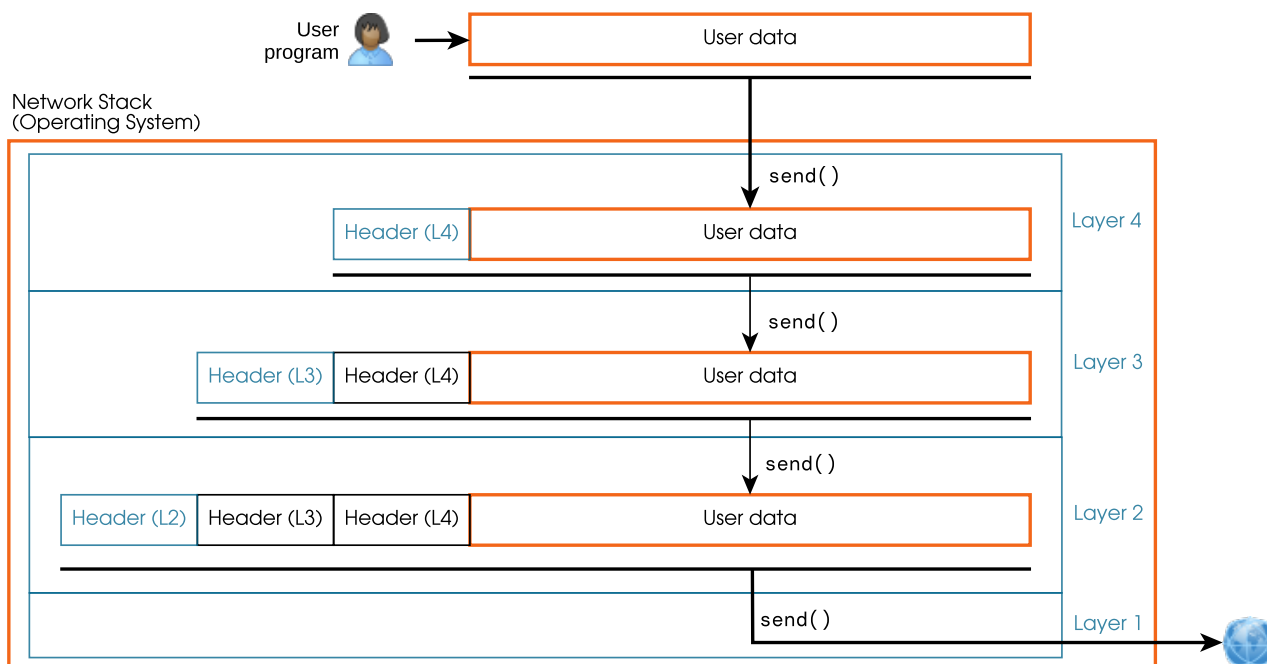
Per coordinar les capes i permetre una comunicació virtual entre elles, cada capa defineix un format de paquet diferent, que encapsula un paquet de la capa superior.

5.1 Enviament de dades

Per implementar aquestes capacitats, cada capa transmet no només les dades (*data*) proporcionades per la capa superior, sinó també una capçalera amb tota la metainformació necessària per a aquella capacitat. Així, cada capa ha de definir el format de paquet, afegint una certa sobrecàrrega (típicament una capçalera) al payload (càrrega útil), p. ex.:



Quan un programa d'usuari vol enviar dades, fa servir el mètode `send` de la capa *superior* de la pila, cosa que acaba fent que la capa física transmeti dades, p. ex., a través d'un cable o d'una antena. La figura següent mostra l'estructura d'una pila de 4 capes:



Els paquets que produeix cada capa s'anomenen genèricament PDUs (Protocol Data Unit, unitat de dades del protocol). Les PDUs de cada capa tenen noms específics com ara frame (trama), datagrama o segment. Aquests noms s'han de fer servir amb precisió en contextos tècnics.

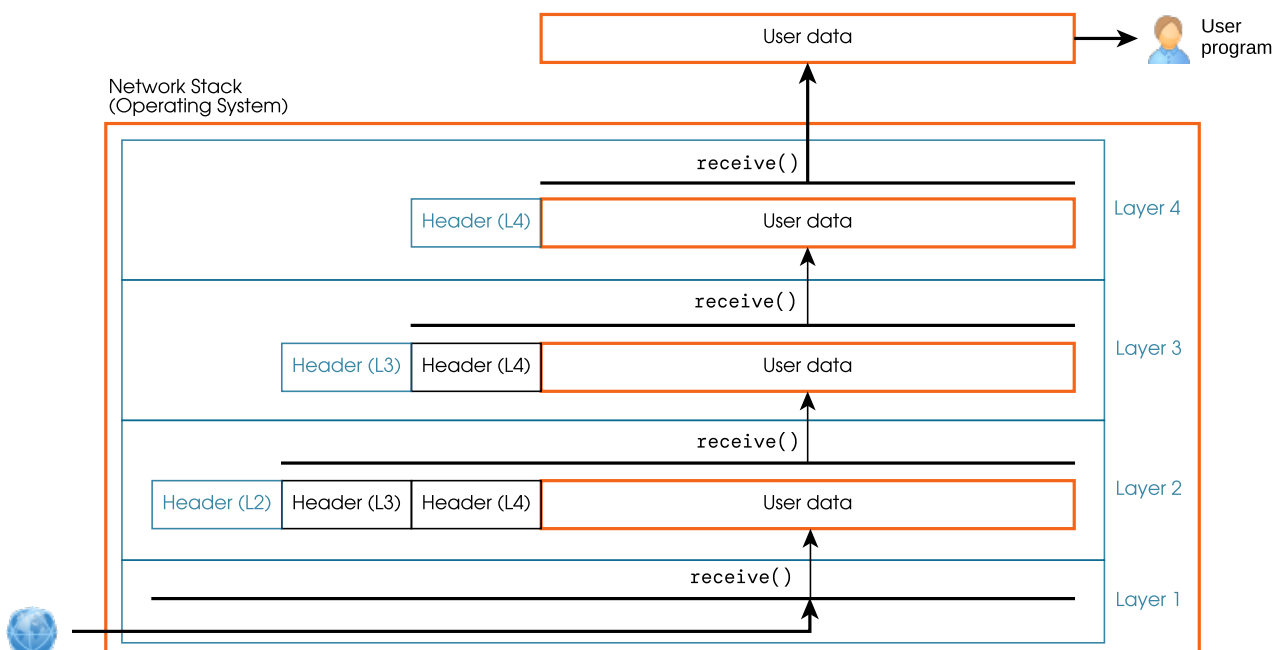
Cada capa inclou la PDU de la capa superior dins la definició del format de paquet de la seva PDU. Aquest procés s'anomena encapsulació.

Exercici 5.1 El programa d'usuari vol enviar 420 bytes de dades fent servir la pila de la figura anterior. Suposa una mida de capçalera per a les capes 2, 3 i 4 de 14 bytes, 20 bytes i 20 bytes, respectivament.

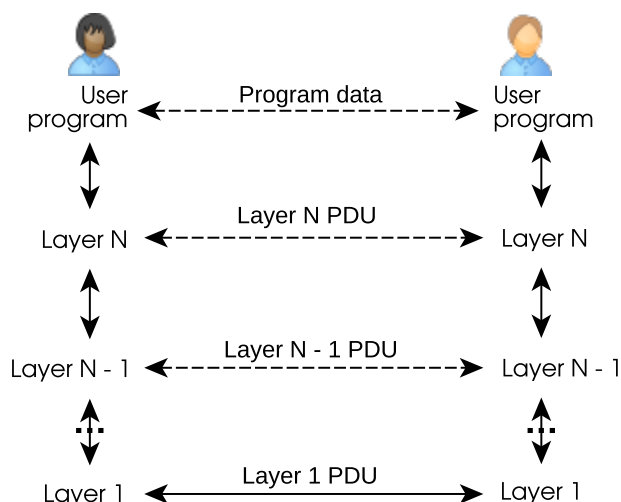
- Quina és la mida del payload i la mida de la sobrecàrrega de la PDU de cada capa?
- Quina és la mida del payload i la mida de la sobrecàrrega de tota la pila?

5.2 Recepció de dades

Quan un paquet arriba a la pila de xarxa del destinatari, es processa en ordre invers, és a dir, les capes es recorren de baix a dalt. Cada capa inspecciona *només* la capçalera corresponent a aquella capa. Si tot és correcte, la capa elimina la capçalera i passa la resta del paquet a la capa superior. Al seu torn, la capa de més amunt passa les dades restants (el payload) al destinatari. Continuant amb l'exemple de la pila de 4 capes:



En una pila de xarxa, només la capa inferior interactua directament amb l'exterior, p. ex., només la targeta de xarxa física d'un ordinador està connectada a un cable que porta dades cap a altres ordinadors. Alhora, les dades enviades per la capa inferior inclouen les PDUs de totes les capes mitjançant l'encapsulació. Gràcies a això, hi ha una comunicació virtual entre cada capa d'un extrem i la seva homòloga a l'altre extrem de la comunicació. La figura següent representa la comunicació real (línia contínua) i virtual (línia discontinua) del nostre exemple:



Exercici 5.2 Un altre exemple de pila de xarxa té 3 capes: L1, L2 i L3, de baix a dalt.

La PDU de la capa L2 és la següent:

0	7 8	15 16	23 24	31
Destination address		Payload length (bytes)		
Payload (variable length)				
⋮				

La PDU de la capa L3 és la següent:

0	7 8	15 16	23 24	31
Fragment offset		Payload length (bytes)		
Payload (variable length)				
⋮				

La capa L1 rep un paquet de dades i la seva funció `receive` retorna les dades següents (suposa enters sense signe i ordre big endian (primer el byte més significatiu) quan calgui:

01 23 00 0a 01 00 00 06 00 aa 00 bb 43 20

- Quin payload de dades s'ha enviat al programa destinatari en aquest paquet?
- Si aquest és l'últim paquet de la comunicació, quantes dades (payload) s'han enviat al programa d'usuari destinatari?
- Com a màxim, quants ordinadors diferents es poden connectar fent servir aquesta pila?
- Enviem un fitxer com a payload d'aquesta pila. Quina és la seva mida màxima possible?

```

.PY .out snippets/encapsulation.py
1 import typing
2 import struct
3
4 def layer2_encapsulate(address: int, data: bytes) -> bytes:
5     """Encapsulate a PDU of layer 3 into a PDU of layer 2."""
6     assert 0x0000 <= address <= 0xffff, "Invalid layer 2 address"
7     assert len(data) <= 0xffff, "Too much data for a layer 2 PDU"
8     return bytes(struct.pack(">H", address)
9                 + struct.pack(">H", len(data))
10                + data)
11
12 def layer3_encapsulate(offset: int, data: bytes) -> bytes:
13     """Encapsulate a chunk of user data into a PDU of layer 3."""
14     assert 0x0000 <= offset <= 0xffff, "Invalid layer 3 offset"
15     assert len(data) <= 0xffff, "Too much data for a layer 3 PDU"
16     return bytes(struct.pack(">H", offset)
17                 + struct.pack(">H", len(data))
18                + data)
19
20 def stack_send(data: bytes, output_file: typing.BinaryIO):
21     layer3_pdu = layer3_encapsulate(offset=0, data=data)
22     layer2_pdu = layer2_encapsulate(address=0xabcd, data=layer3_pdu)
23     output_file.write(layer2_pdu)
24
25 if __name__ == '__main__':
26     with (open(__file__, "rb") as input_file,
27         open("/dev/stdout", "wb") as output_file):
28         payload = input_file.read()
29         stack_send(data=payload, output_file=output_file)

```

Exercici 5.3 El codi anterior agafa unes dades i escriu els bytes que s'enviarien a la capa 1 (la capa física) de la pila de xarxa de l'exercici 5.2. Implementa l'extrem receptor de la pila de la manera següent:

- Implementa una funció `layer2_decapsulate(data: bytes) -> (int, bytes)` que rebi una PDU de capa 2 i retorni la tupla (address, payload), on address és l'adreça de destinació i payload és la PDU de capa 3 encapsulada.
- Implementa una funció `layer3_decapsulate(data: bytes) -> (int, bytes)` que rebi una PDU de capa 3 i retorni la tupla (offset, payload), on offset és l'offset (desplaçament) del payload, i payload és el fragment de dades enviat per l'altre extrem de la comunicació.
- Implementa una funció `stack_receive(data: bytes)` que descodifiqui una PDU de capa 1 (fent servir les dues funcions anteriors) i imprimeixi una línia de missatge semblant a
"Received a chunk of data. Length: 12 bytes, Address: 0xabcd, Offset: 0."
- Completa la funció `stack_send` perquè les dades d'entrada es divideixin en fragments de com a màxim 100 bytes, es produeixi una PDU de capa 1 vàlida per a cadascun d'aquests fragments, i aquestes PDUs de capa 1 es passin a `stack_receive`.
- La teva implementació funcionaria si les PDUs de capa 1 s'ordenessin aleatòriament abans de passar-les a la pila receptora?



La biblioteca `struct` de Python és útil, però no obligatòria, per donar format als bytes dels paquets.

Internet

La part anterior d'aquesta guia d'estudi ha explorat alguns dels principals reptes de comunicar múltiples ordinadors. Internet és una solució que funciona per a aquests reptes: la que els humans han creat durant l'últim mig segle, ara central en moltes activitats socials, comercials i polítiques. Aquesta part estudia les principals funcionalitats que ofereix Internet, així com el conjunt de protocols emprats per aconseguir-les.

6	Internet	26
6.1	La pila TCP/IP	
6.2	Visió general dels protocols TCP/IP	
6.3	Qui mana a Internet?	
7	Capa 2: Comunicació de xarxa (LAN)	29
7.1	Adreçament de capa 2	
7.2	Protocol de capa 2 Ethernet	
8	Capa 3: comunicació entre xarxes (Internet)	33
8.1	Adreçament de la capa 3	
8.2	IP – Internet Protocol	
8.3	Encaminament IP	
8.4	Divisió en subxarxes IP	
8.5	Fragmentació IP	
8.6	ARP – Address Resolution Protocol	
8.7	ICMP – Internet Control Message Protocol	
9	Capa 4: Transport	48
9.1	Adreçament de la capa 4	
9.2	UDP – User Datagram Protocol	
9.3	TCP – Transport Control Protocol	
9.4	TCP: “Connectant”	
9.5	TCP: “Connexió establerta”	
9.6	TCP: “Tancant la connexió”	
10	Capa 7: Aplicació	60
10.1	DNS – Sistema de noms de domini	
10.2	DHCP – Protocol de configuració dinàmica de hosts	
10.3	Sistemes autònoms	

6. Internet

Les comunicacions modernes es basen en piles de xarxa organitzades en capes. Potser acabes de descarregar aquesta guia d'Internet. Quines capes s'utilitzen exactament? Quins protocols s'empren, i amb quina finalitat? Aquest capítol ofereix una introducció per respondre aquestes preguntes.

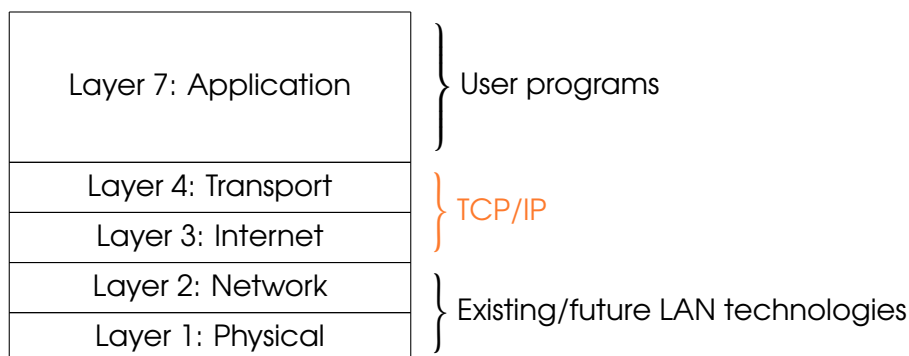
6.1 La pila TCP/IP

Internet és una solució pràctica al problema de la comunicació global entre ordinadors, el desenvolupament de la qual va començar a finals dels anys seixanta i principis dels setanta, i encara continua.

La commutació de paquets es va concebre a principis dels anys seixanta, i la primera implementació d'àrea extensa, ARPANET, va començar el 1969. Un repte clau en aquell moment era interconnectar les xarxes existents, propietat de diferents organitzacions i basades en diferents tecnologies i protocols. Perquè tot fos viable, calia complir tots els requisits següents:

- Les xarxes existents havien de funcionar sense actualitzacions de la infraestructura física ni dels seus protocols, només de la manera com s'utilitzen aquests protocols.
- Les xarxes existents havien de continuar gestionant-se de manera independent a nivell local (organització interna autònoma).
- Els paquets no sempre s'entreguen: alguns es perden.
- Els missatges que arriben poden no estar en ordre, i poden estar repetits.

La solució adoptada per ARPANET –i més tard per tot el món– és el conjunt de protocols TCP/IP, descrit per primera vegada el 1974. És un enfocament per capes, dissenyat per “endollar-se” damunt de les LANs (xarxes d'àrea local) existents, amb l'objectiu d'oferir capacitats de xarxa a totes les aplicacions d'usuari.



En poques paraules, els principals serveis que ofereixen aquestes capes són els següents:

Layer 1 (Física): Enviar dades digitals a través de mitjans analògics.

Layer 2 (Xarxa): Identificar i comunicar dispositius dins de la mateixa LAN.

De vegades s'utilitza el terme capa d'“enllaç de dades” (del model OSI).

Layer 3 (Internet): Identificar i comunicar dispositius a través de totes les LANs.

De vegades s'utilitza el terme capa de “xarxa” (també del model OSI).

Layer 4 (Transport): Identificar i comunicar programes (serveis).

A més (tria'n *només una*):

- a) Assegurar que un flux de dades arriba correctament a l'altre extrem, i en ordre – però s'introdueix una sobrecàrrega i cal establir connexions (TCP).
- b) Utilitzar missatges simples i relativament eficients, amb una sobrecàrrega petita – però els missatges es poden perdre, desordenar o duplicar (UDP).

Layer 7 (Aplicació): Fer qualsevol cosa que l'usuari pugui necessitar.

No t'oblidis de la seguretat (p. ex., criptografia) ni de l'eficiència (p. ex., compressió): la capa 4 i les inferiors no ho proporcionen.

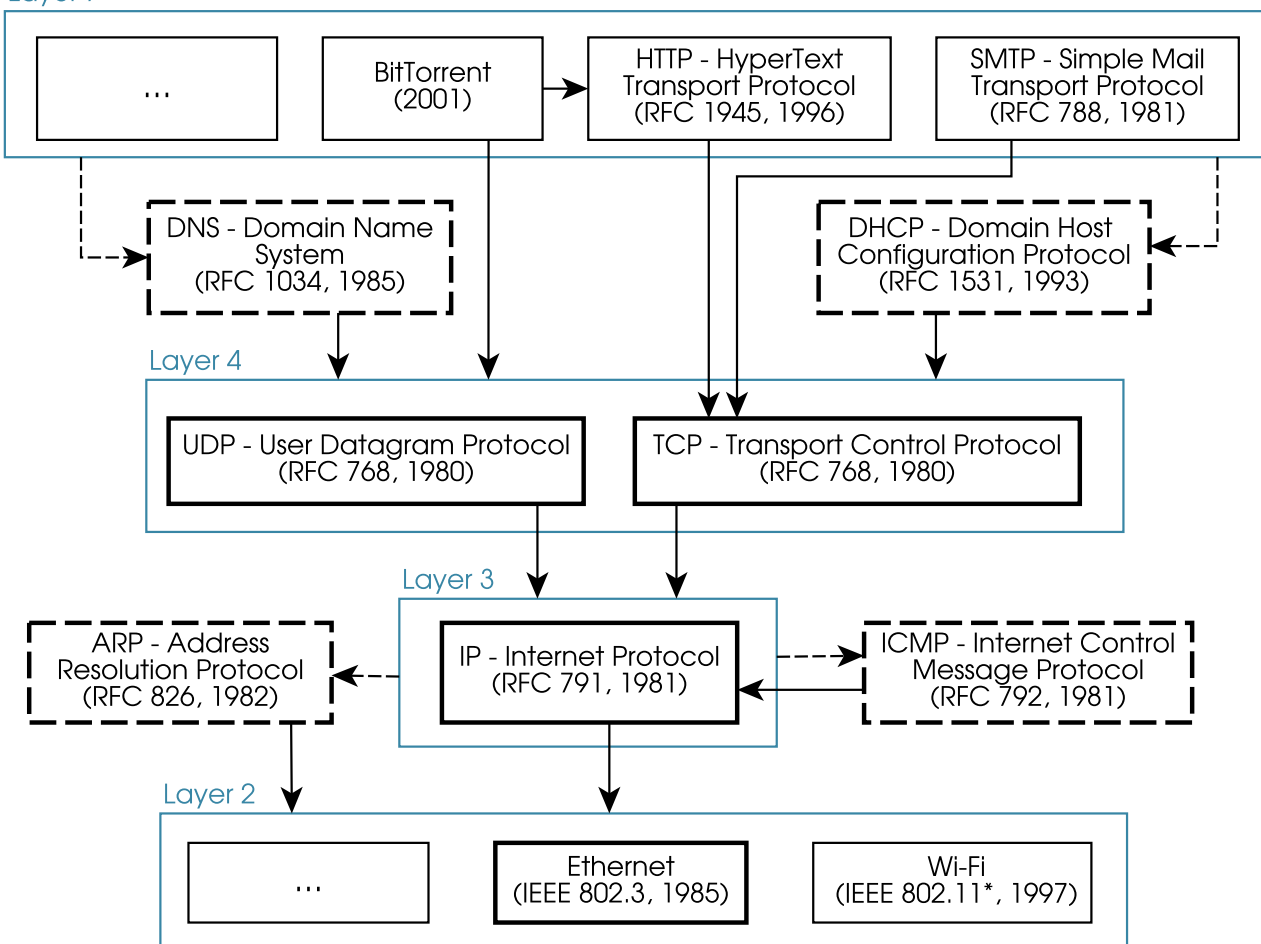
6.2 Visió general dels protocols TCP/IP

Es defineixen múltiples protocols per implementar la funcionalitat de cada capa. Aquestes definicions inclouen el format de paquet requerit, les regles per intercanviar aquests paquets i la seva semàntica. Aquests protocols s'han d'entendre com un sistema, en el qual cada part té tasques molt específiques.

Alguns dels protocols més rellevants per a les comunicacions a través d'Internet es mostren a la figura següent. En aquesta:

- Les fletxes contínues indiquen que un protocol s'encapsula dins d'un altre. Si el destí és una capa, cal utilitzar un dels protocols d'aquesta capa.
- Les caixes discontinües indiquen protocols auxiliars definits per ajudar altres funcions. Les fletxes discontinües indiquen quina capa utilitza quin protocol auxiliar.
- Les caixes amb línies més gruixudes indiquen els protocols destacats en aquesta guia d'estudi.

Layer 7



Exercici 6.1 Volem enviar la frase "Perdó, arribaré 5 minuts tard" per correu electrònic. El nostre client de correu enviarà aquestes dades amb el protocol SMTP, que al seu torn utilitza TCP (part de la capa 4). Suposant que SMTP utilitza capçaleres a la seva PDU, quantes capçaleres contindrà la PDU corresponent de la capa 2?

Exercici 6.2 Per què creus que la capa 3 només té un protocol (dos si comptes IPv4 i IPv6 per separat)?

6.3 Qui mana a Internet?

La Internet Engineering Task Force (IETF) es va crear el 1986 per desenvolupar i publicar estàndards per utilitzar a Internet. El seu principal resultat són els documents "Request for Comment" (RFC), que contenen descripcions tècniques completes d'aquests estàndards.

Aquests RFCs els creen grups de treball d'experts, perquè tothom els pugui utilitzar lliurement i de franc. Qualsevol persona pot participar en aquests grups de treball, inclosos investigadors, governs i agents de la indústria. Les decisions es prenen quan hi està d'acord entre el 80 i el 90% dels participants, de manera que cap entitat controla per si sola el procés d'estandardització.



"Rebutgem els reis, els presidents i les votacions. Creiem en el consens aproximat i en el codi que funciona" — Dave Clark (IETF), 1992.

Consulta el DOI [10.1109/MAHC.2006.42](https://doi.org/10.1109/MAHC.2006.42) per a una breu història de lluita pel poder.

Els estàndards RFC són descriptius, no prescriptius. Això vol dir que ningú vigila l'ús correcte d'aquests estàndards. En canvi, els fabricants tenen l'incentiu de complir els RFCs perquè els seus productes siguin compatibles amb els d'altres. El valor dels RFCs (i de qualsevol altre estàndard) depèn que els usuaris els adoptin. Per aquest motiu, els RFCs es poden actualitzar i retirar.

Els protocols TCP/IP s'imposen perquè són flexibles i funcionen raonablement bé en la majoria de situacions, tot i que no són òptims pràcticament en cap. A més, els protocols TCP/IP són lliures, a diferència dels protocols privatius que obligaven a comprar la solució d'un fabricant concret.



Avui és difícil imaginar un món *no* dominat per l'arquitectura TCP/IP, però es van comercialitzar i mantenir piles completament diferents fins a finals dels noranta. En són exemples AppleTalk i IPX/SPX, d'Apple i Novell, respectivament.

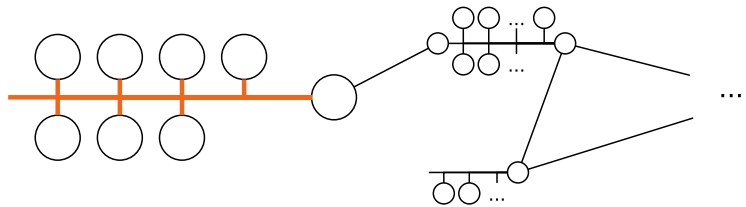
Alguns aspectes de TCP/IP, com ara els identificadors únics (inclòs el DNS, el sistema de noms de domini) i els números reservats, han de ser acordats per tothom perquè Internet funcioni correctament. La IANA (Internet Assigned Numbers Authority) treballa en coordinació amb la IETF per definir els RFCs adequats.



Des del 2016, la IANA és gestionada per una organització sense ànim de lucre amb múltiples parts interessades (ICANN - Internet Corporation for Assigned Names and Numbers). Des del 2004, la responsabilitat dels dominis regionals (p. ex., .cat, .fr, .es) s'ha transferit a organitzacions d'àmbit continental anomenades registres regionals d'Internet (RIRs).

7. Capa 2: Comunicació de xarxa (LAN)

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



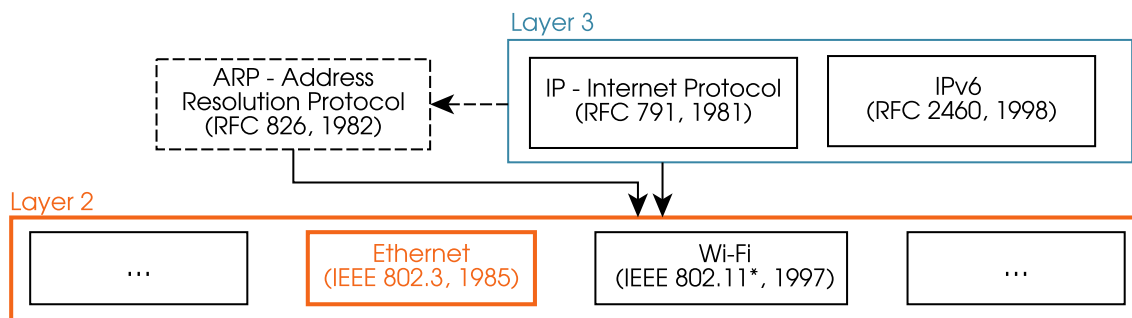
Capacitats

Els protocols de capa 2 permeten que diversos dispositius intercanviïn missatges i s'identifiquin entre ells dins d'una xarxa local (LAN (xarxa d'àrea local)). Amb els protocols de capa 2 no es pot contactar *directament* amb dispositius de fora de la LAN.

La capa 2 abstruïu les capes superiors (incloses les aplicacions d'usuari) dels detalls específics del maquinari connectat. Això fa més fàcil (és a dir, més barat) escriure el codi una vegada i executar-lo a tot arreu.

Els missatges de capa 2 tenen una longitud limitada. La quantitat màxima de dades de payload (càrrega útil) que es pot inserir en un protocol de capa 2 s'anomena unitat màxima de transmissió (MTU), p. ex., 1500 bytes per a Ethernet.

Alguns protocols de capa 2 també poden oferir autenticació d'usuaris i protecció criptogràfica; Wi-Fi n'és un exemple destacat, tot i que les xarxes amb cable també poden fer servir, p. ex., IEEE 802.1X



Protocols

S'han definit i s'estan definint molts protocols dins de la capa 2. Els més importants avui dia són:

- Ethernet (IEEE 802.3), tractat a la Section [7.2](#).
- Wi-Fi (p. ex., IEEE 802.11be, també conegut com a Wi-Fi 7, del 2024).

Dins de TCP/IP, la capa 2 encapsula les PDUs dels protocols següents (tots dos tractats al capítol 8):

- Datagrames IPv4 i IPv6.
- Missatges ARP.



L'ordre `ip link` mostra les teves interfícies i permet configurar-les, incloent-hi l'adreça MAC.

El teu sistema operatiu normalment assigna automàticament noms interns a les interfícies segons el seu tipus, p. ex., `eth0` o `wlan1`. Normalment es poden canviar els noms de les interfícies, perquè aquests noms no formen part de TCP/IP; només es fan

servir dins d'aquell sistema operatiu.

7.1 Adreçament de capa 2

En la majoria de tecnologies LAN, les adreces de capa 2 són identificadors numèrics únics per a cada dispositiu dins de la LAN. El tipus més habitual són les adreces MAC d'Ethernet (tècnicament EUI-48), que ocupen 6 bytes (és a dir, 48 bits). També es fan servir fora d'Ethernet, p. ex., a Wi-Fi i Bluetooth.

Les adreces MAC es presenten a les persones sovint en el format següent:

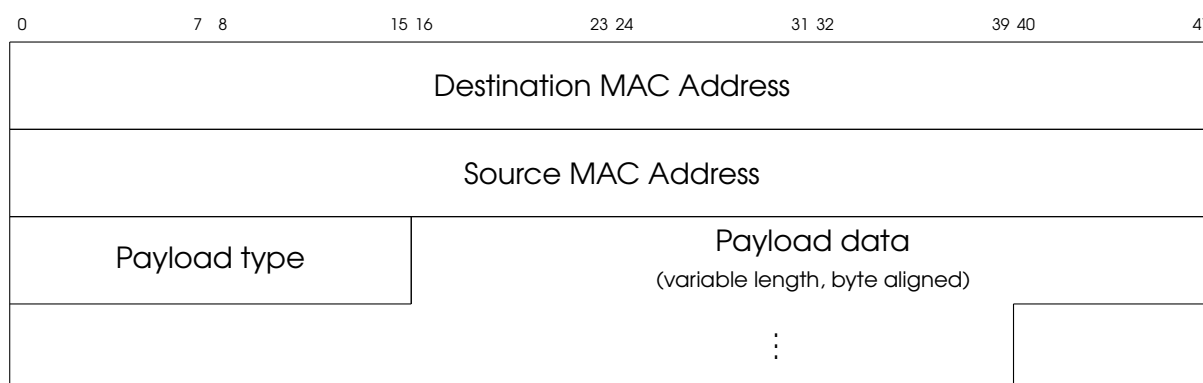
01:23:45:67:89:ab. Algunes adreces tenen un significat especial i no són identificadors de dispositiu vàlids. Per exemple, **ff:ff:ff:ff:ff:ff** és l'adreça de **broadcast** (difusió), que vol dir "tothom a la LAN". A més, **00:00:00:00:00:00** es fa servir per representar una adreça MAC desconeguda (consulta l'[RFC 9542](#) per a una descripció completa de les adreces MAC reservades, incloses les de **multicast** (enviament a un grup)).

El maquinari de xarxa ve amb una adreça MAC integrada, única i predefinida, a punt per fer-se servir. Les MAC dels dispositius es poden configurar a nivell del sistema operatiu, però es mantenen constants mentre s'utilitzen.

7.2 Protocol de capa 2 Ethernet

7.2.1 Format de paquet

La capa 2 d'Ethernet defineix el format següent, que es fa servir en tots els missatges. Els paquets d'aquest tipus s'anomenen **frames** (trames):



- **MAC de destinació:** l'adreça MAC del dispositiu de destinació, o **ff:ff:ff:ff:ff:ff** per a **broadcast**.
- **MAC d'origen:** la MAC del dispositiu que envia el frame.
- **Tipus de payload:** identificador del protocol **encapsulat** al payload, p. ex., **0x0800** → IPv4, **0x86DD** → IPv6 i **0x0806** → ARP. De vegades s'anomena **EtherType**.
- **Payload:** contingut que es demana enviar, p. ex., per part d'IPv4 o ARP. La longitud màxima d'aquest camp, la **MTU**, és de 1500 bytes per a Ethernet.

Exercici 7.1

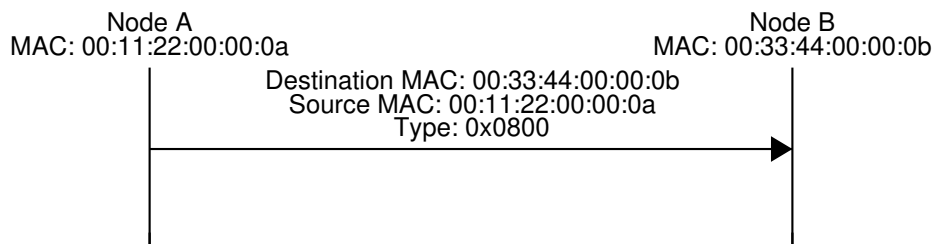
- Quantes adreces MAC diferents hi ha a Ethernet (incloent-hi els **blocs** reservats)?
- N'hi ha prou perquè cada dispositiu amb connexió a internet de la Terra tingui una MAC única?

Exercici 7.2 És vàlid el frame següent?

c2	21	90	15	bc	b6	42	40	e5	f7	5e	32	08	42	77	57
03	ca	27	6b	1f	97	e9	02	24	7f	80	fa	8b	61	59	2a
81	b6	73	b0	b4	e1	75	4f	ef	40	dd	9f	34	3c	48	67

7.2.2 Funcionament

Els missatges dins d'una LAN Ethernet normalment s'envien en mode unicast (enviament a un únic destinatari), és a dir, són missatges d'un a un. Això permet als switches (commutadors) minimitzar el consum d'energia i d'amplada de banda. En unicast, el node del dispositiu d'origen ha de conèixer la MAC de la destinació.



Ethernet també admet broadcast a tots els nodes de la LAN. En aquest cas, l'adreça MAC de destinació ha de ser ff:ff:ff:ff:ff:ff. Quan un dispositiu rep un frame, el descarta tret que el camp de MAC de destinació del frame sigui idèntic a la seva pròpia adreça MAC, o de broadcast.

Un node també descartarà un frame amb un camp de tipus de payload (EtherType) no admès. Si el tipus de payload és admès, es fa servir per decidir quina part del sistema operatiu rep el missatge, p. ex., la pila IPv4 o el subsistema ARP.

Exercici 7.3 Continuant l'exemple de la figura anterior, la targeta de xarxa del node B rep un frame que comença amb els bytes següents. L'hauria d'acceptar B?

00 11 22 00 00 0a ff ff ff ff ff ff ff ff 08 DD ...

Exercici 7.4 Els scripts següents mostren com enviar i rebre frames Ethernet en brut. El client envia 5 frames idèntics d'EtherType 0x1234 i després acaba. El servidor espera fins que rep 5 frames Ethernet d'aquest tipus (comprovat a les línies 12-13) i després acaba.

- Canvia l'adreça MAC i el nom d'interfície del client pels teus.
 - Fes que el servidor rebutgi els frames que no li són adreçats.
 - Amplia aquests scripts per implementar una aplicació senzilla amb capacitats LAN.
- Exemples suggerits:
- Un servei d'eco de capa 2 que distingeixi entre peticions i respostes.
 - Un xat peer-to-peer (de parell a parell) de capa 2 que inclogui el sobrenom de l'origen en tots els missatges.
 - Un servei de calculadora de capa 2 que admeti operacions aritmètiques bàsiques (+, -, *, divisió entera // i, opcionalment, divisió /).
- Què falta perquè la teva aplicació es pugui connectar a la resta d'Internet més enllà de la teva LAN?

  snippets/ethernetclient.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4 source_mac = bytes([0x00, 0x11, 0x22, 0x33, 0x44, 0x55])
5 destination_mac = bytes([0xff, 0xff, 0xff, 0xff, 0xff, 0xff])
6 packet_type = bytes([0x12, 0x34])
7 payload_data = bytes([0x78, 0x6f, 0x69, 0x20, 0x75, 0x61, 0x62])
8
9 # This socket sends Layer 2 packets directly.
10 s = socket.socket(family=socket.AF_PACKET,
11                  type=socket.SOCK_RAW,
12                  proto=socket.ntohs(socket.ETH_P_ALL))
13 s.bind((interface_name, 0))
14
15 for i in range(5):
16     frame = destination_mac + source_mac + packet_type + payload_data
17     s.send(frame)
18     print(f"Sent frame {i + 1}/5: {len(frame)=}, {payload_data=}")

```

  snippets/ethernetserver.py

```

1 import socket
2
3 interface_name = "wlp5s0" # Needs manual configuration
4
5 # This socket receives Layer 2 packets directly.
6 s = socket.socket(family=socket.AF_PACKET,
7                  type=socket.SOCK_RAW,
8                  proto=socket.ntohs(socket.ETH_P_ALL))
9 s.bind((interface_name, 0))
10
11 valid_frames = 0
12 while valid_frames < 5:
13     frame = s.recv(1514)
14     if frame[12:14] != bytes([0x12, 0x34]):
15         continue
16     valid_frames += 1
17     print(f"Received frame {valid_frames + 1}/5, {len(frame)} bytes. "
18           f"Payload: {frame[14:]:!r}")

```



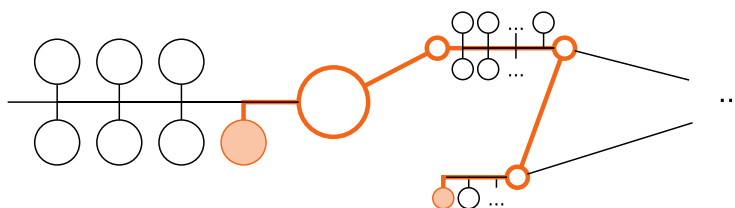
Hauràs d'executar aquests scripts amb privilegis, p. ex., amb `sudo`. Com a alternativa, pots afegir de manera permanent les capacitats `CAP_NET_RAW` al teu binari de python amb

```
sudo setcap cap_net_raw+ep ./venv/bin/python.
```

Després, pots executar `./venv/bin/python script.py` directament sense `sudo`.

8. Capa 3: comunicació entre xarxes (Internet)

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



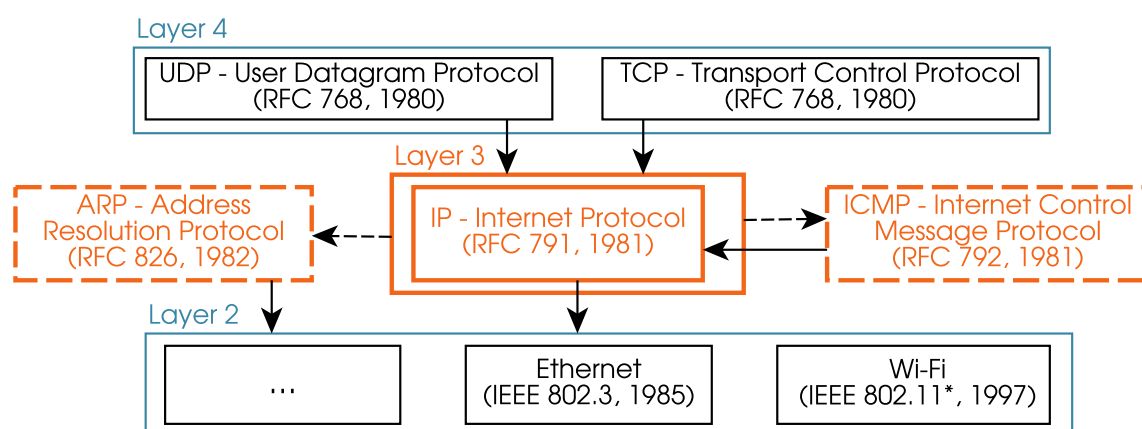
Funcionalitats

El protocol d'Internet (IP) permet intercanviar datagrames (PDU de la capa 3) entre dispositius de diferents LANs (xarxa d'àrea local). El seu sistema d'adreçament identifica els dispositius globalment i fa possible ençaminar els datagrames allà on calgui.

Els dispositius que es comuniquen via IP poden pertànyer a LANs basades en tecnologies diferents i amb tipus d'adreça MAC diferents. Això té dues implicacions:

- Un datagrama que "cap" en la MTU de la LAN d'origen pot ser massa gran per a la MTU de la LAN de destinació (o de qualsevol LAN intermèdia). IP defineix un sistema de fragmentació per dividir els datagrames, que només es reconstrueixen a la destinació final.
- Quan enviem un datagrama, coneixem l'adreça IP del dispositiu destinatari, però no necessàriament la seva adreça MAC. Només necessitem aquesta MAC si formem part de la LAN de destinació. IP fa servir ARP per traduir adreces IP conegudes a adreces MAC del tipus de la LAN de destinació.

Els datagrames poden viatjar per xarxes que no controlen ni l'origen ni la destinació, de manera que els datagrames es poden perdre, desordenar i duplicar. La capa 3 no proporciona mecanismes per gestionar-ho; les capes superiors han d'implementar mecanismes de protecció quan calgui (p. ex., per implementar els fluxos de dades de TCP).



Protocols

La versió 4 d'IP és el protocol que s'utilitza a la capa 3, és a dir, és comú a pràcticament totes les comunicacions de la Internet tal com la coneixem avui. Fa molts anys que Internet lluita per fer la transició cap a IPv6, però el procés no s'ha completat i només IPv4 ofereix cobertura global.

IPv4 delega en el protocol ARP la traducció d'adreces IP conegudes a adreces MAC dins

de cada LAN. ARP *no* fa servir datagrames IP.

Els protocols següents encapsulen les seves PDU en el payload (càrrega útil) dels datagrames IP:

- Transport Control Protocol (TCP)
- User Datagram Protocol (UDP)
- Internet Control Message Protocol (ICMP)



Les ordres `ip address`, `ip route` i `ip neighbor` (entre d'altres) et permeten consultar diversos aspectes de la teva configuració IP.

8.1 Adreçament de la capa 3

Les adreces IPv4 tenen 32 bits. Normalment es presenten a les persones en el format quad decimal, p. ex.,

142.250.201.67

Les adreces IPv6 tenen 128 bits (12 bytes), és a dir, són 4 vegades més grans que una adreça IPv4. S'expressen en format hexadecimal, p. ex., com es mostra a continuació. Fixa't que dos signes de dos punts "::" indiquen "omple amb zeros", i que "::" només pot aparèixer una vegada per adreça IPv6.

fe80::f524:a73a:e946:7c02



Quan una adreça IPv6 forma part d'un URL, s'envolta de claudàtors, p. ex., [https://\[fe80::f524:a73a:e946:7c02\]](https://[fe80::f524:a73a:e946:7c02])

8.1.1 Adreces públiques i reservades

La majoria d'adreces IP són públiques i úniques a tot Internet, és a dir, l'adreça IP anterior té el mateix significat a tot el món: identifica un únic dispositiu connectat. Moltes adreces són reservades/privades i només es poden fer servir dins d'un àmbit concret (p. ex., una LAN) o en una situació concreta. Algunes de les més habituals són les següents (n'hi ha algunes més, també per a IPv6):

Bloc d'adreces IP	Àmbit de l'adreça
127.*.*.*	Aquest ordinador (localhost)
10.*.*.*	Aquesta LAN
172.16.*.*	Aquesta LAN
192.168.*.*	Aquesta LAN
0.*.*.*	Usos especials
169.254.*.*	Usos especials
255.255.255.255	Usos especials

Exercici 8.1

- Quantes adreces IPv4 diferents hi ha (incloent-hi les reservades i privades)?
- Són suficients ara i en el futur?
- És problemàtic que una adreça com [192.168.0.1](#) l'estiguin fent servir simultàniament milers (probablement milions) d'ordinadors a Internet ara mateix?
- Quina d'aquestes correspon a adreces link.local?

8.1.2 Màscares de xarxa

Els 32 bits d'una adreça IP es poden dividir en dues parts mitjançant una màscara de xarxa:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
Network ID																Device ID															

La primera part identifica la LAN o el grup de LANs a què pertany un dispositiu. La segona part identifica un únic dispositiu dins de la LAN. La màscara de xarxa determina quants bits s'assignen a cada part.

A la figura anterior, la màscara de xarxa assigna 10 bits a l'identificador de xarxa i 22 al del dispositiu. Aquesta màscara pot anar des de 0 (tots els bits són identificador de dispositiu) fins a 32 (tots els bits són identificador de xarxa).

Les màscares de xarxa es presenten a les persones de dues maneres principals:

- $/N$, on N és el nombre de bits assignats a l'identificador de xarxa. És el que es fa servir en el format CIDR.
- El format quad decimal ($A.B.C.D$) de N bits a 1, seguits de $(32 - N)$ bits a 0.

Exercici 8.2 La màscara de xarxa de l'exemple anterior es pot expressar en binari (tot i que no és gaire pràctic):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0	0
10 bits										$32 - N = 22$ bits																					

- Expressa la màscara de xarxa de l'exemple anterior en els formats CIDR i quad decimal.
- Quina és la teva IP i la teva màscara de xarxa? Expressa-les de les mateixes dues maneres.



Les màscares de xarxa també es fan servir a IPv6, però amb un valor màxim de N de 128 en lloc de 32.

8.1.3 Blocs d'adreces IP

Les màscares de xarxa es combinen amb adreces IP per definir blocs d'adreces IP que comencen pels mateixos N bits (on N és la màscara de xarxa). Per exemple, $192.168.3.0/24$ (en format CIDR) i $192.168.3.0/255.255.255.0$ fan referència al bloc d'adreces entre $192.168.3.0$ i $192.168.3.255$, ambdues incloses ($192.168.3.*$ en la notació informal de la taula anterior).

Exercici 8.3 Els blocs IP poden ser més petits, iguals o més grans que una xarxa IP. Dona exemples en què un bloc IP faci referència a:

- Una única IP.
- La meitat de les adreces IP de la xarxa $192.168.3.0/24$.
- Totes les adreces de la xarxa $192.168.3.0/24$, més 256 adreces més.
- Totes les IP possibles.

Exercici 8.4

- Es pot expressar qualsevol màscara de xarxa com una adreça IP?
- Enumera totes les màscares de xarxa possibles.

Quan es fa referència a un bloc d'adreces (com una LAN), l'adreça IP d'aquesta combinació IP + màscara de xarxa ha de tenir tots els bits de l'identificador de dispositiu a 0. D'aquesta manera, $192.168.3.0/24$ fa referència correctament a una xarxa, però $192.168.3.1/24$ fa referència, en canvi, a un dispositiu (el que té la IP $192.168.3.1$) d'aquesta xarxa ($192.168.3.0/24$).

8.1.4 LANs i màscares de xarxa

Les màscares de xarxa *no* han de ser necessàriament múltiples de 8. Considera l'exemple següent, en què

es fa servir $192.168.76.1/19$ (la IP + màscara d'un dispositiu) per obtenir la seva adreça de xarxa

($192.168.64.0/19$):

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31	
192								168								76								1							} Device's IP	
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	1	1	0	0	0	0	0	0	0	0	0		1
&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&	&		
1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	} Netmask
=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=		
																	=	=	=	=	=	=	=	=	=	=	=	=	=	=	=	
1	1	0	0	0	0	0	0	1	0	1	0	1	0	0	0	0	1	0	0	0	0	0	0	0	0	0	0	0	0	0	0	} Network IP
192								168								64								0								

Exercici 8.5

- A l'exemple anterior, en realitat només calia transformar un valor decimal a binari, i un valor decimal a binari. Quin?
- Això és sempre cert quan es converteixen combinacions IP+màscara de xarxa a adreces de xarxa?
- Expressa la màscara de xarxa en format quad decimal.

Donat un bloc d'adreces IP + màscara de xarxa i l'adreça IP d'un dispositiu, és directe determinar si la IP d'aquest dispositiu pertany al bloc (si i només si coincideixen els primers N bits, on $/N$ és la màscara de xarxa CIDR).

Exercici 8.6 Determina si cadascuna de les adreces següents pertany a la mateixa xarxa que el dispositiu 123.45.67.89/14:

- 23.45.67.89
- 123.45.203.25
- 123.43.255.255
- 0.45.67.1

Una xarxa IP (una LAN) és un bloc d'adreces, però no totes aquestes adreces es poden assignar a dispositius. Sempre hi ha dues adreces reservades:

- L'adreça amb tots els bits de node a 0: identifica la mateixa LAN.
- L'adreça amb tots els bits de node a 1: identifica el broadcast (difusió) dins de la LAN.

Exercici 8.7

- Enumera totes les adreces que es poden assignar a dispositius a 192.168.54.0/29.
- Indica l'adreça de broadcast d'aquesta xarxa.

La màscara de xarxa $/N$ determina el nombre de dispositius que es poden connectar alhora a una xarxa: $2^{32-N} - 2$. Quan es diu que una LAN és "petita" o "més gran", la mida fa referència a aquest nombre d'adreces de dispositiu disponibles.

A la inversa, el nombre M de dispositius que necessitem connectar a la mateixa LAN determina la mida de LAN més petita (la màscara $/N$ més gran) on caben tots aquests dispositius alhora:

Màscara /N	Total adreces 2^{32-N}	Màx. dispositius $2^{32-N} - 2$
/30	4	2
/29	8	6
/28	16	14
/27	32	30
/26	64	62
/25	128	126
⋮	⋮	⋮



Tots els **routers** (encaminador) necessiten una adreça IP vàlida per cada LAN a la qual estan connectats. Per tant, els routers **sí** que compten per al nombre màxim de dispositius connectats a una LAN.

Exercici 8.8

- Quina és la mida de xarxa mínima on caben 256 dispositius connectats?
- Quina és la màscara /N més llarga que permetria connectar-se a tothom de la teva ciutat?
- Quantes adreces conté una xarxa /0?
- Quantes xarxes /0 poden existir? Tenen nom?

Exercici 8.9 El fragment de codi següent pren una adreça IP i en mostra els 4 bytes expressats en quad decimal i en binari. Amplia (o substitueix) el codi perquè puguis:

- Transformar 32 bits en una cadena quad decimal.
- Mostrar una màscara de xarxa en format d'adreça IP i en format CIDR.
- Donades la IP i la màscara de xarxa d'un dispositiu, determinar si una altra IP pertany a la mateixa xarxa.

  snippets/ipcalculator.py

```

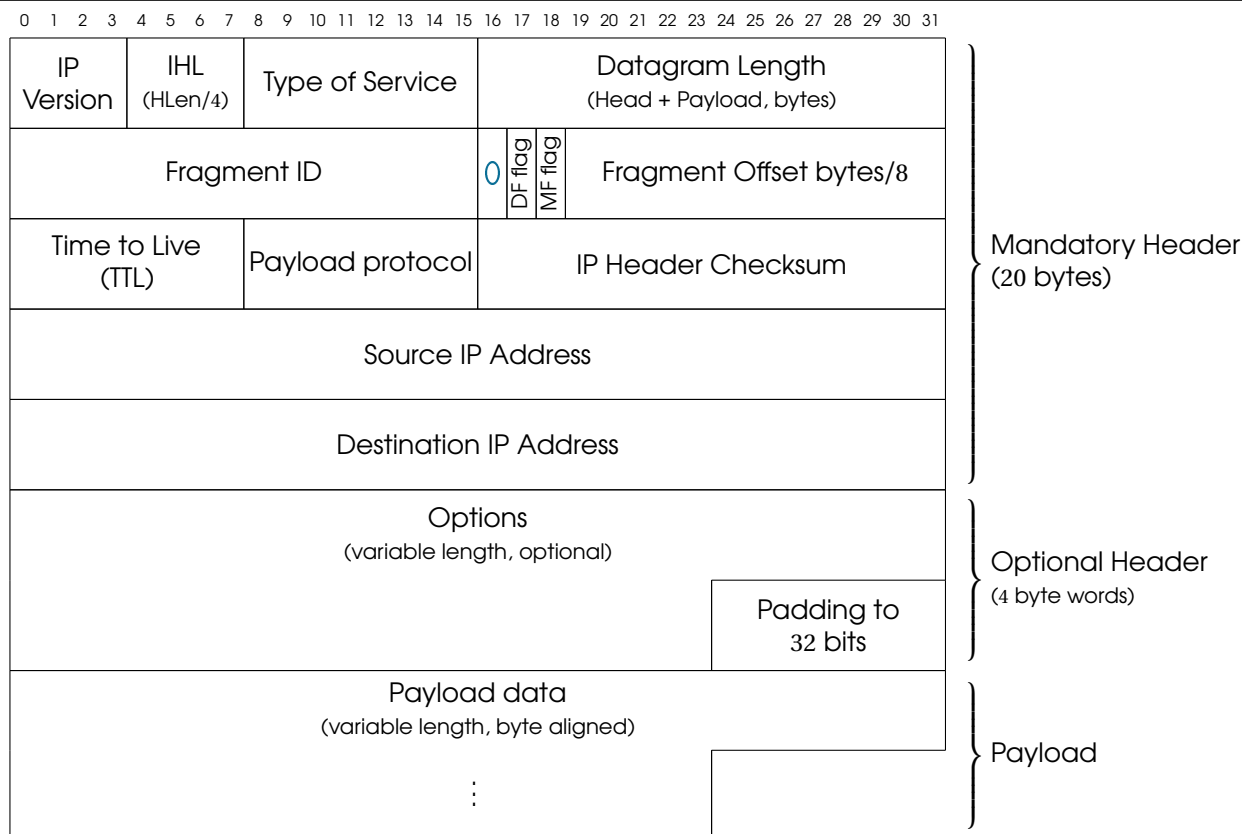
1 def print_ip_binary(ip: str):
2     parts = [int(s) for s in ip.split(".")]
3     print(" ".join(f"{part:8d}" for part in parts))
4     print(" ".join(f"{part:08b}" for part in parts))
5
6 print_ip_binary(ip = "192.168.0.1")

```

8.2 IP – Internet Protocol

8.2.1 Format del paquet

IP defineix un format de **paquet** amb una **capçalera** que té com a mínim 20 bytes. S'hi poden incloure parts opcionals, sempre que la mida total de la capçalera sigui múltiple de 32 bits (4 bytes). El payload pot ser buit, de manera que la mida mínima d'un datagrama és de 20 bytes.



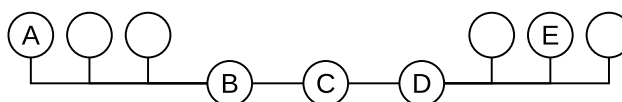
Exercici 8.10

- Com es pot calcular la longitud del payload a partir de la capçalera?
- Quina és la longitud màxima del payload?
- Quins són els valors possibles dels flags (indicador) de la capçalera IP?
- Què és més llarg, una adreça IP o una adreça MAC Ethernet?
- És possible enviar exactament 17 bits de dades de payload en un datagrama IP?
- Com podem saber si un datagrama encapsula dades TCP o UDP?
- Per què la part opcional de la capçalera ha de ser múltiple de 32 bits?

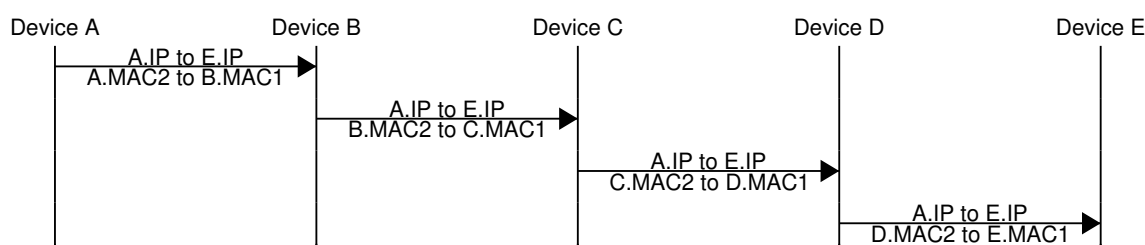
8.2.2 Funcionament

Quan un dispositiu A vol enviar un datagrama IP a un dispositiu B, les IP d'origen i de destinació de la capçalera queden fixades i no canvien per a aquest datagrama.

Si els dispositius A i B pertanyen a la mateixa LAN, el datagrama s'envia directament (p. ex., encapsulat en un frame (trama) Ethernet). Tanmateix, IP pot arribar molt més lluny. Considera l'escenari següent, en què A vol comunicar-se amb E.



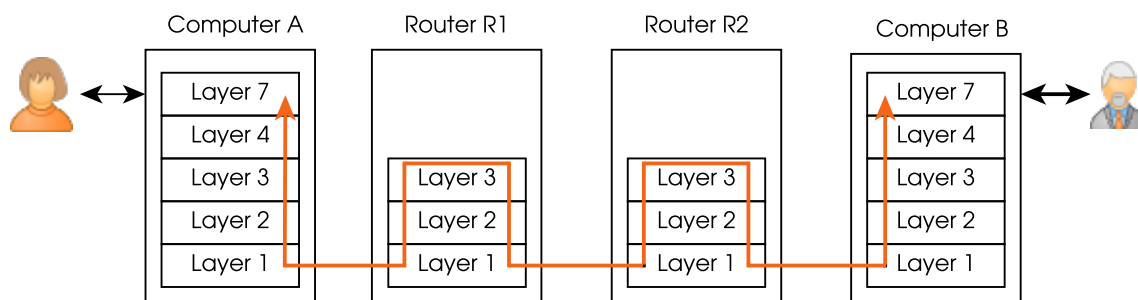
Aquí, els dispositius d'origen i de destinació pertanyen a LANs diferents. Primer, el datagrama es passa al router de l'origen. Després, cada router intermedi el passa al següent fins que el datagrama arriba a l'últim router, que està connectat a la LAN de destinació. Finalment, aquest últim router passa el datagrama a la destinació.



Les adreces IP d'origen i de destinació no canvien al llarg del trajecte. En canvi, l'adreça MAC canvia a cada salt entre LANs.

8.3 Encaminament IP

Els routers són els dispositius encarregats de reenviar els datagrames IP mirant l'adreça IP de destinació de cadascun, present a la capçalera IP. Els routers no necessiten desencapsular les capçaleres de les capes 4 i superiors. Per tant, les seves piles de xarxa només necessiten arribar fins a la capa 3 (i per això de vegades se'ls anomena dispositius de capa 3).

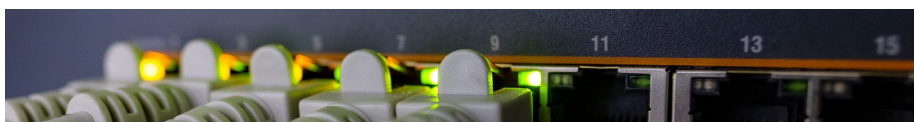


Quan un router detecta un frame de xarxa F en una de les seves interfícies (targetes de xarxa), intenta reconèixer les parts següents d'un datagrama.

Layer 2 header	IP header	Datagram payload
----------------	-----------	------------------

Si les reconeix, el router processa el datagrama de la manera següent:

- Es descarta tot el frame F tret que la MAC de destinació (a la capçalera de capa 2) sigui la de la interfície del router que ha detectat el frame.
- El router mira l'adreça IP de destinació (a la capçalera IP de F) i la seva taula d'encaminament per decidir el següent salt del datagrama, és a dir, el dispositiu al qual aquest router passarà el datagrama, i quina interfície farà servir (vegeu la secció 8.3 més avall).
- El router crea i envia un frame F' fent els dos canvis següents a F :
 - La capçalera de capa 2 de F' canvia completament:
 - La MAC de destinació és la del següent salt, i la MAC d'origen és la del router que reenvia el datagrama. La IP del següent salt la dona la taula d'encaminament (vegeu la secció 8.3), i la seva MAC s'obté a partir d'aquesta IP amb ARP (secció 8.6).
 - La MAC d'origen del frame F' és diferent de la MAC de destinació del frame rebut F si el router ha rebut F per una interfície (p. ex. `eth7`) i el reenvia per una altra (p. ex., `eth9`).



- La capçalera IP de F' difereix de la de F en el següent:
 - El camp TTL (temps de vida) es decrementa en 1. Si el TTL arriba a 0, el datagrama es descarta en lloc de reenviar-se.
 - Quan el camp Options és present, sovint s'actualitza.
 - El checksum (suma de verificació) de la capçalera s'actualitza en conseqüència.



Fixa't que les adreces IP i el payload es copien sense modificar quan es reenvia un datagrama.

Quan un router rep un datagrama IP per reenviar i aquest router està connectat a la mateixa LAN que l'adreça IP de destinació, el router fa un lliurament directe (DD). En cas contrari, el

datagrama es reenvia al següent router en un lliurament indirecte (ID). Un router sap si ha de fer DD o ID, així com l'adreça IP de destinació (i d'origen) que ha de fer servir, gràcies a la seva taula d'encaminament.

Totes les taules d'encaminament tenen com a mínim les columnes següents:

- **Bloc d'adreces de destinació:** una o més adreces IP que es poden expressar com una combinació IP + màscara de xarxa. Una fila de la taula només es pot aplicar a un datagrama si la seva IP de destinació és dins del bloc d'adreces de destinació d'aquella fila:
 - Si només s'aplica una fila de la taula, aquesta fila és la que es fa servir per encaminar.
 - Si s'aplica més d'una fila a un datagrama, es fa servir la fila amb el bloc d'adreces de destinació més petit (el més específic).

Un bloc de destinació default indica el bloc `0.0.0.0/0`. Com que és el bloc més gran possible, només s'aplica si no s'aplica cap de les altres files. L'entrada default no és obligatòria.

- **Gateway/via** (passarel·la): si la fila correspon a un lliurament indirecte, aquesta columna conté l'adreça IP del següent router. En cas contrari, aquesta columna conté `0.0.0.0` per indicar un lliurament directe.
- **Interfície:** nom intern de la interfície de xarxa (p. ex., de la targeta Ethernet) que s'ha de fer servir per lliurar el datagrama. Els sistemes operatius poden fer servir la convenció que vulguin, sempre que les interfícies es puguin identificar de manera única a la taula. Els noms habituals comencen per `eth`, `wlan`, `atm`, i, més recentment, `en` i `wl`.



Tots els dispositius tenen la seva pròpia taula d'encaminament, no només els routers.

Exercici 8.11 A continuació es mostra la taula d'encaminament d'un router petit.

Bloc d'adreces destí	gateway/via	Interfície
<i>default</i>	192.168.0.1	eth0
192.168.0.0/24	0.0.0.0	eth0
172.16.0.4/32	0.0.0.0	eth1
10.49.4.0/28	10.49.4.1	eth2
10.49.4.3/32	10.49.4.2	eth2

- Dibuixa un mapa de xarxa coherent amb la taula d'encaminament que contingui aquest router. Per a les interfícies del router, indica'n el nom i algunes adreces IP i de capa 2 vàlides.
- Com reenviaria aquest router un datagrama adreçat a una adreça IP pública?
- Dona una adreça IP de destinació que es processaria amb la fila `10.49.4.0/28`, i una altra que es processaria amb la fila `10.49.4.2/32`. Pensa un cas d'ús en què tingui sentit tenir aquestes dues regles.

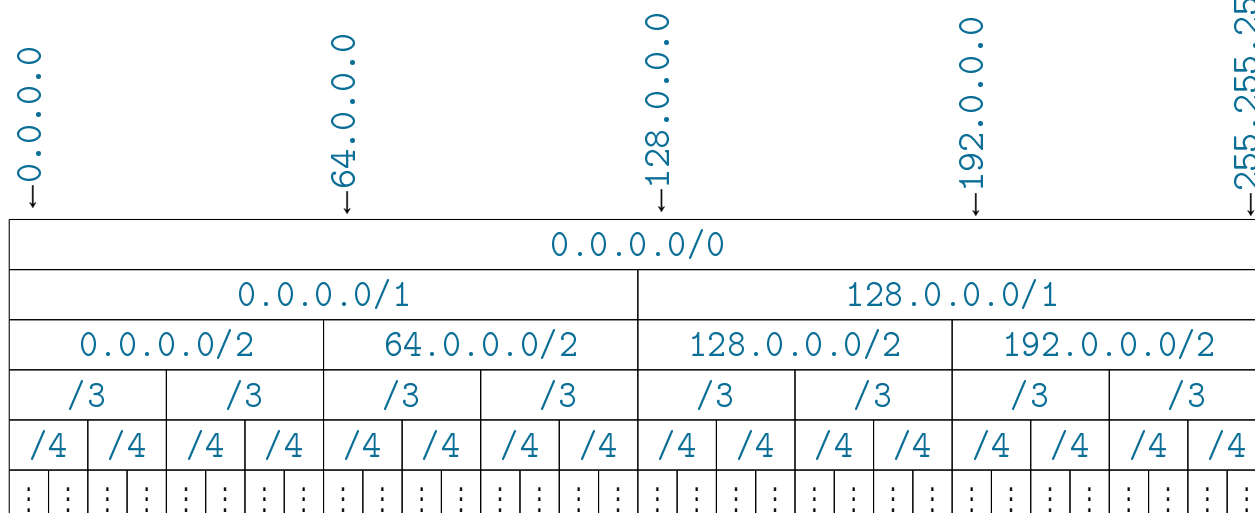
Perquè Internet funcioni, els routers que interconnecten LANs a escala global han d'estar ben configurats. De vegades no és així, i això pot provocar errors d'encaminament. Dos problemes importants que et pots trobar són:

- "Network is unreachable": cap de les files de la taula d'encaminament fa servir un bloc d'adreces que contingui l'adreça de destinació del datagrama. El router no pot saber quin ha de ser el següent salt, de manera que el datagrama no es reenvia. Això provoca un error "ICMP Destination Unreachable" (vegeu la secció 8.7).
- "Time to Live exceeded": cada vegada que es reenvia un datagrama, el seu TTL es decrementa, tal com s'ha dit a la secció 8.2.1. Si diversos routers estan configurats de manera que el datagrama segueix algun tipus de cicle (bucle), el datagrama es reenviaria infinitament. Quan el TTL arriba a 0, el datagrama es descarta i es produeix aquest error. Això provoca un error "ICMP Time Exceeded" (vegeu la secció 8.7).

Exercici 8.12 Es pot produir l'error "Network unreachable" si un router té una entrada *default*?

Exercici 8.13 Proposa un exemple de LANs interconnectades i especifica les taules d'encaminament necessàries perquè un datagrama segueixi un cicle i provoqui l'error "TTL exceeded".

8.4 Divisió en subxarxes IP



Les LANs es poden considerar blocs d'adreces (p. ex., expressats en format CIDR) amb dues adreces reservades. A les xarxes accessibles directament des d'Internet se'ls ha d'assignar un bloc vàlid prou gran per connectar-hi tots els dispositius necessaris. Per garantir que no hi hagi dos dispositius a Internet amb la mateixa IP pública, no pot haver-hi dues LANs que continguin la mateixa adreça (tret que l'adreça sigui privada).

Les 2^{32} adreces disponibles a IPv4 es divideixen en blocs mitjançant màscares de xarxa. Els blocs més grans es poden dividir per la meitat augmentant en 1 la longitud de la màscara de xarxa, com es veu a la figura anterior. Per exemple, el bloc inicial 0.0.0.0/0 (totes les adreces) es pot dividir en dos blocs /1, cadascun amb la meitat de les adreces IPv4 (2^{31} – encara massa gran per a qualsevol LAN real). Aquest procés continua de manera independent per a cada bloc resultant, i s'atura quan la mida de LAN resultant és la desitjada.

Donat un bloc d'adreces /N (p. ex., d'una única LAN existent), potser volem dividir-lo en subblocs i tenir, en lloc seu, diverses LANs més petites (subxarxes). A les subxarxes només podem fer servir adreces del bloc original, però es poden reservar blocs per a un ús futur:

Exercici 8.14 Ets l'administrador de la xarxa 192.168.0.0/24 d'un laboratori de recerca. T'agradaria dividir-la en les tres subxarxes destacades a continuació.

- Fes un diagrama de les tres xarxes, incloent-hi un router connectat a totes tres mitjançant tres interfícies diferents.
- Dona la taula d'encaminament mínima d'aquest router (aquestes xarxes no estan connectades a Internet).
- Quants dispositius es poden connectar en total a aquestes tres xarxes, incloent-hi els routers?
- Quantes adreces de la xarxa original queden sense assignar?
- Quina és la LAN més gran que podem crear amb les adreces sense assignar?

192.168.0.0/24							
192.168.0.0/25				192.168.0.128/25			
192.168.0.0/26	192.168.0.64/26	192.168.0.128/26	192.168.0.192/26	192.168.0.128/26	192.168.0.192/26	192.168.0.128/26	192.168.0.192/26
/27	/27	/27	/27	/27	/27	/27	/27

El procés de crear diverses LANs més petites a partir d'un únic bloc d'adreces s'anomena divisió en subxarxes (*subnetting*). El procés invers, combinar diverses LANs en una de més gran, s'anomena agregació en superxarxes (*supernetting*).

Dues LANs $/N$ es poden combinar si i només si les seves adreces de xarxa (la seva IP sense màscara) comparteixen els primers $N - 1$ bits. Aquest procés pot continuar fins arribar a una xarxa $/0$, sempre que siguem propietaris de les adreces IP de les dues meitats de cada superxarxa.

Exercici 8.15

- Quina altra xarxa pots combinar amb $192.168.0.0/24$ per obtenir una xarxa $/23$ N_{23} ? Quina és la seva adreça de xarxa?
- N_{23} forma part d'exactament una xarxa $/22$, N_{22} . Enumera totes les xarxes $/24$ que hauries de tenir per combinar-les en N_{22} .

La divisió en subxarxes sovint es basa en quantes adreces IP han d'estar disponibles a cada subxarxa. Aquesta mida determina el nombre de bits de la màscara de la subxarxa, com s'ha vist a la secció 8.1.4. Es poden afegir restriccions addicionals, p. ex., fer que les adreces IP d'una subxarxa siguin les més altes, o més baixes que les d'altres subxarxes.



Considera per exemple la xarxa $192.168.0.0/16$. Volem dividir aquesta xarxa en subxarxes i crear la xarxa més petita que pugui connectar 100 dispositius simultàniament.

Com s'ha vist a la secció 8.1.2, això determina que si s'ha de crear una subxarxa a partir d'un per contenir com a mínim 100 nodes, cal dedicar com a mínim 7 bits d'adreça ($2^6 - 2 = 62$, $2^7 = 126$) als identificadors de dispositiu i els $32 - 7 = 25$ restants a identificar la xarxa, és a dir, la subxarxa pot ser tan petita com una $/25$.

0 1 2 3 4 5 6 7 8 9 10 11 12 13 14 15 16 17 18 19 20 21 22 23 24 25 26 27 28 29 30 31	
192.168	Free bits Device ID

Els 9 bits restants (posicions 16 a 24) es poden fixar lliurement si no tenim cap altra restricció. Triar un valor per a aquests bits vol dir triar una de les 2^9 xarxes de mida $/25$ contingudes a la xarxa original, $192.168.0.0/16$. Posar aquests bits lliures a 0 dona el rang d'IP més baix, i posar-los a 1, el més alt.

Quan s'han de crear diverses subxarxes, és essencial verificar que cap subxarxa no estigui continguda en una altra. Si no, la mateixa adreça IP pertanyeria a dues LANs, i l'encaminament seria impossible. Per exemple, si s'han de crear tres subxarxes per a 100, 500 i 200 dispositius a partir de $192.168.0.0/16$, amb rangs d'adreces en aquest ordre, una divisió possible seria:

0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15	16	17	18	19	20	21	22	23	24	25	26	27	28	29	30	31
192.168																0	0	0	0	0	0	0	0	0	Device ID		→ 192.168.0.0/25				
192.168																0	0	0	0	0	0	1	Device ID					→ 192.168.2.0/23			
192.168																0	0	0	0	0	0	1	Device ID					→ 192.168.1.0/24			

Recorda que l'adreça de xarxa de cada subxarxa resultant té tots els bits de l'identificador de dispositiu a 0, i que aquesta adreça normalment es presenta en format quad decimal.

Exercici 8.16

- A l'exemple anterior, per què els bits lliures de la segona subxarxa són 0000001 i no 0000000?
- Repeteix l'exercici assignant les adreces més altes possibles a la tercera subxarxa, després les següents adreces més altes possibles a la segona, i les adreces més baixes possibles a la primera.
- Quina és la IP de broadcast de cada subxarxa?

8.5 Fragmentació IP

Abans que un dispositiu (incloent-hi els routers i els ordinadors normals) pugui enviar un datagrama per una LAN, ha de verificar que la seva longitud en bytes no sigui més gran que la MTU d'aquesta LAN. Si ho és, el datagrama s'ha de fragmentar en dos o més datagrames més petits que sí que càpiguen en la MTU de la LAN.

Els fragments més petits són datagrames complets amb capçalera i payload. Per tant, més fragments impliquen una sobrecàrrega més gran. Cadascun d'aquests fragments s'envia i es reenvia individualment com qualsevol altre datagrama, i pot ser que s'hagi de tornar a fragmentar si passa per una altra LAN amb una MTU encara més petita.

Quan s'ha de fragmentar un datagrama, la pila de xarxa segueix els passos següents:

1. Es comprova el flag DF (don't fragment) del datagrama original. Si $DF=1$ però cal fragmentar, el datagrama no es fragmenta i es produeix un error.
2. Les dades del payload es divideixen en blocs de mida com a màxim S , de manera que $S + H = MTU$ (H és la longitud de la capçalera del datagrama, normalment 20 bytes). Només l'últim bloc pot tenir una mida inferior a S . Això garanteix que cap fragment no superi la MTU, que és precisament l'objectiu de la fragmentació.
3. La capçalera del datagrama original es copia a la dels fragments amb les modificacions següents:
 - El camp Datagram ID pren el mateix valor en tots els fragments d'un datagrama. El sistema operatiu s'assegura de triar un ID que la seva pila de xarxa no hagi fet servir recentment. El camp Datagram ID s'ignora en els datagrames no fragmentats.
 - El camp Offset (desplaçament) de cada fragment indica la posició (índex de byte) del seu primer byte de dades de payload dins del payload del datagrama abans de la fragmentació. Aquest camp no s'expressa en bytes, sinó en paraules de 8 bytes. Donat un offset en bytes, el camp Offset es calcula dividint per 8. Per tant, els offsets dels fragments només poden ser múltiples de 8.
 - El flag MF (more fragments) es posa a 1 (activat) en tots els fragments excepte l'últim. Això ha de ser cert fins i tot si un o més fragments es tornen a fragmentar en una LAN posterior. Així, quan es fragmenta un datagrama D amb $MF=1$, cap dels fragments resultants
4. Els datagrames fragment s'envien de la mateixa manera que s'enviaria un datagrama normal. Els fragments es poden tornar a fragmentar, però només els torna a acoblar el host (equip connectat a la xarxa) de destinació.

Exercici 8.17 Donats l'Offset i el flag MF, com podem distingir un datagrama que no s'ha fragmentat d'un fragment de datagrama?

Exercici 8.18 Per què seria problemàtic posar $MF=0$ a l'últim datagrama produït en fragmentar un datagrama amb $MF=1$?

Exercici 8.19 Un dispositiu ha d'enviar un datagrama amb una capçalera IP de 20 bytes i un payload de 2170 bytes. El dispositiu està connectat a la LAN1, i el datagrama ha de travessar la LAN2 i la LAN3 fins que arriba a la LAN4 de destinació. Les MTU d'aquestes LANs són les següents:

LAN	LAN1	LAN2	LAN3	LAN4
MTU	1500	710	4096	864

- Indica l'offset i la longitud del payload (tots dos en bytes), així com els flags MF i DF dels 4 datagrames que arriben a la LAN4 com a resultat d'enviar el datagrama (suposant que no hi ha pèrdues ni duplicacions).
- Pot el dispositiu d'origen conèixer les MTU de les LANs intermèdies i de destinació?

Exercici 8.20 Els datagrames que contenen fragments poden arribar duplicats i/o desordenats. Com hauria de gestionar aquests datagrames la pila de xarxa receptora?

8.6 ARP – Address Resolution Protocol

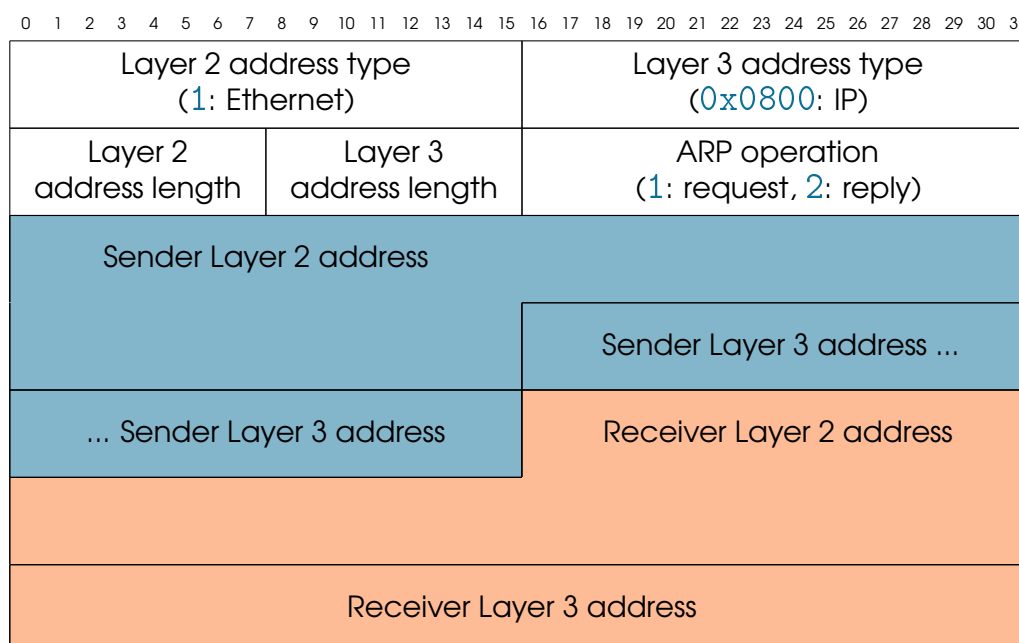
L'Address Resolution Protocol (ARP) el fa servir la capa 3 (IP) per obtenir l'adreça MAC (física) de destinació corresponent a una adreça IP coneguda.

Els paquets ARP s'envien com a payload de frames de capa 2 (p. ex., Ethernet). Els paquets ARP *no* són datagrames i *no* contenen capçaleres IP.



La IP que consulta la capa 3 és la del router per al lliurament indirecte i la del dispositiu de destinació per al lliurament indirecte. Recorda també que la IP de destinació del datagrama no canvia durant el reenviament.

8.6.1 Format del paquet



El protocol ARP *no* fa servir capçaleres IP.



"Sender" i "receiver" s'han d'interpretar com "... d'aquest missatge ARP".

Exercici 8.21

- Quina és la finalitat dels 4 primers camps d'un paquet ARP?
- Quin efecte tenen en els camps restants?

Exercici 8.22 Quantes adreces vàlides apareixen en un frame Ethernet que transporta una petició ARP? I en una resposta ARP?

8.6.2 Funcionament

Quan un node **(A)** necessita traduir una adreça IP a una adreça MAC, envia un frame de broadcast (MAC de destinació **ff:ff:ff:ff:ff:ff** que conté un paquet ARP. En aquest paquet ARP, l'operació es posa a REQUEST (1), la IP de destinació és la que es vol traduir, i la MAC de destinació es posa a **00:00:00:00:00:00**.

Quan una pila de xarxa **(B)** veu un paquet de petició ARP, envia un paquet de resposta ARP només si la IP que té configurada és l'adreça IP de destinació de l'ARP. Aquesta resposta és idèntica a la petició amb aquests canvis:

- El codi OP del paquet ARP es posa a REPLY (2).
- "Origen" ara fa referència a **(B)** (a la petició feia referència a **(A)**), i "destinació", a **(A)**.
- En una resposta s'omplen totes 4 adreces del paquet (no poden ser **00:00:00:00:00:00**).

Exercici 8.23 Quan s'executa l'ordre `ip route` en un node, la sortida conté:

```
default via 192.168.0.1 dev wlp5s0 src 192.168.0.2 metric 600 [...]
```

Al seu torn, l'ordre `ip link` conté les línies següents:

```
2: enp42s0: <NO-CARRIER,BROADCAST,MULTICAST,UP> mtu 1500 state DOWN [...]
   link/ether d8:43:ae:61:ed:a4 brd ff:ff:ff:ff:ff:ff
3: wlp5s0: <BROADCAST,MULTICAST,UP,LOWER_UP> mtu 1500 state UP [...]
   link/ether 5c:e4:3a:18:97:1b brd ff:ff:ff:ff:ff:ff
```

- El node ha de fer el lliurament indirecte d'un datagrama, però no coneix l'adreça MAC del seu router gateway. Indica el valor de tots els camps (incloent-hi la capçalera Ethernet i el missatge ARP) de la petició ARP que enviarà el node per obtenir la MAC del router.
- Descriu els valors dels camps de la resposta ARP (pots fer servir qualsevol adreça vàlida com a adreça MAC del router).

No és gaire probable que l'adreça MAC associada a una IP canviï sovint. Quan es rep una resposta ARP vàlida, el sistema operatiu actualitza una memòria cau afegint-hi (o actualitzant-hi) les traduccions IP ↔ MAC rebudes. Si el sistema operatiu ha d'enviar un datagrama a aquesta IP abans que caduqui l'entrada de la memòria cau, es fa servir directament l'adreça MAC de la taula sense necessitat de missatges ARP addicionals. L'ordre `ip neighbor` mostra el contingut d'aquesta taula, p. ex.,

```
192.168.0.1 dev wlp5s0 lladdr cc:2d:14:fc:22:60 REACHABLE
```



ARP es pot fer servir per escanejar les IP utilitzades en una LAN. En particular, un node pot fer servir ARP per verificar que ningú més no fa servir la seva adreça IP configurada.

ARP es va proposar en una època en què la seguretat de xarxa encara no era un problema. Com a conseqüència, no proporciona cap mecanisme per verificar que una resposta ARP determinada sigui correcta. Això es pot aprofitar per enverinar la taula de memòria cau ARP d'un dispositiu enviant respostes ARP gratuïtes (no sol·licitades) a la víctima amb associacions IP ↔ MAC falses.

Els sistemes operatius moderns normalment ignoren les respostes ARP no sol·licitades, i els switches (commutador) moderns filtren els missatges ARP que no coincideixen amb les adreces IP i MAC reals. Tot i així, IPv6 abandona ARP en favor del protocol Neighbor Discovery (ND), més complex.

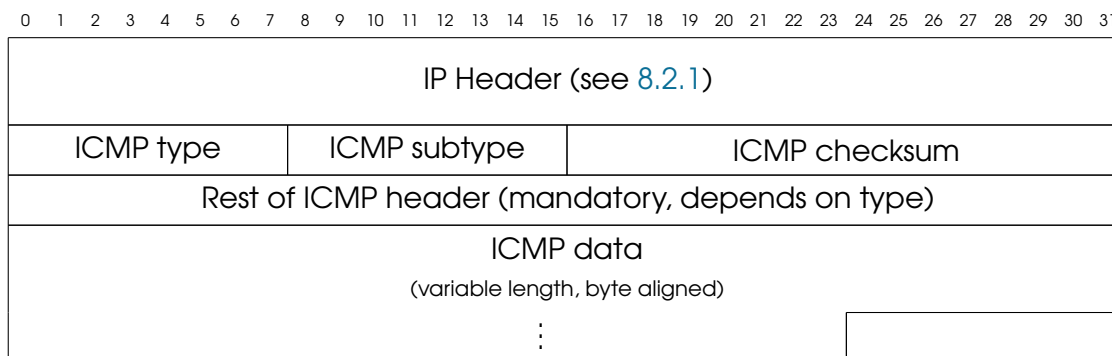
Exercici 8.24 Quin impacte podria tenir enverinar la memòria cau ARP d'una víctima, canviant la MAC emmagatzemada del router gateway per la MAC de l'atacant?

8.7 ICMP – Internet Control Message Protocol

L'Internet Control Message Protocol (ICMP) és un protocol auxiliar (tècnicament part d'IP) que es fa servir per provar i depurar les comunicacions de xarxa (i entre xarxes).

8.7.1 Format del paquet

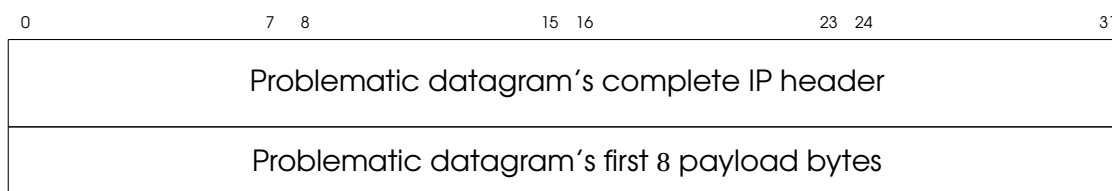
Els missatges ICMP s'encapsulen com a payload de datagrames IP. A la capçalera IP, el protocol del payload es posa a 0x01 ("ICMP").



Les combinacions de tipus/subtipus ICMP més habituals que és probable que et trobis avui dia:

Tipus ICMP	Subtipus ICMP	Error?	Significat
8	0		Missatge ICMP Echo Request (ping).
0	0		Missatge ICMP Echo Reply (resposta ping).
3	0–16	✓	Destination Unreachable. Subtipus: causa.
11	0	✓	Time to Live Exceeded: TTL arriba a 0.

En aquests missatges ICMP d'error, el camp de dades ICMP conté informació sobre el datagrama que ha causat el missatge ICMP d'error:



En els missatges echo request i echo reply, el camp de dades ICMP és arbitrari dins dels límits dels datagrames IP. La fragmentació de datagrames ICMP (p. ex., payloads de ping grans) és possible, però molts routers ignoren els fragments ICMP.

8.7.2 Funcionament

Els missatges d'error ICMP els envien els routers i els hosts normals, desencadenats per altres datagrames:

- La família de missatges "Destination unreachable" s'envia quan un datagrama no es pot lliurar a la seva destinació, p. ex., per una taula d'encaminament mal configurada, un port tancat (vegeu la secció 9), o simplement perquè el host de destinació no està engegat.
- El missatge "TTL exceeded" s'envia quan un router rep un datagrama per a lliurament indirecte amb TTL=1 (vegeu la secció 8.3).

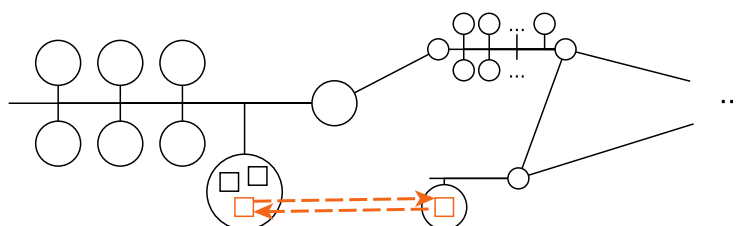
Per evitar quantitats infinites de trànsit, *no* es generen errors ICMP quan els datagrames que els provoquen són:

- Datagrames ICMP (d'error o no).
- Fragments de datagrama, excepte el primer.

Exercici 8.25 Com podem distingir el primer fragment d'un datagrama dels fragments següents?

9. Capa 4: Transport

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



Capacitats

La capa de Transport comunica processos (programes) que s'executen en hosts (equips connectats a la xarxa) potencialment diferents. Aquests processos s'identifiquen mitjançant adreces de capa 4 anomenades ports.

La capa de transport ofereix dues maneres diferents de comunicar-se: sense connexions (UDP) i amb connexions (TCP).

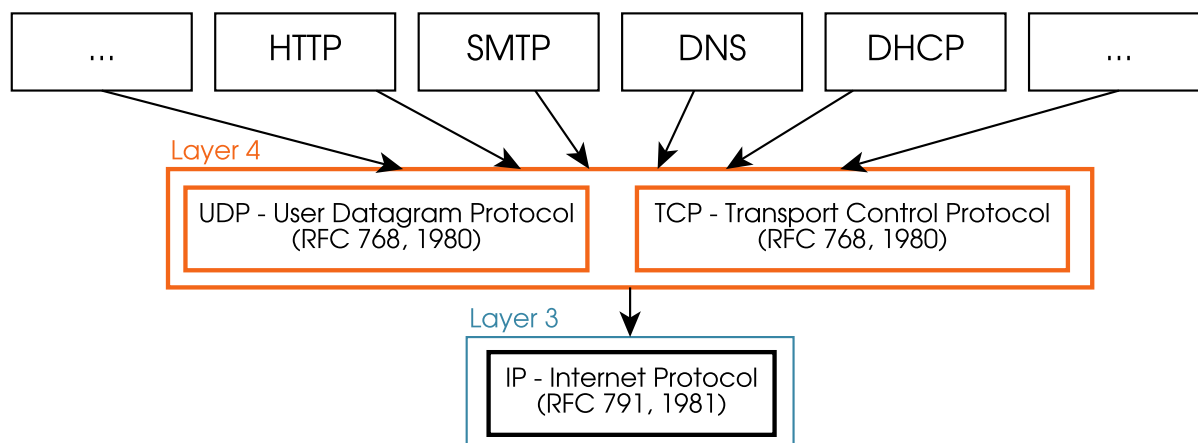
UDP no estableix cap connexió amb el host de destinació perquè la prioritat és enviar unes dades de payload (càrrega útil) tan ràpid com sigui possible. En canvi, UDP envia paquets anomenats datagrames d'usuari directament a la destinació, sense esperar cap mena de confirmació de recepció per part de la capa UDP de la destinació. Les capçaleres UDP són simples i petites, i també mantenen petita la sobrecàrrega d'aquest protocol.

TCP estableix una connexió abans d'enviar cap dada de payload per assegurar-se que tots dos extrems estan preparats. A més, TCP espera la confirmació (acusament de recepció) de cada byte de dades de payload. Si no n'hi ha, les dades es retransmeten després d'un període d'espera, cosa que dona fiabilitat a TCP. TCP també fa servir una manera molt enginyosa de limitar la velocitat d'intercanvi de dades (control de flux) perquè ordinadors ràpids i lents puguin coexistir i comunicar-se amb TCP/IP. Per oferir totes aquestes funcionalitats, TCP és un protocol més complex que UDP, fa servir capçaleres més grans i introdueix una sobrecàrrega més gran.

Exercici 9.1 Quines de les aplicacions següents creus que fan servir UDP i quines TCP?

- Jocs multijugador
- La WWW
- BitTorrent
- Reproducció de vídeo en temps real (*streaming*)

Protocols



La capa de transport és a la part superior de la pila que proporciona el sistema operatiu. Pràcticament totes les aplicacions, i per tant els seus protocols de comunicació, fan servir TCP, UDP (o fins i tot tots dos) per intercanviar dades per la xarxa. A la figura anterior es mostren alguns protocols importants que fan servir TCP o UDP.

Tant TCP com UDP encapsulen les seves PDU en datagrames IP. Generalment es fa servir IPv4, però IPv6 també és una possibilitat quan està disponible.



Ni TCP ni UDP canvien de cap manera quan es fa servir IPv6 en lloc d'IPv4, és a dir, no hi ha cap protocol TCPv6. En canvi, `tcp6` i altres expressions que inclouen TCP i 6 es refereixen a TCP (normal) sobre IPv6.

9.1 Adreçament de la capa 4

Les "adreces" de TCP i UDP s'anomenen ports i són enters sense signe de 16 bits. Per als humans, els ports s'expressen principalment de dues maneres:

- Com a número, p. ex., 22, 80 o 443.
- Com el nom del servei que habitualment fa servir aquest port, p. ex., `sshd`, `http`, `https`.

Quan es rep una PDU de capa 4, el sistema operatiu fa servir el port de la seva capçalera per saber quin dels processos en execució ha de rebre les dades. Al seu torn, els processos s'han de vincular a un port TCP o a un port UDP (o a tots dos) per indicar al sistema operatiu de quin port esperen rebre dades.

Només un procés alhora es pot vincular a un port per enviar dades, però els ports TCP i UDP es tracten de manera independent:

- Si un procés està vinculat al port `TCP/100`, no rebrà el trànsit UDP adreçat al port 100.
- Si un procés està vinculat al port `TCP/100`, un altre procés (o fins i tot el mateix) es pot vincular a `UDP/100`.

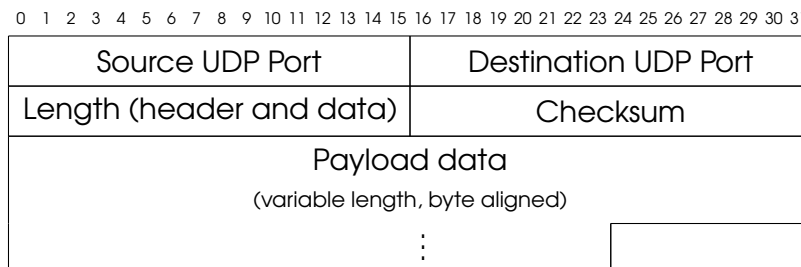
El port número 0 està reservat, de manera que el rang de ports utilitzable és `1-65535`. Els ports `1-1023` només es vinculen a processos amb privilegis (p. ex., un servidor http), mentre que les aplicacions client (p. ex., un client http) només es poden vincular als ports `1024-65535` segons la seva disponibilitat.

Exercici 9.2 Quants servidors diferents es poden executar en una màquina per oferir alguna funcionalitat a altres hosts de la xarxa?

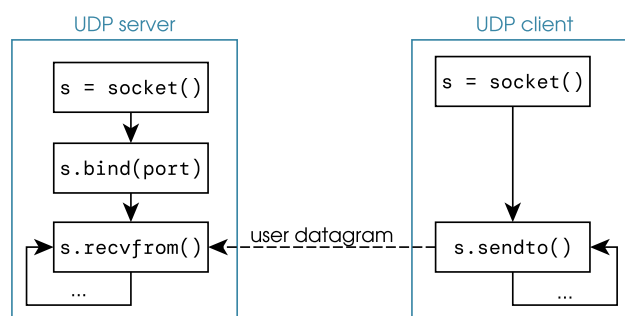
9.2 UDP – User Datagram Protocol

9.2.1 Format dels paquets

Tots els datagrames d'usuari que fa servir UDP tenen una capçalera de 8 bytes i l'estructura següent:

**Exercici 9.3**

- Quina és la mida màxima de payload que es pot enviar en un únic datagrama d'usuari?
- És desitjable la fragmentació quan es fa servir UDP?

9.2.2 Funcionament

Els clients UDP primer creen un socket (punt de connexió) i després envien directament els datagrames d'usuari amb `sendto`. No cal establir ni tancar cap connexió.

Els servidors UDP primer s'han de vincular al port que volen exposar als clients, i després reben cada datagrama en una crida individual a la funció de lectura (`recvfrom`).

La funció `bind` és purament interna al servidor: implica canvis en la seva configuració (la taula que associa números de port i processos), però no genera cap missatge.

Exercici 9.4

- Pot funcionar un servidor UDP sense enviar cap dada per la xarxa?
- Pot un client rebre trànsit UDP sense vincular-se a un port?
- Poden dos hosts diferents vincular-se al mateix port i tot i així intercanviar datagrames d'usuari?

Exercici 9.5 El fragment de codi següent mostra l'esquelet bàsic d'un servidor UDP:

```
.PY snippets/udpserver.py
1 import socket
2
3 port = 4567
4 s = socket.socket(family=socket.AF_INET, type=socket.SOCK_DGRAM)
5 s.bind(('', port))
6 while True:
7     user_datagram = s.recvfrom(1024)
8     print(f"Received UDP on {port=}: {user_datagram!r}")
```

- Executa el servidor i, en un altre terminal, executa `ncat -u localhost 4567` i escriu una mica de text per provar el servidor.
- Implementa l'extrem client de la comunicació i prova'l amb el servidor.
- Quina informació important obtens si canvies `s.recvfrom` per `s.recvmsg`?

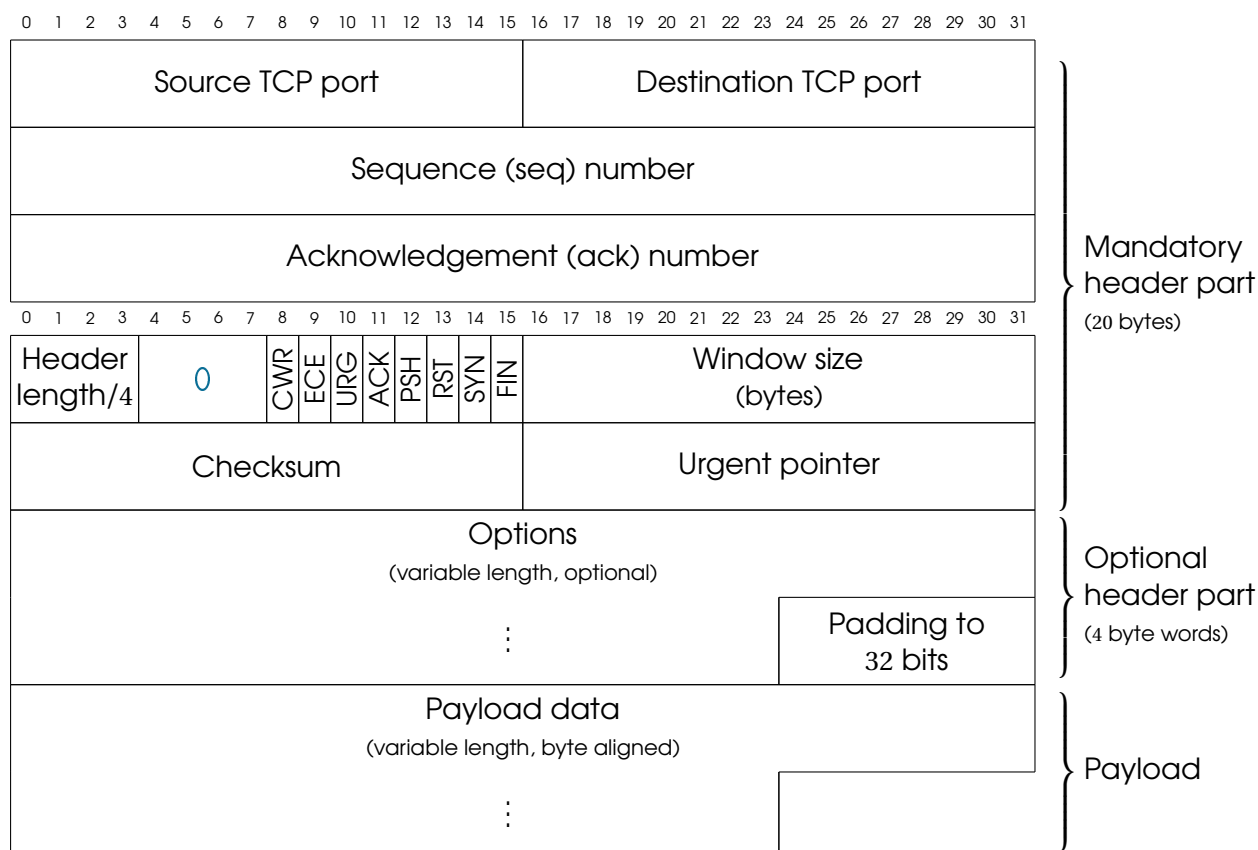


Aquest codi fa servir sockets UDP normals, de manera que no necessita permisos d'administrador ni la capacitat `CAP_NET_RAW`, com passava a l'exercici 7.4, que en canvi requeria accés a sockets de capa 2 (p. ex., Ethernet).

9.3 TCP – Transport Control Protocol

9.3.1 Format dels paquets

Les PDU de TCP s'anomenen segments. La seva capçalera fa com a mínim 20 bytes, però pot ser més llarga quan es fan servir certes opcions:



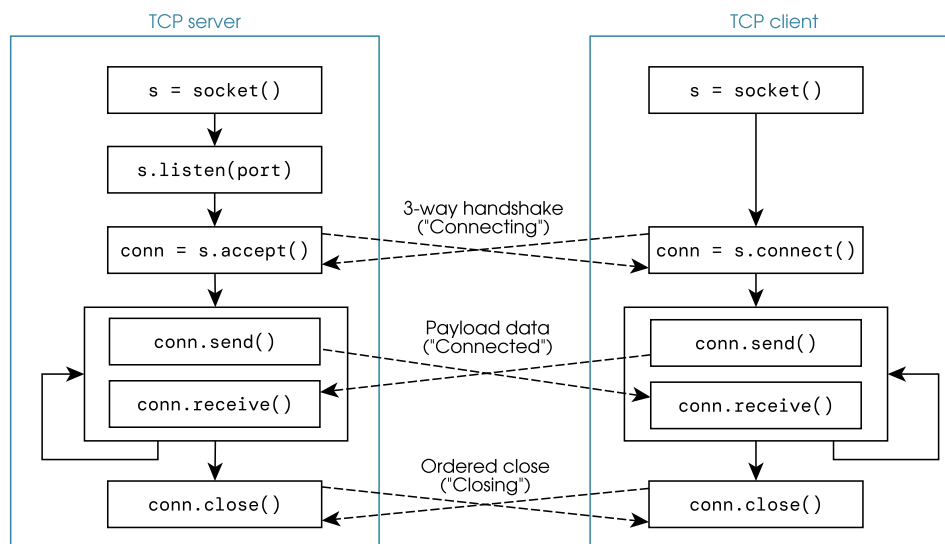
Les capçaleres TCP contenen vuit flags (indicadors) d'1 bit. D'aquests, quatre es fan servir per a funcionalitats bàsiques de TCP que es tracten a les seccions següents. En l'ordre en què apareixen:

- ACK: flag de confirmació – “el meu número ack és vàlid”
- RST: flag de reiniçi – “avorta la connexió”
- SYN: flag de sincronització – “obro el meu extrem de la connexió”
- FIN: flag de finalització – “no enviaré més dades de *payload*”

Exercici 9.6 Com pot saber l'extrem receptor la mida de les opcions i de les dades de payload?

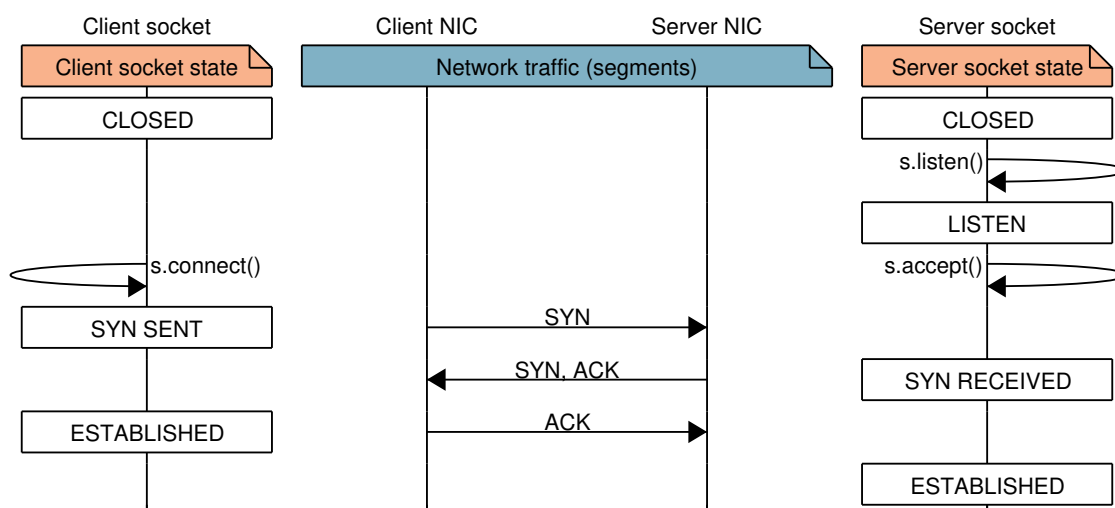
9.3.2 Funcionament

TCP és un protocol amb estat. Els sockets TCP passen per diversos estats durant la comunicació. Cada extrem de la comunicació té el seu propi socket amb el seu propi estat. A grans trets, les connexions es poden classificar com a “Connectant”, “Connexió establerta” o “Tancant la connexió”. Les dades de payload només s'han d'intercanviar mentre s'està a “Connexió establerta”.



Els sockets TCP poden estar en un d'11 estats. Aquests estats, juntament amb dos comptadors de 16 bits (número de confirmació i número de seqüència) permeten la fiabilitat de la comunicació que s'espera de TCP. El significat d'aquests estats i de les seves transicions s'explica als capítols següents.

9.4 TCP: "Connectant"



Quan es crea un socket, comença a l'estat **CLOSED**. L'extrem que actua com a servidor comença a escoltar al port configurat per al servei i passa a l'estat **LISTEN**. En aquest estat no s'intercanvia payload.

Una connexió s'estableix mitjançant un 3-way handshake (negociació inicial) fent servir els flags de capçalera **SYN** i **ACK**. Gràcies a aquest handshake, cada extrem de la comunicació arriba a l'estat **ESTABLISHED** en un moment diferent. Un cop hi arriben, estan preparats per intercanviar dades de payload.



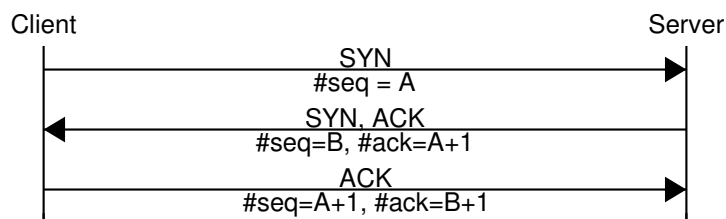
A TCP, tots dos extrems es comuniquen com a peers (parells) en igualtat de condicions: no importa qui ha iniciat la comunicació. Dit això, normalment un extrem fa una "obertura activa" (el client) i l'altre una "obertura passiva" (el servidor).

Cada extrem de la comunicació porta el compte dels seus comptadors de número de confirmació (**#ack**) i de número de seqüència (**#seq**). El valor de **#seq** el tria cada extrem per separat (i normalment a l'atzar): $A, B \in [0, 2^{32} - 1]$.

Durant l'obertura de la connexió, cada extrem envia exactament un segment amb el flag **SYN** activat. Aquests segments intercanvien (sincronitzen) els seus números de seqüència inicials **A** i **B**.

Els segments amb el flag `ACK` activat (tots excepte el primer segment del client) tenen el `#ack` fixat a "el següent `#seq` que s'espera de l'altre extrem". Aquest `#ack` s'ha de llegir com "he rebut correctament tots els segments corresponents fins a `#ack`, sense incloure'l".

Els números `#ack` i `#seq` esperats durant el `3-way handshake` són:



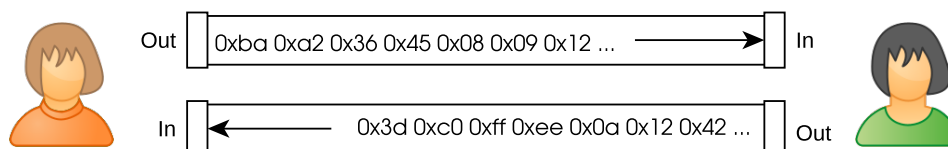
Exercici 9.7

- Com pot saber un extrem que l'altre extrem ha rebut el `#seq` que ha triat?
- Per què el tercer segment té el `#seq` fixat a $A + 1$ i no a A ?
- Quin `#seq` tindrà el següent segment que envii el servidor?

9.5 TCP: "Connexió establerta"

9.5.1 Fluxos i números de seqüència

TCP ofereix la il·lusió d'un flux de dades: dos conductes de dades en les direccions d'entrada i de sortida (una connexió full-duplex). Aquests conductes són fiabls, és a dir, s'hi apliquen mecanismes per evitar la pèrdua, la duplicació i la reordenació de dades.



A cada byte de dades de payload se li assigna un número de seqüència (`#seq`) a la capçalera TCP. El primer byte de dades de payload té `#seq = A + 1`, on A és el número de seqüència inicial establert durant l'obertura de la connexió (vegeu la secció 9.4). A cada nou byte de dades se li assigna el `#seq` següent (sense salts).

#seq	A	A + 1	A + 2	A + 3	A + 4	A + 5	A + 6	A + 7	A + 8	A + 9	...
contingut	SYN	0x48	0x69	0x21	0x0a	0x48	0x6f	0x77	0x20	0x61	...
		'H'	'i'	'!'	'\n'	'H'	'o'	'w'	' '	'a'	...

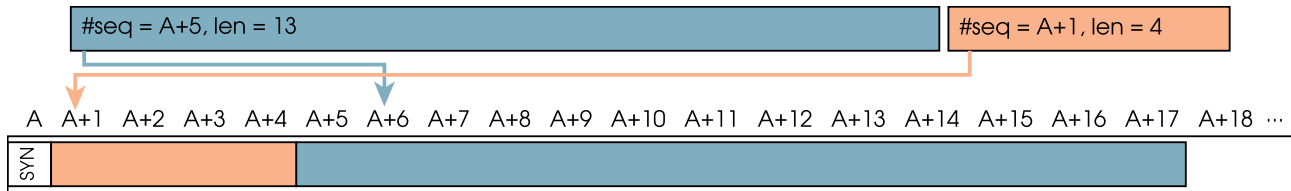
Exercici 9.8 El `#seq` que ve després de $2^{32} - 1$ és 0. Per què?

A TCP, les dades de payload s'envien sempre juntament amb la *primera* posició que ocupen dins del flux i la seva longitud. Per exemple, a partir de les dades que es mostren més amunt, es podrien generar dos segments:

1. `#seq = A + 1`, `len = 4`, dades = 0x48 69 21 0a ('Hi!\n')
2. `#seq = A + 5`, `len = 13`, dades = 0x48 6f 77 20 61 72 65 20 79 6f 75 3f 0a

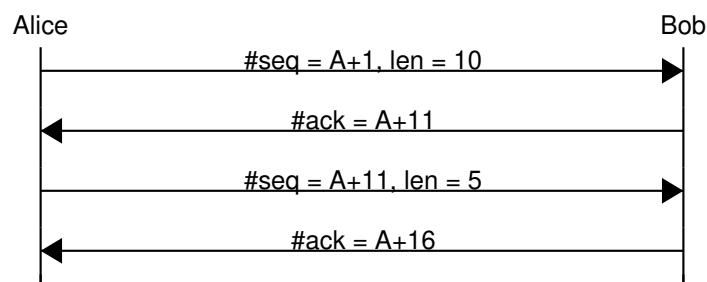
Exercici 9.9 Quin seria el `#seq` del següent segment amb dades de payload noves?

L'extrem receptor d'un segment TCP fa servir aquesta informació per recompondre una còpia de les dades en l'ordre correcte, fins i tot si els segments arriben desordenats. Aquestes dades es passen a la capa superior a través d'un buffer (memòria intermèdia), o directament si el flag TCP `PSH` (push) està activat.



9.5.2 Fiabilitat i retransmissió

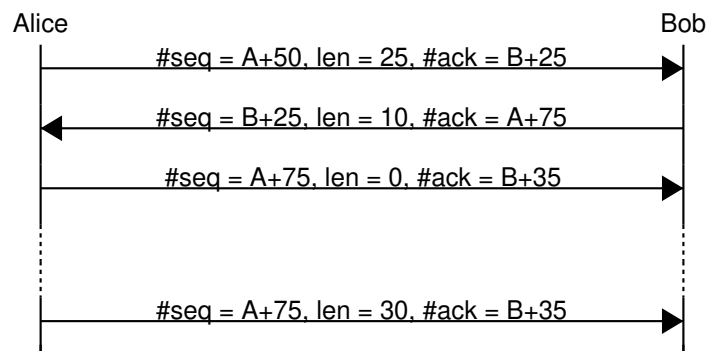
TCP necessita que l'altre extrem confirmi totes les dades transmeses. Cada vegada que es reben dades en un segment TCP, l'extrem receptor envia una confirmació encara que en aquell moment no hagi d'enviar cap dada. El número de confirmació (#ack) de la capçalera TCP serveix per indicar a l'altre extrem que s'han rebut totes les dades fins a #ack (sense incloure'l).



El camp #seq d'una capçalera TCP fa referència als números de seqüència de l'emissor. El camp #ack fa referència als números de seqüència del receptor.

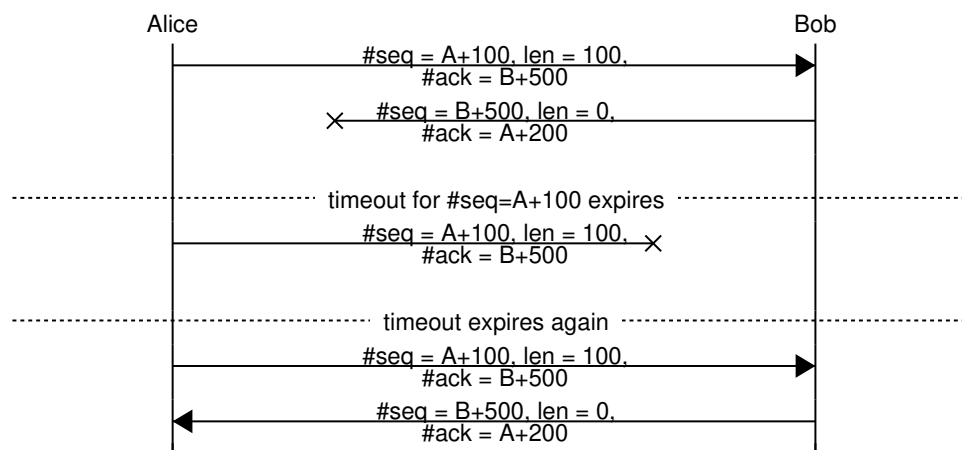
Les dades de payload s'inclouen a les confirmacions TCP sempre que n'hi hagi de disponibles. Recíprocament, el valor més recent de #ack s'inclou sempre que s'envien dades, és a dir, les confirmacions viatgen a cavall de les dades de payload.

Per exemple, si l'Alice envia una petició de 25 bytes, en Bob pot enviar la seva resposta de 10 bytes i la confirmació de les dades de l'Alice en el mateix segment TCP. L'Alice ha de confirmar les dades d'en Bob i, si cal, aviat envia un segment sense dades. L'Alice tornarà a enviar dades més endavant quan calgui.



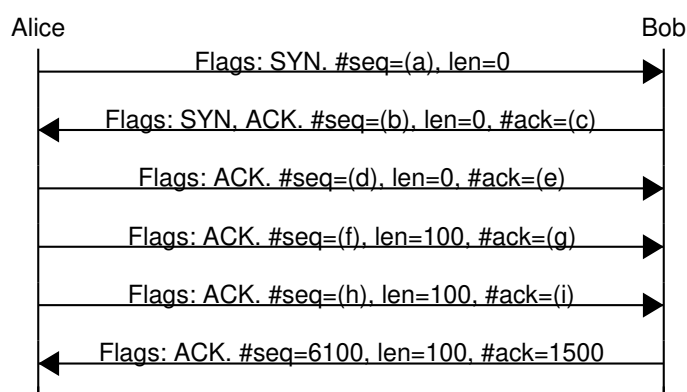
Un segment TCP que no es confirma després d'un cert període d'espera es retransmet. Aquest temps el decideix el sistema operatiu de cada extrem a partir del temps mitjà que es triga a enviar un segment i rebre'n una resposta (el temps d'anada i tornada, RTT).

Les confirmacions s'envien encara que siguin idèntiques a una d'anterior. Això és perquè les confirmacions també es poden perdre.



Exercici 9.10 Suposant que tots dos extrems segueixen TCP correctament:

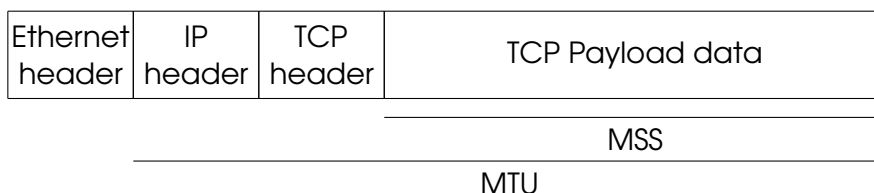
- Proporciona un conjunt de valors vàlids per a (a) fins a (i).
- Com canvia la història si (f) i (h) són idèntics/diferents?



9.5.3 Límits de transmissió

Les dades per transmetre sovint es produeixen a ràfegues. Voldríem enviar-ne tantes com fos possible tan aviat com fos possible, però TCP hi imposa dos límits.

El primer límit, la mida màxima de segment (MSS, Maximum Segment Size), és la quantitat màxima de dades de payload per segment permesa en una connexió TCP.



La raó d'aquest límit és evitar la fragmentació per motius de rendiment, p. ex., a causa de l'elevat cost en temps i memòria del reassemblatge.

Per defecte, la MSS és de 536 bytes a IPv4, però es pot negociar amb les opcions de la capçalera TCP durant la fase d'obertura de la connexió. A IPv6, la MSS per defecte és de 1220 bytes.

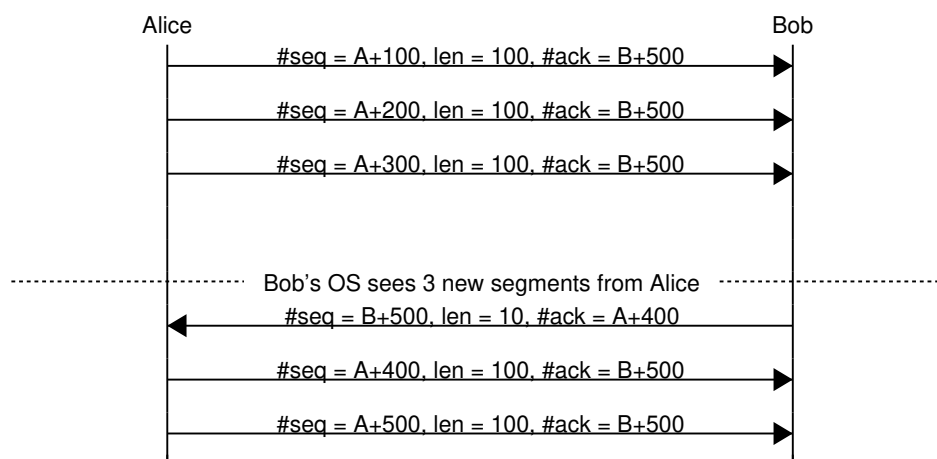
Exercici 9.11 Ethernet té una MTU de 1500 bytes. Suposant que els datagrames només travessen 2 xarxes Ethernet:

- Quina és la MSS més gran que no pot causar fragmentació, MSS_{max} ?
- Quina és la sobrecàrrega per a MSS_{max} i per a $MSS_{max} + 1$?

El segon límit afecta la quantitat màxima de dades de payload que es poden enviar abans que l'altre extrem respongui amb un nou #ack: l'anomenada mida de la finestra de transmissió. Un host no enviarà dades (per moltes que n'hi hagi de disponibles, o per urgents que siguin) fins que no arribin aquests #ack. Quan l'ordinador més lent respon, ho fa sempre

amb el #ack més recent.

Exercici 9.12 Quines són les mides de MSS i de finestra de transmissió més probables en aquest exemple?



El valor de la finestra de transmissió s'anuncia amb tots i cadascun dels segments TCP: aquest valor és un camp obligatori de les capçaleres TCP. A mesura que la connexió avança, cada host pot canviar el valor que anuncia segons la seva pròpia càrrega i necessitats.



Cada host anuncia la seva pròpia mida de finestra.

Aquest límit serveix per evitar que els ordinadors ràpids saturin els més lents (o sobrecarregats): sense el límit, l'ordinador més lent podria necessitar un buffer de mida infinitament creixent per emmagatzemar els segments encara no processats enviats per l'ordinador més ràpid. Amb aquest límit, en el punt de saturació, tots dos extrems envien segments al mateix ritme encara que triguin temps molt diferents a produir-los i processar-los.

9.6 TCP: "Tancant la connexió"

9.6.1 FIN, #seq i #ack

Un cop la connexió és ESTABLISHED, s'hi manté fins que un dels dos extrems decideix tancar-la.

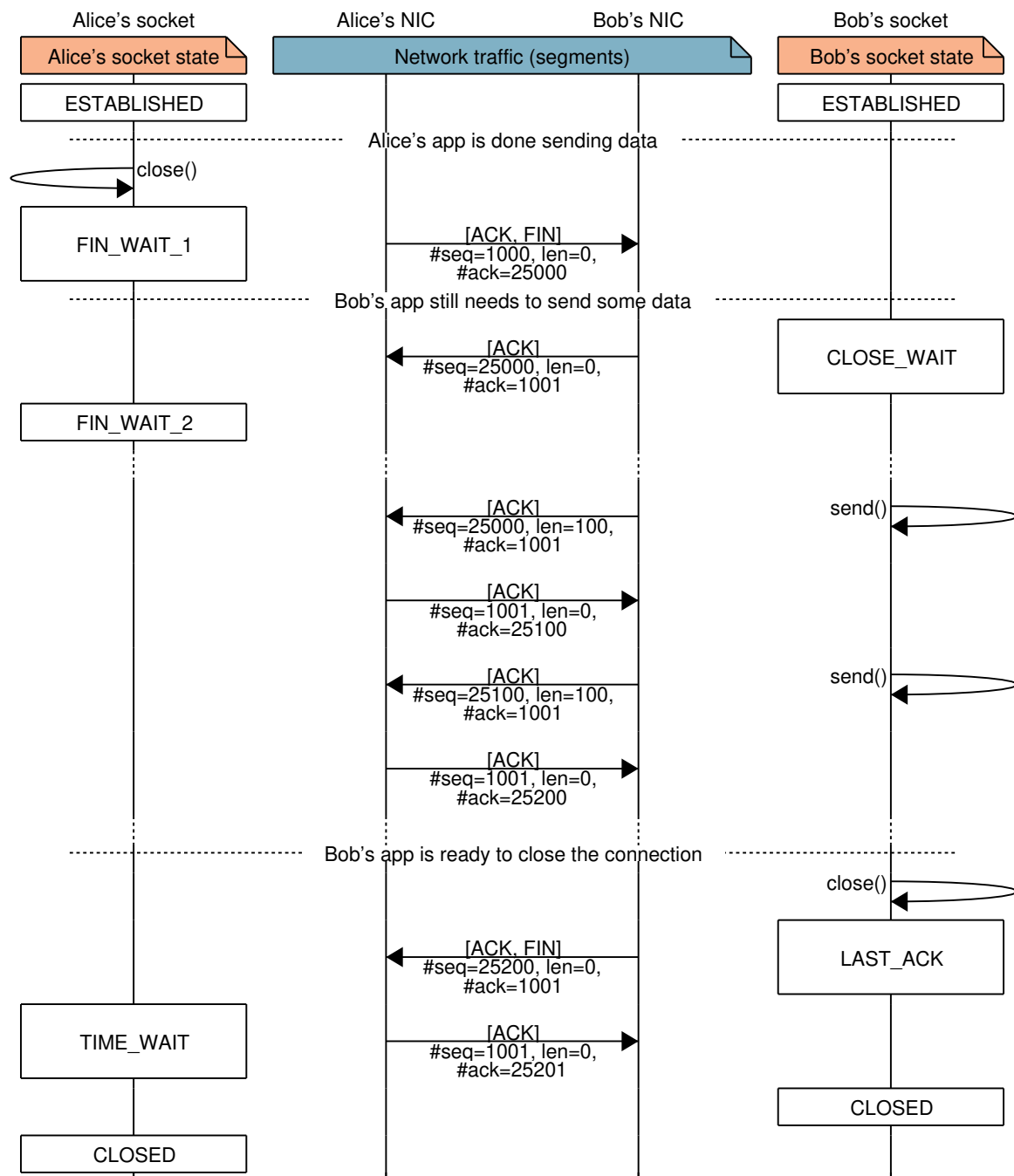
Un node anuncia que ha acabat d'enviar dades i que vol tancar la connexió enviant un segment amb el flag FIN activat a la capçalera. L'altre extrem pot continuar enviant dades, i en aquest cas espera respostes amb valors de #ack actualitzats. L'altre extrem també pot optar per tancar la connexió tan bon punt veu el primer FIN. En tots dos casos, si s'envien k bytes de dades i A és el #seq inicial, s'espera un segment amb #ack = $A + k + 2$.

#seq	A	A + 1	...	A + k	A + k + 1
contingut	<u>SYN</u>	Dades ...	...	... dades	<u>FIN</u>

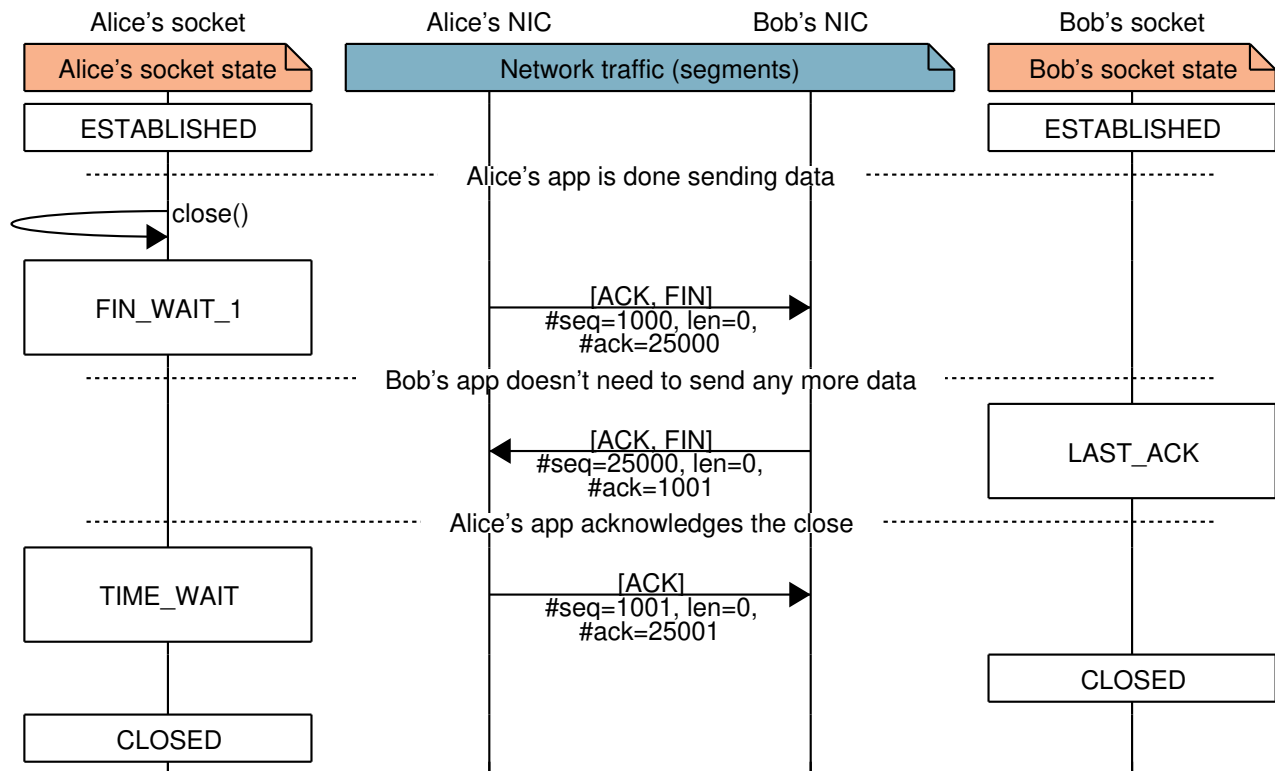
Fixa't que el flag FIN ocupa ell mateix un número de seqüència i s'ha de confirmar com el SYN (vegeu la secció 9.4), p. ex.:

9.6.2 Tancament bilateral

Després que un extrem tanqui el seu costat de la connexió, l'altre pot continuar enviant dades indefinidament. Quan el segon costat acaba d'enviar dades, també envia un segment amb el flag FIN activat.



Fixa't que, si l'altre extrem vol acabar la connexió en rebre un segment FIN, el procés de tancament es condensa en 3 segments en lloc de 4, p. ex.,



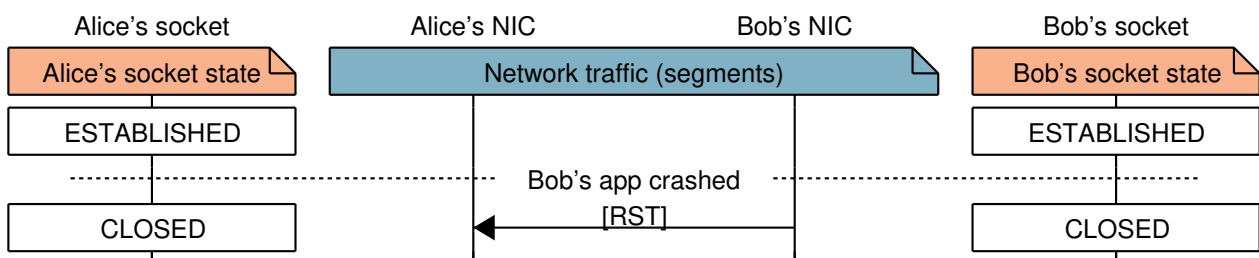
Fixa't que:

- Cada extrem envia només un segment amb el flag FIN, excepte en cas de retransmissió.
- El segment amb el flag FIN pot contenir dades de payload. En aquest cas:
 - Les dades de payload ocupen números de seqüència *anteriors* al del FIN.
 - L'altre extrem ha de fer ACK tant de les dades de payload com del FIN.

9.6.3 Tancament unilateral

De vegades, cal acabar les connexions i no hi ha prou temps o interès per passar pel procés de tancament bilateral de diversos passos. Aquest pot ser el cas, p. ex., quan un **procés** falla, quan es bloqueja un port TCP o UDP determinat, i en altres situacions d'error.

En aquest escenari, el host que ha detectat la situació anòmla simplement envia un segment amb el flag **RST** (reset) activat. Els extrems receptors no han de confirmar el RST; en canvi, els recursos de la connexió s'alliberen directament, p. ex.,

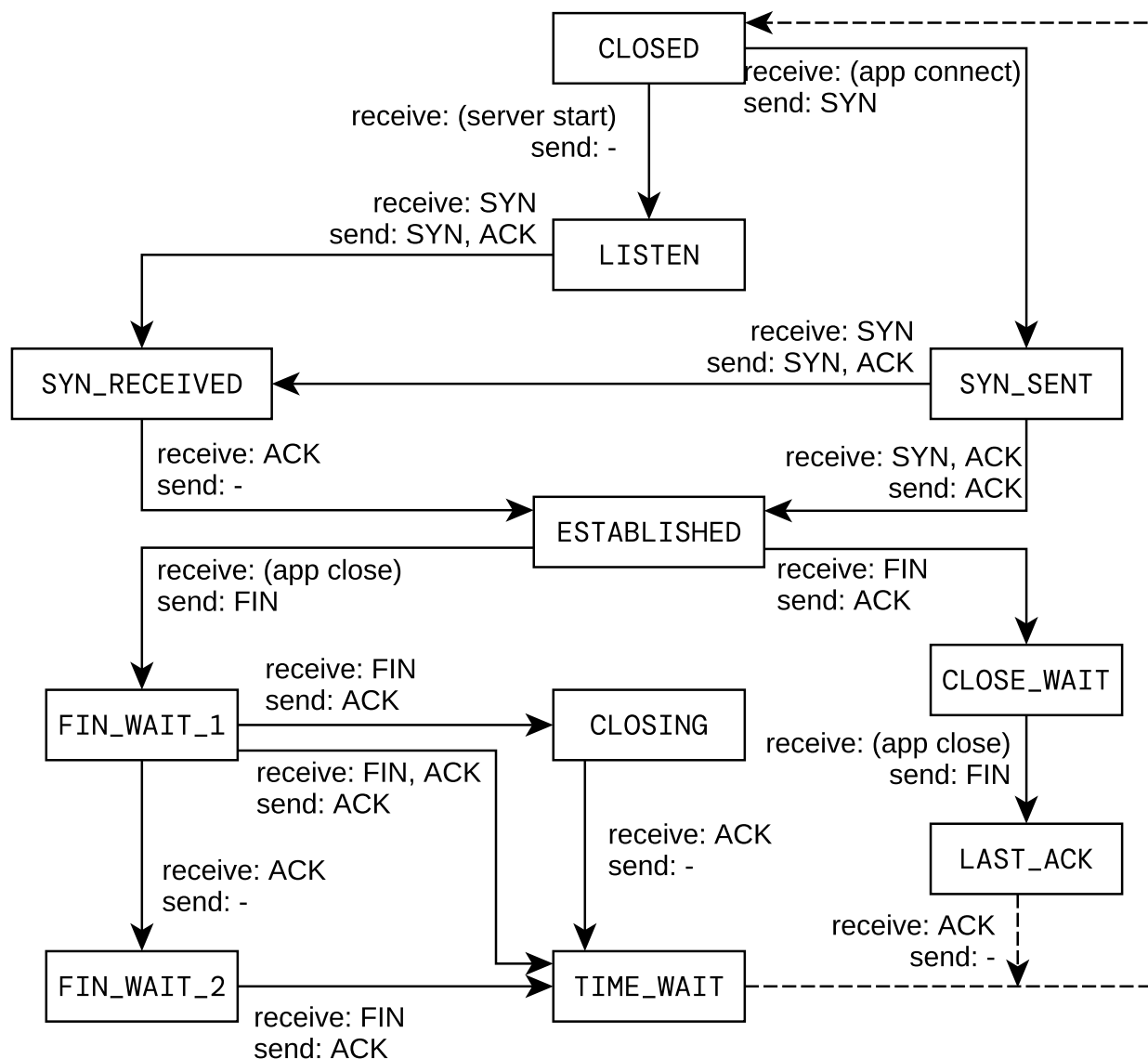


Seguir el procediment de tancament bilateral o unilateral de la connexió ajuda a reduir els recursos de càlcul necessaris per mantenir les piles de xarxa de milers de milions de dispositius arreu del món.



Una connexió *no es pot* reobrir després de tancar-se. El flag RST s'hauria pogut anomenar amb més precisió "terminació forçada".

Exercici 9.13 El diagrama següent mostra els 11 estats de TCP i (gairebé) totes les transicions d'estat possibles.



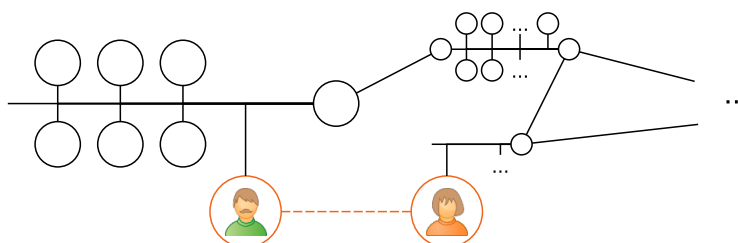
- Indica el cicle de vida del socket d'un servidor HTTP com un camí al graf.
- Indica el cicle de vida del socket del client HTTP corresponent com un camí al graf.
- L'estat **CLOSING** no es descriu a les seccions anteriors. Quina creus que n'és la finalitat?



Aquest diagrama està adaptat de "TCP/IP Illustrated, Volume 2: The Implementation," de Gary R. Wright i W. Richard Stevens.

10. Capa 7: Aplicació

Layer 7: Application
Layer 4: Transport
Layer 3: Internet
Layer 2: Network (LAN)
Layer 1: Physical



La capa superior de la pila de xarxa TCP/IP inclou totes les aplicacions que utilitzen TCP o UDP per al transport. És, amb diferència, la que conté el nombre més gran de protocols, i on aquests protocols es creen i se substitueixen més ràpidament.

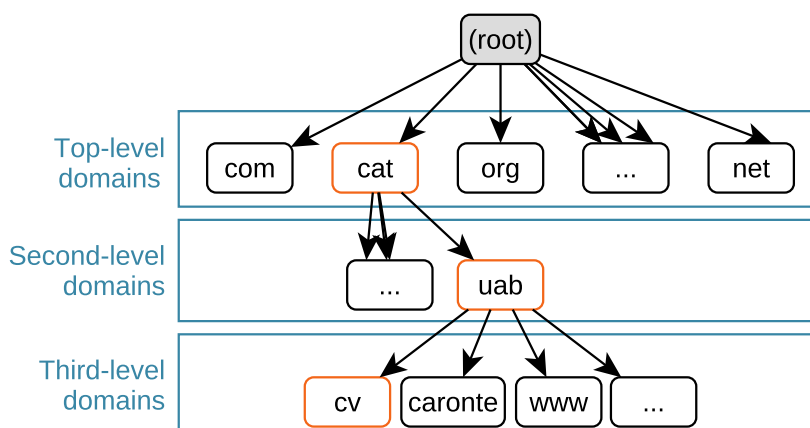
A més de les aplicacions d'usuari, hi ha diversos sistemes i protocols auxiliars que ajuden en la configuració i el funcionament de tots els hosts (equips connectats a la xarxa) d'Internet. En aquest capítol s'aborden alguns dels més crítics.

10.1 DNS – Sistema de noms de domini

El protocol DNS és responsable de traduir noms de domini com ara ietf.org o cv.uab.cat a adreces IP que es poden posar dins dels datagrames.

Els serveis DNS funcionen tant sobre TCP com sobre UDP, és a dir, es poden habilitar tots dos protocols de transport. En tots dos casos, s'utilitza el port 53.

10.1.1 Jerarquia



Els noms de domini d'Internet són seqüències de noms separats per punts. Aquests noms es presenten del més específic al més general de manera jeràrquica. Per exemple, a cv.uab.cat, [cv](http://cv.uab.cat) pertany a uab.cat, i [uab](http://uab.cat) a cat. El nom de més a la dreta, p. ex., cat o com, ha de ser un dels dominis de primer nivell de la IANA.

Els servidors DNS s'organitzen de la mateixa manera jeràrquica que aquests noms de domini. Hi ha 13 servidors DNS arrel (de Root-A a Root-M), que són responsables de tots els dominis de primer nivell. Pots preguntar a qualsevol d'ells per com, però també per su, i en sabran la resposta.

Això no vol dir que cada servidor arrel conegui tots els dominis *dins* del seu domini de primer nivell (p. ex., uab.cat): només coneixen aquests dominis de primer nivell. (p. ex., cat). La

feina dels servidors DNS arrel és redirigir-te a un altre servidor DNS encarregat del domini de primer nivell que has demanat.

De manera semblant, els servidors DNS responsables de cat coneixeran qualsevol domini *.cat existent (p. ex., uab.cat), però no els subdominis (p. ex., cv.uab.cat). La feina d'aquests servidors DNS de domini de primer nivell és redirigir-te a altres servidors DNS autoritatius per al domini de segon nivell sol·licitat (p. ex., uab.cat). Els dominis de segon nivell poden llavors estructurar la seva zona de noms com vulguin, incloent-hi qualsevol nombre i profunditat de subdominis, així com la traducció de nom a IP *dins* de la zona d'autoritat.

Exercici 10.1 Tothom fa servir el DNS constantment. Quins serien els riscos de tenir un sistema de traducció de noms centralitzat en lloc de la naturalesa distribuïda del DNS?

10.1.2 Registres

El DNS es pot veure com una base de dades distribuïda amb diferents tipus d'entrades anomenades registres. Els principals són:

A: Una entrada A associa un nom de domini a una adreça IPv4.

AAAA: Una entrada AAAA associa un nom de domini a una adreça IPv6.

NS: Una entrada NS apunta a un servidor DNS que pot ajudar.

MX: Una entrada MX proporciona el nom de domini del servidor de correu electrònic que gestiona els missatges d'un domini.

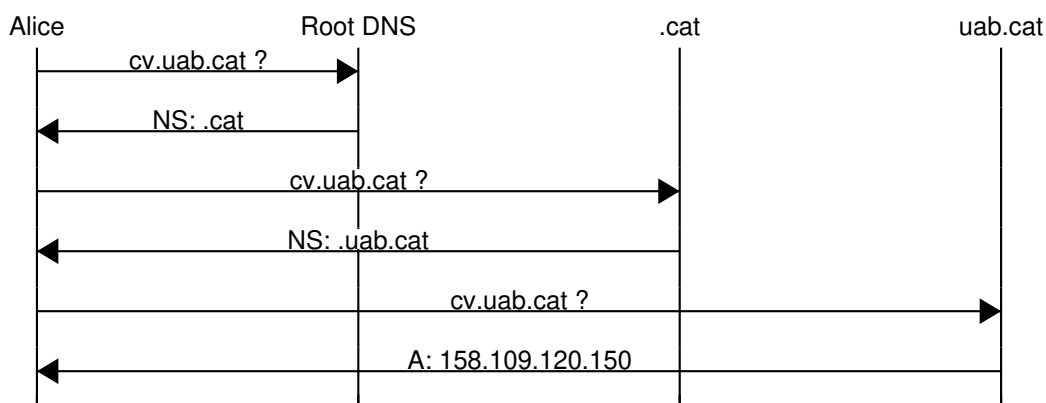
CNAME: Una entrada CNAME associa un nom de domini a un altre nom de domini.

Les aplicacions fan peticions als servidors DNS indicant el tipus de registres que els interessin. Les respostes del servidor poden ser els registres A o MX demanats, però el client també pot rebre una redirecció a un altre DNS (NS) o un àlies a un altre nom de domini (CNAME).

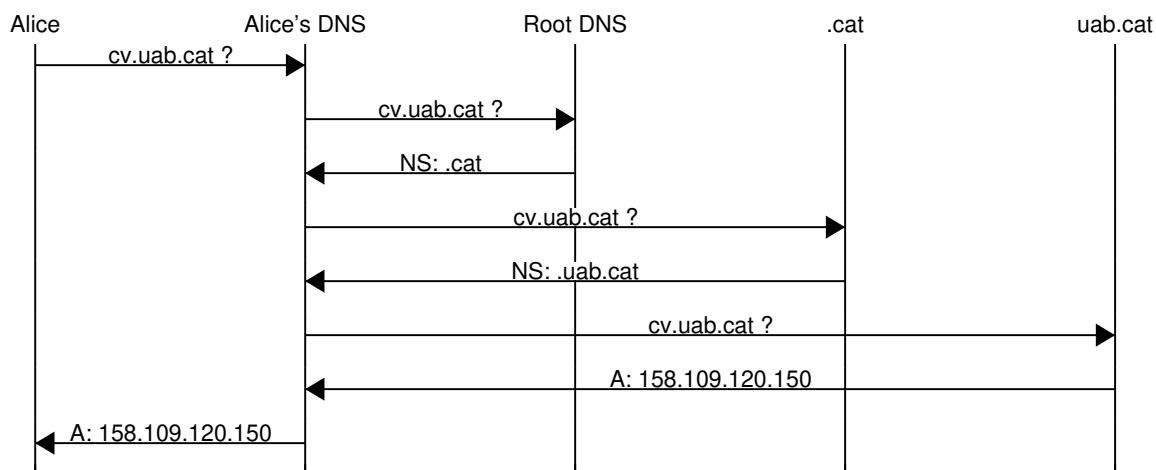
10.1.3 Resolució

La naturalesa distribuïda i jeràrquica de la base de dades del DNS fa que de vegades calgui intercanviar múltiples missatges per satisfer una consulta DNS.

Per exemple, l'Alice pot fer una resolució completament iterativa tota sola. Les dues primeres respostes contenen una entrada NS que apunta al següent servidor DNS i l'adreça IP d'aquest servidor DNS, però no l'adreça IP del cv.uab.cat demanat.



El proveïdor d'Internet (ISP) o l'empresa de l'Alice probablement oferiran servidors DNS la feina dels quals és rebre les peticions de l'Alice i resoldre-les per ella de manera recursiva.



La combinació de la resolució recursiva i les taules de memòria cau fa del DNS un sistema força eficient, amb una disponibilitat altíssima i una latència baixa.

Exercici 10.2 Pots utilitzar les eines `host` i `dig` per fer consultes DNS i veure'n els resultats. Per exemple, la meua sortida de `dig uab.cat -t ns` conté les línies següents:

```
;; QUESTION SECTION:
;uab.cat.                IN      NS
;; ANSWER SECTION:
uab.cat.                 172800 IN      NS      dns.uab.es.
;; ADDITIONAL SECTION:
dns.uab.es.             172112 IN      A        158.109.0.1
```

- Quin tipus de registre he demanat?
- Quins dos tipus de registre he rebut?
- Per què he rebut dos tipus de registre?
- Fes una resolució iterativa manual de `cv.uab.cat`. Pots començar amb alguna cosa com `dig @202.12.27.33 cv.uab.cat` per consultar el servidor arrel M.

10.2 DHCP – Protocol de configuració dinàmica de hosts

Per poder participar plenament a Internet, un host ha de tenir correctament configurats els aspectes següents:

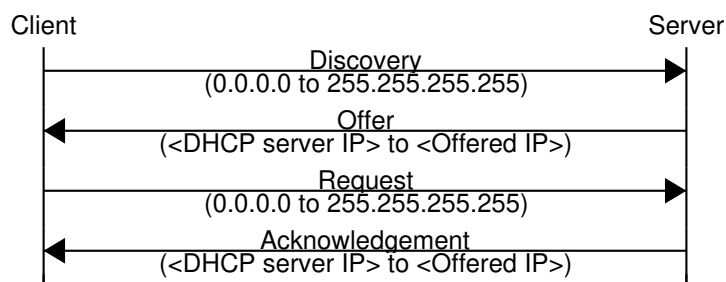
- **Adreça IP:** L'adreça IP és necessària per identificar un node a la xarxa, i perquè aquest node sàpiga quins missatges van adreçats a ell.
- **Màscara de subxarxa:** Els nodes l'han de conèixer per determinar quan han de lliurar els datagrames directament, i quan indirectament.
- **IP del gateway (passarel·la):** Cal l'adreça IP d'almenys un router (encaminador) per lliurar indirectament missatges amb destinació fora de la LAN (xarxa d'àrea local) actual.
- **Adreça IP del servidor DNS:** Tot i que no és estrictament necessària per utilitzar TCP/IP, l'adreça d'un servidor DNS és extremadament útil, i necessària per resoldre noms de domini a adreces IP.

Tradicionalment, aquests paràmetres s'havien de configurar manualment a cada ordinador. Avui dia, la majoria dels ordinadors d'usuari es configuren automàticament amb DHCP. Els sistemes operatius moderns contenen un client DHCP que consulta periòdicament la LAN fins que queda configurat.



El maquinari de xarxa ve amb una adreça MAC per defecte preconfigurada. A més, un node necessita conèixer la seva adreça MAC per fer funcionar el protocol DHCP. Per tant, les adreces MAC no es configuren mitjançant DHCP.

DHCP funciona sobre UDP. Els servidors DHCP escolten al port 67, i els clients escolten al port 68. L'intercanvi DHCP ideal consta de 4 missatges: Discovery, Offer, Request, Acknowledgement.



• Discovery

L'objectiu d'aquest missatge és trobar servidors DHCP disposats a proporcionar una configuració.

En la primera comunicació, el client no coneix la IP del servidor DHCP. En lloc d'això, utilitza:

- La seva adreça MAC com a adreça MAC d'origen.
- L'adreça de broadcast (difusió) de la LAN (**ff:ff:ff:ff:ff:ff**) com a adreça MAC de destinació.
- **0.0.0.0** com a IP d'origen, que vol dir "desconeguda".
- L'adreça de broadcast local, **255.255.255.255**, com a IP de destinació.

• Offer

Zero o més servidors respondran al client amb ofertes de configuració. Aquestes configuracions (p. ex., adreces IP) no són propietat del client fins que el servidor envia un DHCP Acknowledgement.

El servidor porta el control d'un conjunt d'adreces que assigna dinàmicament als clients. Les adreces IP s'ofereixen durant un període de temps fix, després del qual caduquen tret que es renovin (vegeu més avall). Les adreces que no es renoven tornen al conjunt i s'ofereixen de nou als clients a mesura que fan peticions.

- El servidor pot utilitzar la seva adreça IP real i la que ofereix al client.
- El client sap que n'és el destinatari perquè el servidor té en compte l'adreça MAC d'origen del Discovery del client.

• Request

El client tria una de les ofertes rebudes i demana que es completi l'assignació.

- El client utilitza l'adreça de broadcast local, **255.255.255.255**, com a destinació.
- El client manté **0.0.0.0** com a origen perquè l'adreça oferta encara no és seva.

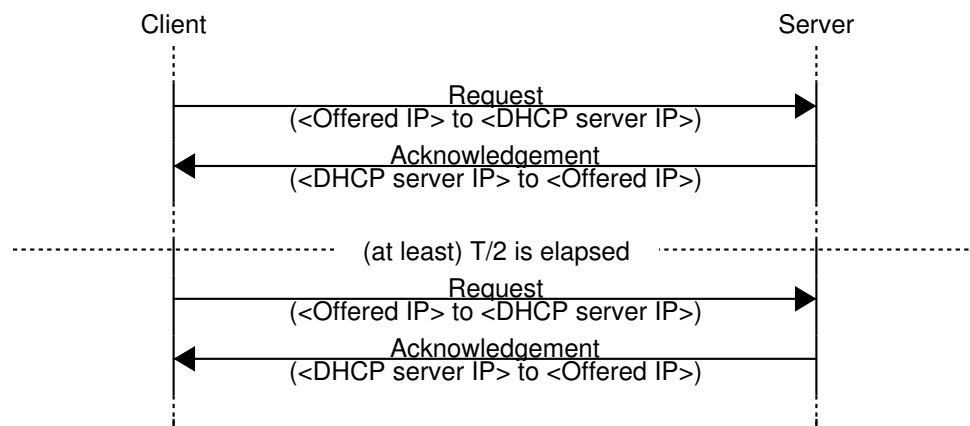
• Acknowledgement (ACK)

El servidor confirma la petició. Un cop el client rep aquest ACK, comença a utilitzar la configuració oferta com a pròpia. Aquest ACK no s'ha de confondre amb el de TCP.

• Negative Acknowledgement (NACK)

També és possible que el servidor denegui la petició, p. ex., si ha passat massa temps des de l'oferta. En aquest cas, s'envia un Acknowledgement negatiu (NACK). Si es rep un NACK, el client ha de tornar a la fase de Discover.

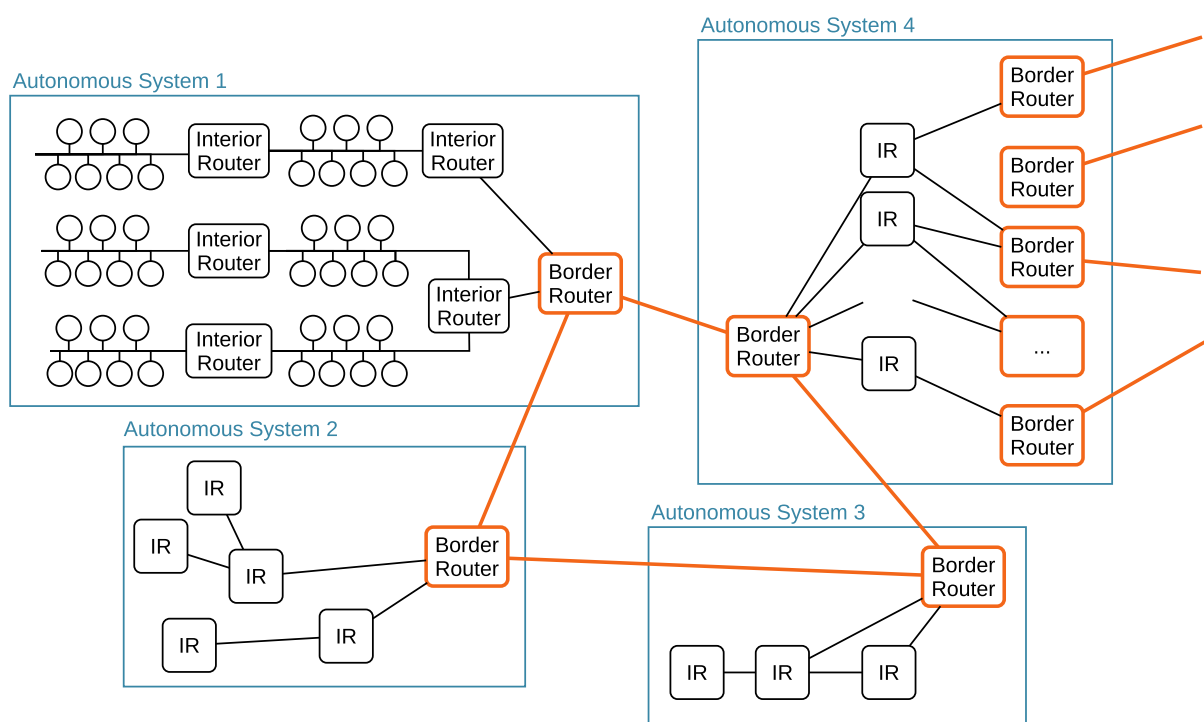
Quan un servidor DHCP cedeix una adreça a un client, ho fa durant un període de temps limitat T . Aquest període el fixa el servidor DHCP, i es pot configurar. Un cop transcorregut $T/2$, els clients poden intentar renovar la configuració cedida. Per fer-ho, es repeteix el cycle Request-Acknowledgement al llarg del temps fins que un dels dos extrems no el completa a temps, o bé el servidor DHCP envia un NACK.



Exercici 10.3 El router domèstic típic implementa alhora un client DHCP i un servidor DHCP. Explica com és possible i per què és útil.

10.3 Sistemes autònoms

L'encaminament es fa salt a salt, utilitzant taules d'encaminament tal com s'ha explicat a la Secció 8.3. Aquestes taules s'han de coordinar a gran escala per garantir la connectivitat del graf amb tots els hosts d'Internet.



10.3.1 Organització

Per facilitar aquesta coordinació, les LAN s'agrupen en sistemes autònoms (AS). Aquests gestionen blocs d'adreces IP que no se solapen (w.x.y.z/N), amb propietat i gestió separades. Moltes empreses, universitats i ISP tenen els seus propis AS, que estan l·listats públicament. Per exemple, la xarxa de la UAB pertany a l'AS13041.

Els punts d'intercanvi d'Internet (IXP) són un tipus especial d'AS el propòsit dels quals és interconnectar altres AS, en lloc d'allotjar serveis i/o clients. Per descomptat, fer funcionar aquests IXP no és gratuït, així que s'estableixen relacions comercials amb els AS interconnectats. Alguns IXP es comprometen a donar servei a qualsevol que compleixi els seus requisits bàsics, no estan controlats per cap operador únic, i ofereixen preus transparents i sovint equitatius als seus clients. Aquest tipus d'IXP s'anomena punt neutre (NP), en contraposició als punts no neutres utilitzats, p. ex., per un sol operador de xarxa.

Tots els AS fan funcionar routers frontera (BR) per interconnectar AS. Quan un router frontera rep un datagrama amb destinació dins de l'AS, el reenvia a través dels routers interns (IR) de l'AS. De manera semblant, quan un host dins d'un AS necessita contactar amb un host d'un altre AS, els datagrames es reenvien a través de routers frontera fins que arriben a l'AS de destinació i, finalment, a la LAN de destinació.

10.3.2 Configuració de la taula d'encaminament

Configurar manualment les taules d'encaminament de tots els routers frontera i routers interns seria massa costós i propens a errors. En lloc d'això, s'utilitzen diferents algorismes per configurar-les automàticament.

- Dins de l'AS, els routers interns i els routers frontera utilitzen un dels protocols de gateway interior disponibles. Sovint s'utilitza RIP (Routing Information Protocol) per la seva configuració relativament senzilla. Dit això, els AS poden triar lliurement l'algorisme que utilitzen internament.

RIP implementa una variant de l'algorisme de camí més curt de Bellman-Ford per minimitzar el nombre de salts que els datagrames han de fer dins de l'AS. Periòdicament (p. ex., cada 30 s), cada IR i BR:

1. Anuncia la seva distància *percebuda* als altres routers de l'AS.
2. Actualitza les seves distàncies percebudes a partir dels anuncis dels altres routers.
3. Actualitza en conseqüència la seva taula d'encaminament cap a totes les LAN de l'AS.

Exercici 10.4 RIP utilitza el nombre de salts com a mètrica de cost. Per què podria ser problemàtic? Com es pot resoldre aquest problema?

- Entre AS, només cal configurar els routers frontera; els routers interns simplement han de dirigir el trànsit exterior a través dels routers frontera.

Els routers frontera executen un protocol de gateway exterior a més del protocol de gateway interior utilitzat dins de l'AS. Els diferents AS s'han de posar d'acord en l'algorisme que utilitzen, de manera que normalment s'utilitza BGP (border gateway protocol).

BGP és un protocol més complex que RIP. Aquesta complexitat és necessària perquè BGP coordina routers frontera que pertanyen a AS diferents, és a dir, amb propietaris diferents:

- RIP utilitza un únic nombre (la distància, és a dir, el nombre de salts) per guiar un algorisme de camí més curt sobre un graf. BGP utilitza camins en lloc de distàncies. Això permet a BGP triar rutes amb més salts, però més convenients per a l'AS, p. ex., en termes de temps, cost econòmic, càrrega interna, etc.
- BGP es va dissenyar tenint en compte la seguretat i la confiança, i inclou mecanismes per verificar l'autenticitat dels camins anunciats per un router frontera.



BGP també es pot utilitzar com a protocol de gateway interior de l'AS. Normalment això només ho fan AS molt grans.

Exercici 10.5 Com pot BGP traduir interessos econòmics en configuracions d'encaminament?

Missions

L'última part d'aquesta guia proposa 9 problemes complexos anomenats missions amb un toc de ciència-ficció retrofuturista. Aquests problemes vertebreren els continguts de les parts anteriors, amb el focus posat en les aplicacions i amb l'objectiu d'assolir una comprensió profunda dels conceptes.

Es recomana als estudiants que treballin totes aquestes missions en ordre i que n'elaborin les solucions en forma d'informes formals. També se'ls anima a compartir-les amb la seva comunitat d'aprenentatge immediata (p. ex., la seva classe) i a intercanviar-se comentaris per mitjà de cartes formals.

11	Missions	67
11.1	Missió 1: Compartir és estimar	
11.2	Missió 2: Pergamí infinit	
11.3	Missió 3: Pasturant el caos	
11.4	Missió 4: Inter { planetary net	
11.5	Missió 5: Els diners són l'arrel	
11.6	Missió 6: Empipa dos planetes sencers	
11.7	Missió 7: TCP o no TCP	
11.8	Missió 8: Que em parles a mi?	
11.9	Missió 9: Vestíbul interplanetari	

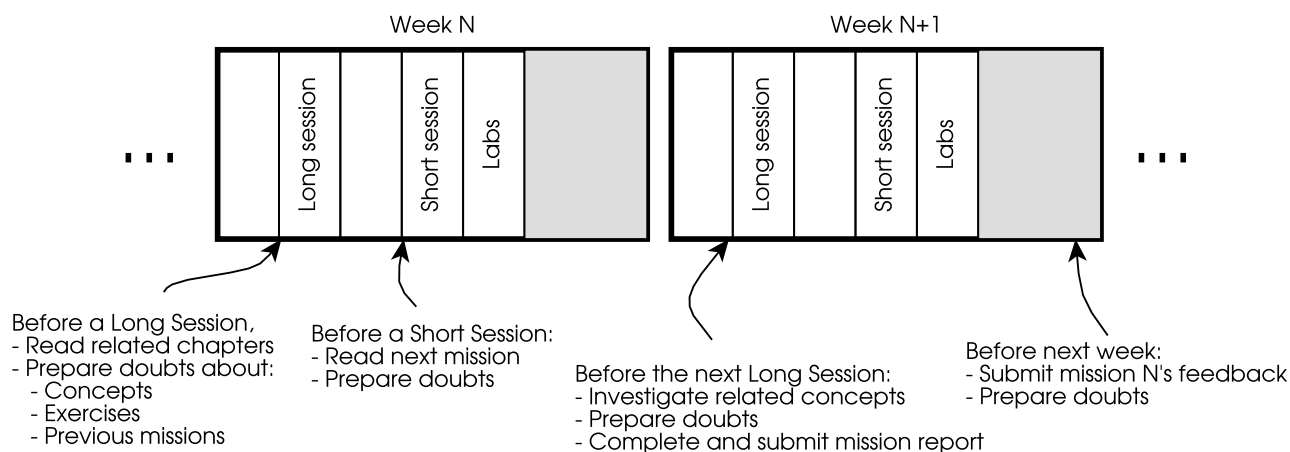
11. Missions

Continguts de les missions

Capítols		Missions								
		M1	M2	M3	M4!	M5!	M6	M7!!	M8	M9
§2	Travessant el vel analògic-digital	✓	○							
§3	Adreçament i topologies LAN (xarxa d'àrea local)	✓	○	○	○	○				○
§4	Format de paquet i MTU	✓	✓	○	○	✓	✓	✓	○	
§5	Piles de xarxa i encapsulació	↗	✓	○	○	○	✓	○		
§6	Internet: visió general de TCP/IP	↗	↗	↗	✓	○	○	○	○	○
§7	Capa 2: comunicació LAN		✓	○	✓	✓		○	○	
§8.1	Adreçament de capa 3		↗	✓	✓		○			○
§8.2	Funció i capçalera IP		↗	✓	○	✓	○			
§8.3	Encaminament IP				✓	○		○		✓
§8.4	Subxarxes IP			✓	✓					✓
§8.5	Fragmentació IP		↗			✓		○		
§8.6	ARP				↗	✓		○	○	
§8.7	ICMP				↗				✓	
§9.1	Adreçament de capa 4						✓		✓	
§9.2	UDP							✓	○	
§9.3	TCP						↗	✓	○	
§9.4	Obertura de connexió TCP						✓	✓	○	
§9.5	Intercanvi de dades TCP						↗	✓	○	
§9.6	Tancament de connexió TCP							↗	✓	
§10.1	DNS								✓	
§10.2	DHCP									✓
§10.3	ASs, IXPs, NPs									✓

✓: focus principal — ○: repàs — ↗: anticipació — !: crític

Ritme suggerit



11.1 Missió 1: Compartir és estimar



Fa uns 75 anys, es va enviar una expedició d'alt secret a través de l'espai amb l'objectiu de colonitzar un altre planeta. L'expedició va arribar sana i estalvia a la seva destinació, però l'ateratge va trencar els instruments de comunicació i els va deixar incomunicats. Ets un colon de tercera generació, futur enginyer, i aquesta és la primera missió que t'assigna l'assemblea: **compartir la nova i lluent calculadora. Si us plau, llegeix-ho tot amb atenció.**



Important: Es recomana que cada estudiant publiqui un **informe formal** (en PDF) al "fòrum d'intercanvi de la Missió 1" del Campus Virtual abans de la següent sessió de 2h. Després, comptem amb tu per publicar una **carta formal de resposta amb comentaris** (també en PDF) sobre almenys una altra solució, abans de la següent sessió d'1h.

Detalls de l'informe

Ara mateix, la colònia consta de 3 bases separades (3 edificis). Aquestes bases tenen 3, 200 i 5000 dispositius electrònics, respectivament. La base amb 3 dispositius s'ha construït específicament per allotjar una calculadora nova, potent i que consumeix molta energia (és un dels 3 dispositius). Tothom a la colònia vol tenir accés a la calculadora, però cap dels dispositius té capacitats de xarxa *encara*. El teu objectiu és proposar un disseny de xarxa convincent que connecti tothom i els deixi a tots satisfets.

Prepara un informe breu amb el teu disseny, amb propostes de solució clares i completes per als punts següents:

1. Com es travessa la **barrera analògic-digital** per permetre la comunicació digital? Sigues concret en les tecnologies que proposis.
2. Com es connecten físicament els dispositius? Proporciona un diagrama amb la **topologia** exacta.
3. Quin **sistema d'adreçament** s'utilitzarà per identificar de manera única cada dispositiu? Dona dos exemples d'adreça, un dels quals ha de ser el de la nova calculadora.
4. Quin **format de paquet** faran servir els dispositius? Proporciona un diagrama concret del format i els bytes d'un paquet d'exemple complet.

Detalls dels comentaris

Després de publicar la teva solució al fòrum, busca'n una altra i **reclama-la** responent-hi amb un missatge breu de "Hi faré comentaris" (una carta de comentaris per solució).

Després prepara una carta formal en PDF i publica-la al mateix fil quan estigui llesta (trobaràs una carta de comentaris d'exemple al Campus Virtual). En aquesta carta, inclou els teus comentaris (tant positius com negatius), explicant qualsevol cosa que consideris rellevant o que creguis que pot ajudar aquella solució. Assegura't de tractar els punts següents:

1. Hi veus alguna falla important o algun punt incomplet que impediria que la proposta es fes realitat?
2. Si tornessis a fer el teu informe, adoptaries alguna de les propostes d'aquesta solució?
3. Has pogut interpretar el contingut del missatge inclòs a l'informe? Inclou els bytes d'una possible resposta, basant-te en el que has entès d'aquell informe.

11.2 Missió 2: Pergamí infinit



Després de revisar la primera ronda d'informes, l'assemblea de la colònia està entusiasmada amb la idea d'explorar aquesta nova idea de *xarxa d'ordinadors*. Tanmateix, abans d'invertir en cap de les propostes, l'assemblea demana dissenys més complets i concrets. En concret, hauràs d'afegir la possibilitat d'enviar missatges arbitràriament llargs, potencialment infinits, i parar atenció a l'estructura de la pila de xarxa (detalls a continuació).

S'espera que cadascun de vosaltres publiqui un **informe formal** (en PDF) al "fòrum d'intercanvi de la Missió 2" del Campus Virtual abans de la següent sessió de 2h en què es programi una nova missió (consulta el calendari al Campus Virtual). Després d'això, comptem amb tu perquè publiquis una **carta formal de resposta amb comentaris** (també en PDF) sobre almenys una altra solució, abans de la següent sessió d'1h.

Detalls de l'informe

El nou informe ha de ser autònom, de manera que la persona que el revisi no necessiti llegir el primer. La proposta que descriguis en aquest informe no ha de seguir exactament la primera: pots manllevar tantes idees i introduir tants canvis com consideris oportú.

1. Resumeix la **capa física i la disposició topològica** de les teves xarxes d'ordinadors (tractades als punts 1 i 2 de la missió anterior).
2. Indica la MTU de la teva xarxa i **per què cal definir un valor finit de MTU**. Amb aquesta restricció, explica **com permetràs enviar missatges de longitud arbitrària (infinita)**.
3. Descric el **sistema d'adreçament** (actualitzat) i les seves limitacions. Pots mantenir o canviar el sistema que vas proposar a la missió anterior, sempre que t'asseguris que és vàlid.
4. Proposa una **pila de xarxa de 3 capes** (anomenem-les L3, L2 i L1, de dalt a baix) en què la funcionalitat de longitud arbitrària dels missatges es resolgui exclusivament a L3, l'adreçament exclusivament a L2, i L1 se centri només en les conversions digital/analògica i analògica/digital.
5. Inclou un exemple en què s'envii un missatge de longitud 1.5 vegades la MTU. En el teu exemple, proporciona els **bytes d'entrada i de sortida de cada capa** a l'origen i a la destinació de la comunicació.

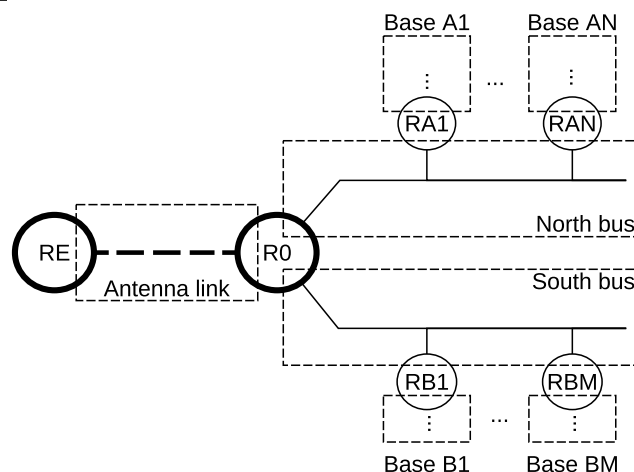
Detalls dels comentaris

Segueix un procés de comentaris semblant al de les missions anteriors, assegurant-te de tractar:

1. Hi veus algun defecte greu o algun punt incomplet que impediria que la proposta es fes realitat?
2. Hi ha algun límit real a la longitud dels missatges de la proposta que revises?
3. Has pogut interpretar el contingut del missatge inclòs a l'informe? Inclou els bytes d'una possible resposta de la mateixa longitud, basant-te en la teva comprensió d'aquest informe.
4. Hi ha alguna part de la proposta que revises que millori la teva?

11.3 Missió 3: Pasturant el caos

A causa de diferències polítiques, i malgrat tots els esforços de l'assemblea de la colònia, cada base del planeta ha adoptat una tecnologia LAN (xarxa d'àrea local) diferent (!). Tot i que les comunicacions dins de cada base són satisfactòries, ara per ara és impossible connectar directament dos ordinadors de dues bases diferents. Un cop més, la colònia et demana ajuda.



Gràcies als sacrificis de tothom, la colònia ja està en condicions de construir i instal·lar un nou router (encaminador) i dos busos cablejats. Al diagrama següent es representen amb una línia més gruixuda. El teu objectiu a llarg termini és fer possible la comunicació entre xarxes entre dos nodes qualssevol de dues bases qualssevol. De moment, dissenyaràs la infraestructura física i lògica.

Detalls de l'informe

Totes les bases fan servir el protocol IPv4, i hi ha diverses bases connectades tant al bus Nord com al bus Sud. Cada base conté una única LAN. Coordina't amb els altres perquè *no hi hagi dues LAN de la mateixa mida*.

1. Dins del teu grup:

- Determina quantes adreces necessites perquè cada dispositiu es pugui identificar de manera única dins la teva LAN. Tria la mida de LAN mínima que satisfaci aquest requisit i quantifica l'“espai” lliure disponible a la teva LAN.
- Indica quants dels 32 bits d'una adreça IPv4 necessites per identificar un sol dispositiu connectat a la LAN de la teva base. Quin concepte utilitzat a IP està més estretament relacionat amb aquest nombre? Per a què pot ser útil aquesta estructura?

Network ID	Host ID
------------	---------

2. En coordinació amb els altres grups de l'aula:

- Crea una LAN per al bus Nord i descriu totes les adreces físiques i lògiques que hi estan connectades.
- Assegura't que els blocs d'adreces IPv4 assignats a les LAN siguin de mida diferent i no se solapin.
- Fent servir la pissarra o un document compartit, fes una llista de totes les LAN, amb les seves mides i els rangs d'adreces (actualitzats).

Detalls de la retroacció

Segueix un procés de retroacció semblant al de les missions anteriors, i assegura't de tractar aquests punts:

1. Les LAN estan dimensionades correctament?
2. Hi ha cap col·lisió entre els blocs?
3. L'informe original extreia tota la informació sobre l'estructura de les adreces IP?

11.4 Missió 4: Inter { planetary net

Després d'uns quants anys d'ús, la colònia per fi comença a apreciar el valor d'interconnectar xarxes heterogènies mitjançant el protocol IP. Tant, que han invertit en una potent HugeAntenna™ que, segons creuen, pot connectar la colònia amb la Internet de la Terra.



L'assemblea de la colònia està preocupada perquè no sap si les bases es podrien comunicar entre elles i, encara més important, amb la nova antena. A més, tem la represàlia despòtica de la Terra si no aconsegueixen establir una connexió amb Internet. Un cop més, et demanen ajuda.

⚠️ Atenció! Aquesta missió és crítica. Pren-te el temps que calgui, demana tota l'ajuda que necessitis i no perdis l'ànim. Les missions anteriors es poden considerar un entrenament per a aquesta – et recomanem molt que les acabis amb cura abans d'intentar aquesta.

Detalls de l'informe

Recorda la topologia de xarxa que es demanava a la missió anterior. A hores d'ara, hauries de tenir accés a un acord consensuat sobre la distribució d'adreces físiques i lògiques dels dispositius de la colònia (si no és així, digues-ho!).

Considera els *routers* (encaminadors) presents a la colònia: $\{RA_i\}_{i=1}^N$ estan connectats al Bus Nord, $\{RB_i\}_{i=1}^N$ al Bus Sud, i R_0 enllaça tots dos busos. La nova antena es connectarà a un tercer controlador d'interfície de xarxa (NIC) de R_0 .

El teu objectiu és ajudar a configurar totes les màquines de la colònia (especialment els routers) perquè la informació pugui circular entre les bases i, sobretot, entre la colònia i la Internet de la Terra.

1. Resumeix les adreces físiques i lògiques de totes les màquines de la colònia perquè el teu informe sigui autocontingut. Et suggerim que reutilitzis la configuració que vas elaborar a la missió anterior, després de corregir qualsevol problema detectat. Fes servir el format CIDR sempre que sigui aplicable.
2. Proporciona les taules d'encaminament més senzilles que funcionin que siguis capaç de concebre per a les màquines següents:
 - Un router RA_i que trïis, i una màquina que no sigui router M_i connectada a RA_i .
 - Un router RB_j que trïis, i una màquina que no sigui router M_j connectada a RB_j .
 - El router R_0 .
3. Crea dos diagrames de tipus swimlane que mostrin *tots* els paquets transferits dins de la colònia per als dos intercanvis següents. Per a cada paquet d'aquests diagrames, indica *totes* les adreces que conté. Assegura't d'especificar en algun lloc la MTU de totes les LAN (xarxes d'àrea local) implicades en els intercanvis.
 - M_i envia unes dades a M_j , i M_j li torna una resposta.
 - M_i envia un missatge a un servidor de la Terra, però no rep cap resposta.

Detalls de la retroacció

Fes servir un procés de retroacció semblant al de les missions anteriors, i assegura't de tractar aquests punts:

- Hi ha cap problema amb l'assignació d'adreces físiques i lògiques?
- S'ha fet servir correctament el format CIDR?
- Les taules d'encaminament contenen tots els camps necessaris?
- Es pot simplificar més alguna de les taules?
- Els lliuraments directes i indirectes es representen correctament als diagrames swimlane?
- Les MTU són coherents amb els diagrames?

11.5 Missió 5: Els diners són l'arrel

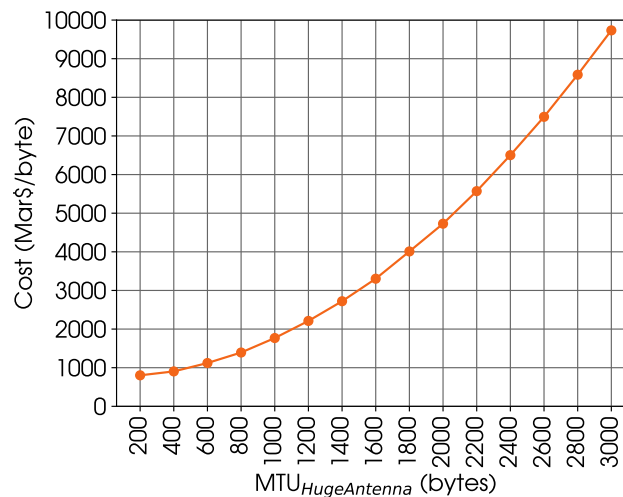
Tota la colònia bull d'energia intel·lectual després d'instal·lar la HugeAntenna™ i connectar-la a la Internet de la Terra – pensa en totes les coses xules que s'han perdut fins ara! Malauradament, ara que la colònia està enganxada a Internet, el tirànic govern de la Terra ha decidit apujar molt el cost d'usar l'enllaç de l'antena interplanetària. Els preus abusius causen greus problemes a la societat colonial, i demostren un cop més que els diners – o, potser, la seva manca – poden ser l'arrel de tot mal. Ajudaràs la colònia a recuperar l'estabilitat financera?



⚠️ Atenció! Aquesta missió és crítica. Pren-te temps, demana tota l'ajuda que calgui i no perdís l'ànim. Les missions anteriors són un entrenament per a aquesta – et recomanem molt que les acabis amb cura abans d'intentar-la.

Detalls de l'informe

La Terra està disposada a negociar el preu en Mar\$ (Dollaroos marcians) per byte transmès per la HugeAntenna™. Aquest preu dependrà de la MTU de l'antena, com mostra la figura següent. Fixa't que $MTU \in \{200, 400, \dots, 3000\}$. L'assemblea necessita una anàlisi quantitativa que justifiqui la millor elecció de MTU.



Per simplificar l'anàlisi, considera un escenari representatiu en què un usuari d'una de les bases de la colònia envia *payloads* (càrrega útil) d'exactament 1900 bytes directament sobre IP.

- Proporciona un mapa topològic de la xarxa de la colònia fins a l'enllaç entre la HugeAntenna™ i la Terra, sense incloure'l. En aquestes LANs (xarxa d'àrea local), indica la MTU més petita que evita la fragmentació dins la colònia. Pots reutilitzar la topologia de missions anteriors o crear-ne una de nova.
- Considera almenys 4 valors possibles per a la MTU de la HugeAntenna, 2 menors de 1900 bytes i dos de majors. Per a cada valor de $MTU_{HugeAntenna}$, indica:
 - Quants paquets s'envien per la Huge Antenna per transmetre el payload.
 - Quins camps no són idèntics en aquests paquets (només si s'envia més d'un paquet).
 - El cost total d'enviar el payload.

Coordina't amb els altres enginyers per trobar el valor òptim global de $MTU_{HugeAntenna}$.

- Compara el trànsit total (en bytes) transmès dins la colònia en enviar el primer payload (amb totes les memòries cau buides) respecte al segon payload (amb les memòries cau plenes i vàlides).

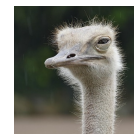
Detalls de la retroacció

Segueix un procés de retroacció similar al de les missions anteriors, i assegura't de tractar:

- Les MTU de la colònia estan configurades com es demana?
- Es considera correctament la fragmentació en determinar el cost?
- Es comptabilitza bé la sobrecàrrega deguda a ARP en calcular el trànsit total?

11.6 Missió 6: Empipa dos planetes sencers

Han passat anys a la colònia des que vas ajudar a configurar la gran antena per comunicar-vos amb la Terra. Fa poc, la colònia ha descobert *una altra* colònia secreta, a Saturn – i la Terra ni es va molestar a dir-nos-ho!



L'assemblea ha decidit fer una petita broma i et torna a demanar ajuda. El teu objectiu és segrestar la propera connexió TCP Saturn ↔ Terra i després enviar un memoràndum fals fent-te passar per Saturn. L'assemblea demana que sigui elegant.

Detalls de l'informe

L'enginyera en cap de la colònia ha fet una exploració i està disposada a ajudar-te. Ha recopilat la informació següent:

- La teva HugeAntenna™ de la colònia es comunica amb una altra HugeAntenna™ de la Terra. La Terra fa servir aquesta mateixa antena per comunicar-se amb una altra HugeAntenna™ de Saturn. Quan la Terra rep el primer paquet d'una connexió TCP, en verifica l'origen.
- Després d'aquest primer paquet, la Terra no verifica l'origen de cap paquet. De fet, els missatges falsos o inesperats posteriors s'ignoren, sense activar cap alarma. Calculant bé el moment, ho aprofitaràs per injectar els teus paquets falsos.
- El primer dia de cada mes, exactament a les 4:20 (hora de Nova Barcelona, Terra), la Terra rep el primer paquet d'una connexió TCP de Saturn. Aquestes connexions serveixen per intercanviar memoràndums oficials, i l'assemblea vol que en segrestis la propera.
- Saturn usa una pila TCP/IP obsoleta i insegura amb aquest defecte: sempre posa el número de seqüència TCP a 0 al primer missatge d'una connexió TCP.
- No podràs veure els missatges entre la Terra i Saturn. Però l'enginyera en cap va capturar aquesta frame (trama) Ethernet, que va iniciar la connexió TCP anterior (diguem-ne C_{-1}) entre Saturn i la Terra (00 d0 va seguit de f1 bb i 45 10 va seguit de 00 28):

```
00 09 bf 42 07 64 00 d0    f1 bb 0f 5a 08 00 45 10
00 28 d2 71 40 00 40 06    00 00 ca f7 44 05 4c 4c
15 15 9f ee 00 a9 00 00    00 00 00 00 00 00 50 02
01 00 00 00 00 00
```

Amb això, l'assemblea demana un informe amb els aspectes següents:

1. Dibuixa un diagrama swimlane de l'establiment de la connexió anterior (C_{-1}) entre Saturn i la Terra. A més, tabula *tots* els camps de capçalera d'aquests paquets (i els seus valors).
2. Explica el teu pla d'atac, és a dir, com segrestaràs la connexió. Indica clarament:
 - Quines suposicions fas sobre la propera connexió (p. ex., quins camps seran idèntics o podrien canviar).
 - Quants missatges enviaràs i quan tens previst enviar-los.
3. Mostra la seqüència completa de bytes d'una frame que contingui el teu memoràndum de broma.

Detalls del retorn

Segueix un procés de retorn com el de les missions anteriors, tractant aquests aspectes clau:

- S'ha descodificat bé la frame capturada per l'enginyera en cap?
- Has ideat un pla que necessiti menys missatges?
- Has pogut descodificar el memoràndum de broma? Era elegant?

11.7 Missió 7: TCP o no TCP



Anys d'èxit continuat han consolidat l'ús d'IPv4 per interconnectar xarxes dins de Mart i amb altres planetes. Mentrestant, els equips d'enginyeria de la colònia han descobert les limitacions de l'IPv4 nu i la necessitat d'usar almenys un dels principals protocols de transport per comunicar-se amb la Internet de la Terra.

Implementar aquesta quarta capa i afegir-la a les piles de xarxa existents de cada node de la colònia és una tasca descomunal. Per això, en una primera fase només s'implementarà un dels dos protocols de transport principals. La colònia et demana ajuda per comparar quantitativament aquests dos protocols i decidir quin triar.

⚠️Atenció! Aquesta missió és molt crítica. Pren-te-la amb calma, demana tota l'ajuda que calgui i no perdís l'ànim. Les missions anteriors es poden considerar un entrenament per a aquesta — et recomanem molt que les acabis amb cura abans d'intentar-la.

Detalls de l'informe

L'assemblea et demana que consideris amb cura els punts següents a l'informe:

1. Perquè l'informe sigui autocontingut, inclou-hi la topologia de xarxa de la colònia en format CIDR quan escaigui, i especifica valors raonables d'MTU (pots reutilitzar els de missions anteriors). Completa la descripció especificant l'amplada de banda (velocitat de l'enllaç, en bytes per segon – suposem que és constant).
2. Un dels dos protocols de transport candidats usa un paràmetre anomenat MSS. Indica quin i especifica el valor més gran que permetria la comunicació entre dues màquines qualssevol de la colònia sense requerir fragmentació. Usa aquest valor a la resta de l'informe.
3. Tria un node N_1 d'una base connectada directament al Bus Nord. Tria un altre node N_2 d'una base connectada directament al Bus Sud. Per a cadascun dels dos protocols de transport principals, calcula el temps total des que N_1 comença a transmetre fins que N_2 rep el primer byte de dades del payload (càrrega útil). Per fer-ho, suposa:
 - Si el protocol és orientat a connexió, les dades només es transfereixen un cop establerta la connexió.
 - Tots els nodes i routers (encaminadors) tenen potència de càlcul infinita, és a dir, encapsular/desencapsular dades no introdueix cap retard.
 - Les velocitats de transmissió segueixen sent finites, tal com les has definit abans a l'informe.
 - Les taules d'encaminament són òptimes.
 - Totes les memòries cau comencen buides per a cada protocol, i les entrades romanen vàlides un cop establertes.

Et recomanem incloure diagrames de carrils (swimlane) per justificar la resposta.

4. Quins protocols de transport inclouen mecanismes de control de congestió? Quines condicions caldrien per permetre una transmissió contínua de N_1 a N_2 sense esperes degudes al control de congestió?
5. Indica tres casos d'ús, és a dir, aplicacions d'usuari: el primer adequat només per a un dels protocols de transport, el segon adequat per a l'altre protocol de transport, i el tercer adequat per a tots dos.

Detalls de la retroacció

Segueix un procés de retroacció similar al de les missions anteriors, i assegura't de tractar aquests aspectes clau:

- L'MSS triat és coherent amb els valors d'MTU especificats?
- El temps de transmissió de cada paquet està ben calculat?
- Veus algun problema amb l'encaminament, la fragmentació o l'ARP?
- Es consideren correctament tots els elements del control de congestió?
- Els casos d'ús proposats estan ben triats?

11.8 Missió 8: Que em parles a mi?



La teva colònia ha estat utilitzant amb èxit (segons Saturn, també *abusant de*) la pila TCP/IP per connectar-se a la Internet de la Terra. Tanmateix, fins ara, en cert sentit no existíeu perquè no teníeu *nom*. Això està a punt de canviar, perquè la colònia està instal·lant el seu primer servidor del Sistema de Noms de Domini (DNS), que serà l'autoritat del nou domini de primer nivell (TLD) `.mars`. Es necessita la teva ajuda per configurar els nous serveis que s'oferiran a tot el Sistema Solar.

Detalls de l'informe

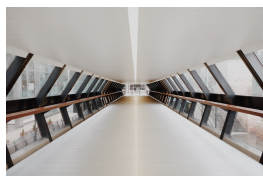
1. Resumeix la topologia de xarxa de la teva colònia i inclou els rangs d'adreces IP i els valors d'MTU assignats a cada LAN (xarxa d'àrea local) en notació CIDR. Això farà que el teu informe sigui autocontingut i ajudarà els revisors a fer-te comentaris (pots reutilitzar o actualitzar la configuració de sessions anteriors).
2. Indica clarament quina màquina allotjarà el teu servidor DNS i quina és la seva adreça IP.
3. Especifica 5 serveis $S_1, S_2, \dots, S_5$ (diferents del DNS) que s'allotjaran a la teva colònia. Un d'ells s'allotjarà a la mateixa màquina que el servidor DNS. Per a cada servei, proporciona:
 - Una breu explicació del servei i del protocol de capa d'aplicació que utilitza.
 - L'entrada o entrades DNS relacionades amb aquest servei que s'han d'incloure al nou servidor DNS.
 - Un exemple de consulta (p. ex., `service.mars`) per a la qual es retornaran aquestes entrades, qui enviarà aquesta consulta i quan.
4. La màquina que allotja el servidor DNS també allotja un altre servei. Com pot distingir entre els dos tipus de trànsit?
5. Algú que utilitza el node N_1 de la teva colònia implementarà un script per detectar quan els serveis d'Internet deixen d'estar disponibles. Per fer-ho, farà servir missatges ICMP periòdics per *comprovar* que estan en línia. Explica exactament com funcionarà això i, amb diagrames de carrils (*swimlane*), descriu tot el trànsit dins de la colònia derivat d'una sola *comprovació* per a cadascun dels escenaris següents:
 - (a) N_1 comprova el servei `cv.uab.cat` mitjançant resolució iterativa sobre UDP començant pel servidor arrel M.
 - (b) N_1 comprova el servei S_1 (indica'n el nom) mitjançant resolució iterativa sobre TCP començant pel servidor arrel M.
 - (c) N_1 comprova el servei S_1 mitjançant resolució recursiva sobre UDP amb el servidor DNS4EU sense filtrar.
 Als diagrames, per a cada missatge intercanviat, indica:
 - El tipus de missatge (de la capa superior) i, si escau, el seu subtipus.
 - Totes les adreces implicades.
 - Tota la informació relacionada amb el DNS que conté el missatge.

Detalls de la retroacció

Tingues en compte requisits i un procés similars als de les missions anteriors, i assegura't de tractar els aspectes clau següents:

- S'utilitza l'ICMP d'una manera lògica i efectiva?
- Es gestionen correctament les resolucions DNS iterativa i recursiva?
- Es representa correctament el cicle de vida de la connexió TCP?

11.9 Missió 9: Vestíbul interplanetari



En part gràcies a la teva ajuda per configurar la xarxa i els serveis de la colònia, la teva colònia bull d'activitat cultural i científica. El vestíbul principal del port espacial interplanetari de la colònia rep cada dia milers de visitants de diferents planetes. Aquests visitants reclamen una connexió a Internet ràpida i senzilla des d'aquest vestíbul, i l'assemblea de la colònia t'ha designat per ajudar amb la configuració.

Detalls de l'informe

L'assemblea t'ha demanat que configuris un servidor DHCP per als dispositius dels passatgers. El Sistema Autònom número 7847 t'està cedint amablement les 1024 adreces més altes del seu bloc, que seran les que proporcionarà aquest servei.

Cada 2 hores arriba al port espacial una nau amb fins a 300 persones (cadascuna amb un únic dispositiu, afamat d'Internet). Algunes marxen immediatament mentre que d'altres es queden més temps al port espacial, però com a màxim hi ha 1000 dispositius que necessiten estar connectats alhora. Just abans que el port espacial comenci a operar al matí, totes les adreces gestionades pel servidor DHCP han d'estar disponibles per ser concedides, perquè les primeres 5 naus arriben completament plenes.

1. Esbrina qui t'està cedint generosament les 1024 adreces IP públiques i quin és el seu bloc complet.
2. Proporciona una taula d'encaminament actualitzada per al router (encaminador) R_0 . Aquest és el router connectat directament a la HugeAntenna™ de la colònia, als Busos Nord i Sud, i ara també a la nova LAN (xarxa d'àrea local) dedicada als passatgers. Pots suposar que aquesta nova LAN està connectada a una interfície afegida de nou.
3. Quin és el temps de concessió (Lease Time) més llarg que pots configurar que sigui compatible amb l'hora punta del matí? Justifica la teva resposta.
4. Fent servir diagrames de tipus swimlane, proporciona un escenari concret en què el màxim de 1000 dispositius estiguin connectats simultàniament en algun moment durant l'hora punta. Indica quants missatges de cada subtipus DHCP hi intervenen.
5. Si un passatger s'amagués a la terminal del port espacial durant la nit, quin mecanisme faries servir per assegurar que la seva adreça IP assignada estigui disponible per ser concedida l'endemà al matí?

Detalls de la retroacció

Considera requisits i un procés similars als de les missions anteriors, assegurant-te de tractar els aspectes clau següents:

- La taula d'encaminament és correcta i tan senzilla com sigui possible?
- Has trobat un Lease Time més llarg que sigui compatible amb els requisits?
- Hi veus cap problema en la descripció de l'escenari amb 1000 dispositius concurrents?

Índex de Conceptes

3-way handshake, 52, 53

A, 61

AAAA, 61

ACK, 48, 51, 53, 54

acknowledgement
number, 52, 55

address, 15, 19, 21

address block, 30

analog, 26

ARP, 29–31, 33, 39, 44

ARPANET, 26

AS, 64

ASCII, 9

authentication, 29

authoritative, 61

bandwidth, 31

BGP, 65

big endian, 9, 23

binary mode, 19

bind, 49, 50

bitdepth, 9

bitwise, 10

body, 18

boolean logic, 10

BR, 65

broadcast, 30, 31, 36, 43,
45

buffer, 53

bus, 13, 14, 17, 18

byte boundary, 19

cache, 45, 62

channel, 18

checksum, 39

CIDR, 35, 41

client, 15, 31, 49

CNAME, 61

collision, 13

complete graph, 13

connection, 26, 48

cryptography, 29

data, 9

data line, 18

data link, 17

datagram, 21, 29, 33, 34,
39, 43, 49, 55, 60

decapsulate, 39

decode, 19

default routing table, 40

device, 13

DF, 43

DHCP, 62

digital, 26

direct delivery, 39, 40

DNS, 28, 60

DNS record, 61

domain name, 60

echo, 31

encapsulation, 21, 22, 29,
30

end-to-end, 15

error detection, 18

escape sequence, 17

Ethernet, 29, 31

EtherType, 31

exponential, 13

exterior gateway protocol,
65

field, 11

field (packet), 18

field (physics), 12

FIN, 51, 56

fixed-point, 9

flag, 19

floating-point, 9

flow control, 48

fragmentation, 33, 43, 50

frame, 21, 30, 31

full duplex, 53

gratuitous, 45

header, 18, 21, 22, 37, 39,
43, 51

hexadecimal, 10, 15, 19

hop, 39, 64, 65

host, 48

HTTP, 49

HTTPs, 49

hub, 14

IANA, 28

ICANN, 28

ICMP, 34, 46

IETF, 28

indirect delivery, 40, 44–46

information, 9

interface, 39

interior gateway protocol,
65

internal router, 65

Internet, 25, 26, 31

IP, 33, 37, 44, 46, 49, 60

IP address, 35

IP address block, 35

IP Options, 39

IPv4, 29–31, 33, 34, 41, 49,
55

IPv6, 29, 30, 33, 34, 49, 55

ISP, 61

iterative, 61

IXP, 64

LAN, 14, 15, 26, 29, 31, 33,
41

layer, 21, 22, 27, 53

lease, 63

link local, 34

little endian, 9

local broadcast, 63

MAC, 29, 33, 39, 44

medium, 11

message, 9, 20

metadata, 18

MF, 43

MSS, 55

MTU, 17, 29, 30, 33, 43, 55

multicast, 30

multiplex, 11, 18

MX, 61

netmask, 34, 35, 41

- network, 14, 19, 26, 36
- network card, 13
- network interface, 15
- network stack, 24
- NP, 64
- NS, 61

- Offset, 43
- offset, 20
- OS, 15
- overhead, 17, 20–22, 26, 43, 48, 55

- packet, 17, 20, 22–24, 26, 37
- packet format, 18, 19, 21, 22, 27
- packet structure, 18
- packet switching, 17
- parallel, 11
- parsing, 18
- path, 19, 65
- payload, 18, 21–23, 43, 46, 48
- PDU, 21–23, 27, 29, 33, 34
- peer-to-peer, 31
- physical layer, 24
- piggyback, 54
- ping, 46
- point-to-point, 13
- poison, 45
- pool, 63
- port, 14, 46, 48, 49, 52
- power, 31
- preamble, 18
- private IP, 41
- privative, 28
- process, 15, 48, 50, 58

- protocol, 25–27, 29
- PSH, 53
- public, 40, 41
- public IP, 34

- quad decimal, 34–36, 42
- quadratic, 13
- query, 61

- recursive, 61, 62
- reliability, 48, 52, 53
- renew, 63
- reply, 61
- request, 61
- reserved IP address, 34
- retire (standard), 28
- retransmission, 48, 54
- RFC, 28
- RIP, 65
- RIR, 28
- root DNS, 60
- router, 15, 16, 20, 37–39, 43, 44, 46
- routing, 14, 15, 33
- routing errors, 40
- routing table, 39, 40, 46, 64, 65
- RST, 51, 58
- RTT, 54

- scan, 45
- segment, 21, 51, 53
- sequence number, 52
- sequencing, 21
- serial, 11
- server, 15, 31, 49
- service, 15
- sign, 9
- sniff, 19

- socket, 50–52
- sshd, 49
- stack, 21–23, 26
- star topology, 14
- stateful, 51
- stream, 20, 26, 33, 53
- subnet, 41, 42
- subnetting, 42
- suit, 25
- supernet, 42
- swimlane diagram, 16
- switch, 14–16, 31, 45
- SYN, 51, 52

- tail, 18
- TCP, 26, 33, 34, 48, 49, 51, 60
- TCP/IP, 26, 29
- text mode, 19
- throughput, 18
- Token Ring, 14
- top-level domain, 60, 61
- topology, 14
- trailer, 18
- transmission window, 55
- transport, 48
- TTL, 39, 40, 46

- UDP, 26, 34, 48, 49, 60
- unicast, 31
- unsigned, 23
- user datagram, 48–50

- virtual, 21, 22

- WAN, 14, 15
- Wi-Fi, 29

- zone, 61